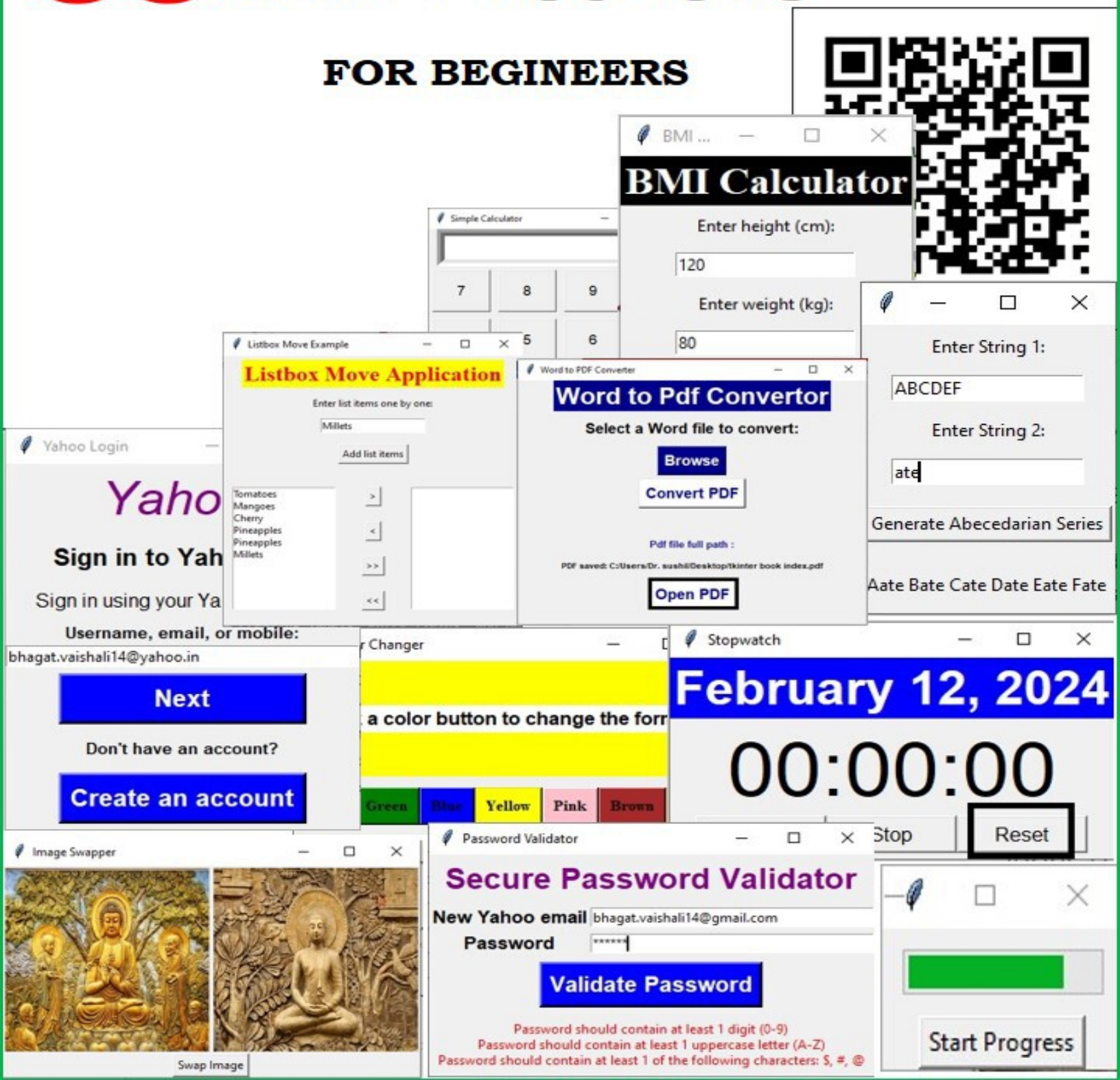
VAISHALI B. BHAGAT

PYTHON

35 TKINTER MINI PROJECTS

FOR BEGINEERS



PYTHON

Tkinter

35

Mini projects

practical guide for Begineers

VAISHALI B. BHAGAT

Cover design by Dr. Sushil B.Naik

ISBN - Paperback:9798883772695

For
My Father

CONTENTS

ACKNOWLEDGMENTS

I would like to express my deep and sincere gratitude to my parents for their love, prayers, caring and sacrifices for educating and preparing for my future.
I am extremely grateful to my husband Dr. Sushil Naik, without whose support I would not have been able to complete this book successfully.

Vaishali B.Bhagat

Simple Tkinter Application to Show a Welcome Message

Project #1

Creating a Tkinter App to Show a Welcome Message

Introduction:

Welcome to our Tkinter tutorial! In this program, we'll explore how to make a simple graphical user interface (GUI) using Python Tkinter. Our aim is to build a program that greets users with a welcome message when they click a button. To do this, we'll be using Tkinter, the most popular library for making GUIs in Python. We'll use two widgets: a Button labeled "Click Me" and a Label to display the text. Initially, the label won't show any text. However, when the button is clicked, it will update to show the message "Welcome to Python Programming world!" in a bold font. Let's dive in and create our first Tkinter app!

Requirements:

Text Editor or IDE: You'll need a text editor to write the code. I've used the Thonny IDE for writing and executing this program. However, you can use any Python IDE or text editor of your choice to run the program.

Python with Tkinter: Ensure that you have Python installed on your system. Tkinter is usually included with Python installations by default, so there's typically no separate installation needed. It's assumed that you have a basic understanding of Python programming language concepts such as functions, variables, and control flow. We won't go deeply into these topics in this book.

Tkinter Widgets: We'll use two Tkinter widgets to design the GUI: Label and Button.

Label widget:

Label widget is GUI element to display static text or image on Tkinter window. They are specially used for displaying information, instruction and non edited text. We can also use them for displaying heading on window. They can display text in different fonts, sizes, and colors. Labels can also display images. They do not accept user input.

Label Widget Parameters:

text: Specifies the text displayed on the label.
bg: Specifies the background color of the label.
fg: Specifies the foreground (text) color of the label.
width: Specifies the width of the label (in characters or pixels).
height: Specifies the height of the label (in characters or pixels).
font: Specifies the font used for the label text.
relief: Specifies the appearance of the label border (e.g., tk.FLAT, tk.SUNKEN, tk.RAISED). The default value is FLAT.
anchor: Specifies the exact position of the text for the label content (e.g., tk.N, tk.W, tk.CENTER)

Button Widget:

Buttons are interactive elements that users can click to perform actions or triggers events. They are used to create clickable buttons on Tkinter window.

Button Widget Parameters:

text: Specifies the text displayed on the button.
Command: Sets to the function call when function is called
bg: Specifies the background color of the button.
fg: Specifies the foreground (text) color of the button.
width: Specifies the width of the button(in characters or pixels).
height: Specifies the height of the button(in characters or pixels).
font: Specifies the font used for the button text.
relief: Specifies the appearance of the button border (e.g., tk.FLAT, tk.SUNKEN, tk.RAISED). The default value is FLAT.
anchor: Specifies the exact position of the text for the label content (e.g., tk.N, tk.W, tk.CENTER)
Label and button widgets have more parameters than those explained above. We will discuss them in detail later.

To create a button and label widget using Tkinter in Python, you can follow these steps:

Create a Label Widget

1. First, you need to import the Tkinter module. Typically, you import it as tk for easier reference.

   ```
   import tkinter as tk
   ```
2. Create the main window using the Tk() constructor.

   ```
   root = tk.Tk()
   ```
3. Create a label widget using the Label() constructor. Specify the parent widget (in this case, the main window root), along with any optional parameters like text or font.

   ```
   label = tk.Label(root, text="Hello, World!")
   ```
4. Use the pack() method to pack the label widget into the main window. This makes it visible within the window.

   ```
   label.pack()
   ```

Create a Button Widget

1. Similar to creating a label widget, create a button widget using the Button() constructor. Specify the parent widget (root), along with text and any other optional parameters.

```
button = tk.Button(root, text="Click Me")
```

2. Pack the button widget into the main window using the pack() method.

```
button.pack()
```

3. Finally, start the Tkinter event loop using the mainloop() method on the main window. This method listens for events (e.g., button clicks) and updates the GUI accordingly.

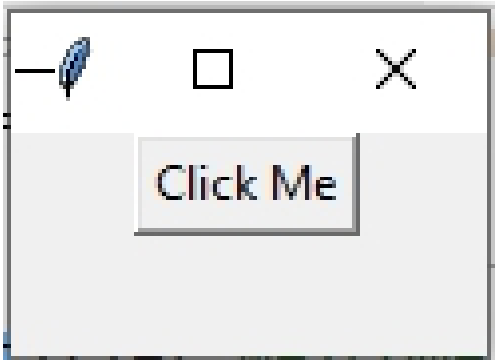
root.mainloop()

Solutions:

```
import tkinter as tk
# Create the main Tkinter window
root = tk.Tk()
# Set the title of the window
root.title("Welcome Message !!!")
# Define a bold font for the label
bold_font = ("Arial", 15, "bold")
# Creating a button with a command to update the label text
button = tk.Button(root, text="Click Me", command=lambda:
welcome_label.config(text="Welcome to Python Programming
world!"))
button.pack()  # Place the button in the window
# Creating a label with an empty text and the specified bold font
welcome_label = tk.Label(root, text="", font=bold_font)
welcome_label.pack()  # Place the label in the window
# Start the Tkinter event loop
root.mainloop()
```

Output:

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.



After clicking the ‘Click Me’ button, following window is displayed on your screen.

Simple Grade Display GUI

Project #2

Design a Simple GUI using Tkinter in Python to display grades. The grades assigned to the students of a certain engineering college are O, A, B, C, F. Write a program that prompts the users to enter a character O, A, B, C, F and display their grades as Outstanding, Very Good, Good, Average, and Fail respectively using if-elif-else.

Introduction:

In this program, we'll design a simple Graphical User Interface (GUI) using Tkinter to display grades. As you know, Tkinter is a popular Python library for building desktop applications. Our goal is to design an application that allows users to input their grades and displays the corresponding grade descriptions.

In many educational institutions, grades are represented by letters such as O (Outstanding), A (Very Good), B (Good), C (Average), and F (Fail). Our program will prompt users to enter one of these characters, and based on their input, it will provide the corresponding grade description.

We'll create simple graphical user interface where users can easily input their grades and view the corresponding descriptions. We'll utilize the if-elif-else statement in Python to handle the logic of determining the grade description based on the user's input.

By the end of this program, you'll be able to build a simple GUI application with Tkinter and applying conditional logic to provide useful feedback to users based on their input. Let's get started!

Requirement:

We will use the same widgets (Labels and Buttons) that were discussed in our previous programs. Additionally, we'll introduce a

new widget: the Entry widget. We'll discuss its functionality and usage as we proceed with the program.

Entry widget:

The Entry widget in Tkinter is used to create a single-line text entry field where users can input text or numbers. It allows users to type text or numbers directly into the GUI application. The Entry widget is commonly used in Tkinter applications for accepting user input, such as usernames, passwords, search queries, or any other type of textual data entry.

Tkinter Entry Widget Properties:

Text Variable (textvariable): Associates a Tkinter variable with the Entry widget, allowing you to set or retrieve the text content dynamically.
Width (width): Sets the width of the entry field in characters.
Background (bg): Sets the background color of the entry field.
Foreground (fg): Sets the text color of the entry field.
Font (font): Sets the font style and size for the text in the entry field.
Border Width (borderwidth): Sets the width of the border around the entry field.
Relief: Determines the appearance of the border. It can be set to "flat", "raised", "sunken", "groove", or "ridge".
State: Sets the state of the entry field. It can be set to "normal", "readonly", or "disabled".
Insert Background (insertbackground): Sets the color of the insertion cursor (the blinking vertical bar).
xscrollcommand: Attaches a scrollbar for horizontal scrolling.
insertwidth: Defines the insertion cursor width (default: 3 pixels).
readonlybackground: Sets the background color for read-only mode.
invalidcommand: Triggers a function when invalid input occurs.

Solutions:

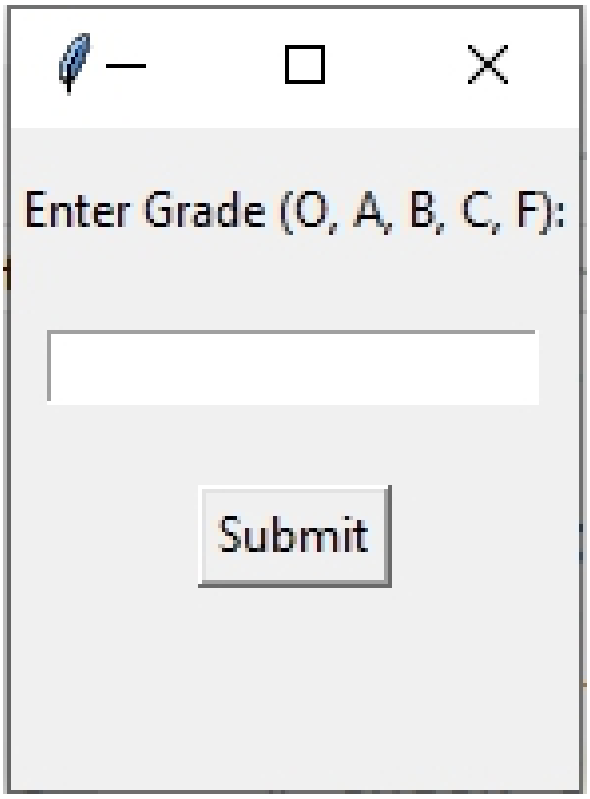
```
import tkinter as tk

def display_grade():
```

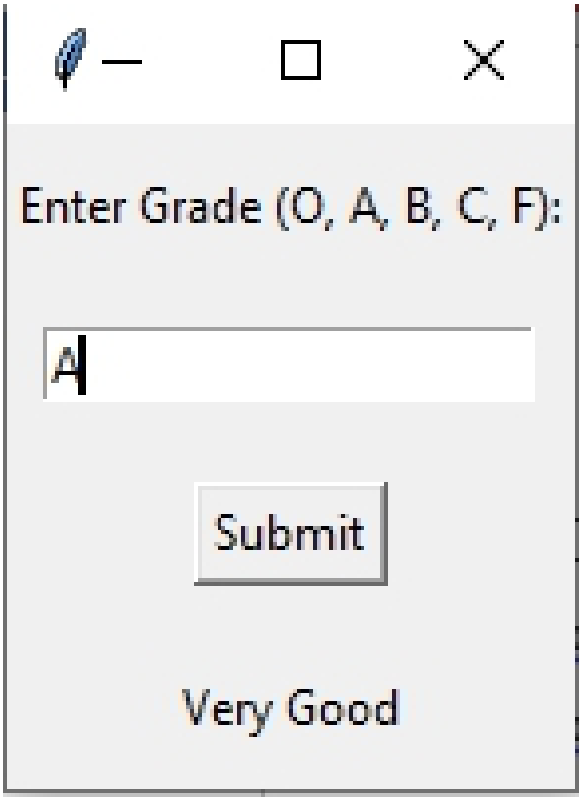
```
    # Get user input from the entry widget and convert it to uppercase
    user_input = entry.get().upper()
    # Check the user input and display the corresponding grade
description
    if user_input == 'O':
        result_label.config(text="Outstanding")
    elif user_input == 'A':
        result_label.config(text="Very Good")
    elif user_input == 'B':
        result_label.config(text="Good")
    elif user_input == 'C':
        result_label.config(text="Average")
    elif user_input == 'F':
        result_label.config(text="Fail")
    else:
        # Display an error message for invalid grades
        result_label.config(text="Invalid Grade")

# Create the main window
root = tk.Tk()
root.title("Grade Display Program")

# Create and place widgets

# Label to instruct the user to enter a grade
label = tk.Label(root, text="Enter Grade (O, A, B, C, F):")
label.pack(pady=10)

# Entry widget for user to input a grade
entry = tk.Entry(root)
entry.pack(pady=10)

# Button to submit the grade and trigger the display_grade function
submit_button = tk.Button(root, text="Submit",
command=display_grade)
submit_button.pack(pady=10)

# Label to display the result or error message based on the entered
grade
result_label = tk.Label(root, text="")
```

```
result_label.pack(pady=10)

# Start the Tkinter event loop
root.mainloop()
```

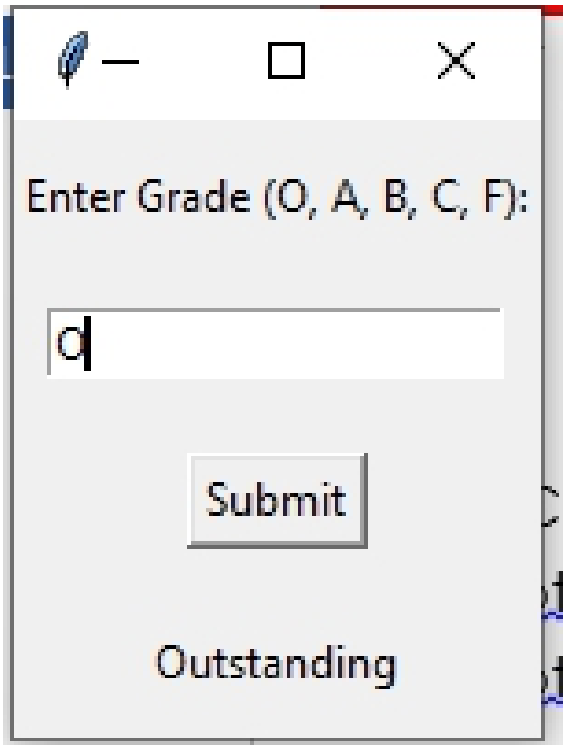
Output:

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.





After entering grade A in the entry field, the window displays the grade description, indicating that A is very good.



After entering grade O in the entry field, the window displays the grade description, indicating that O is Outstanding

Simple Abecedarian Series Generator Application

Project #3

Abecedarian refers to a series or list in which the elements appear in alphabetical order. Design a simple Python GUI form using Tkinter to generate Abecedarian series.

Introduction:

In this program, we will design a simple GUI form to generate Abecedarian series. An Abecedarian series, as the name suggests, arranges elements in alphabetical order. Our goal is to create a simple graphical user interface (GUI) that takes two strings as input and generates an Abecedarian series based on these inputs.

For example, if str1 is "ABCDEF" and str2 is "ate", the program will output an Abecedarian series starting with "Aate", followed by "Bate", "Cate", "Date", "Eate", and "Fate".

This program will provide a user-friendly interface where users can input their desired strings and receive the corresponding Abecedarian series as output. Let's dive into the code and explore how this program works!

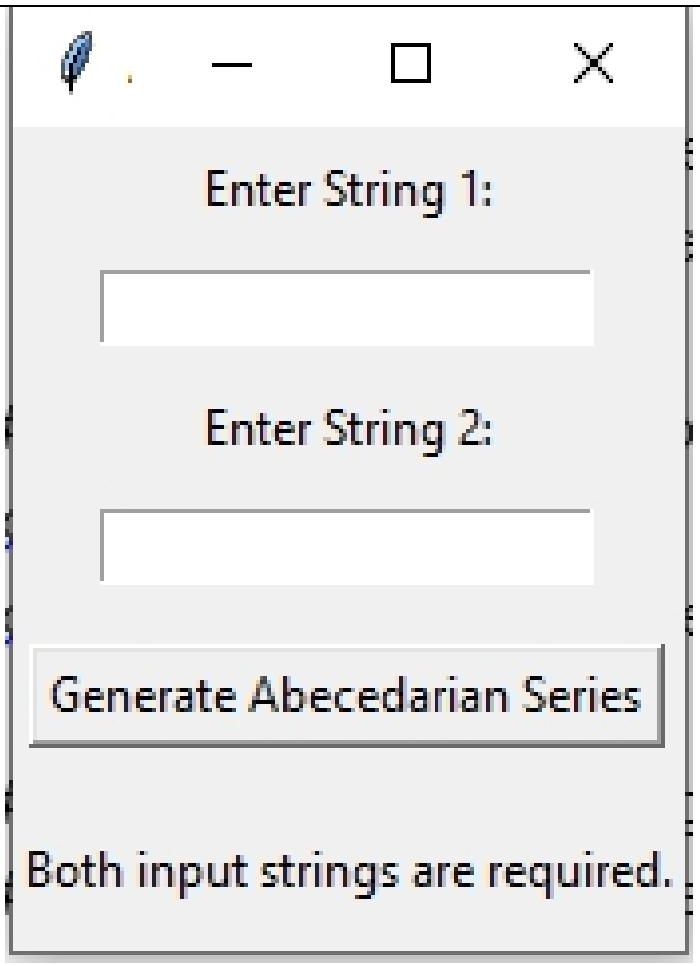
Solutions:

```
import tkinter as tk

def generate_abecedarian_series():
    # Get user input strings from entry widgets
    str1 = entry_str1.get()
    str2 = entry_str2.get()

    # Check if both input strings are non-empty
```

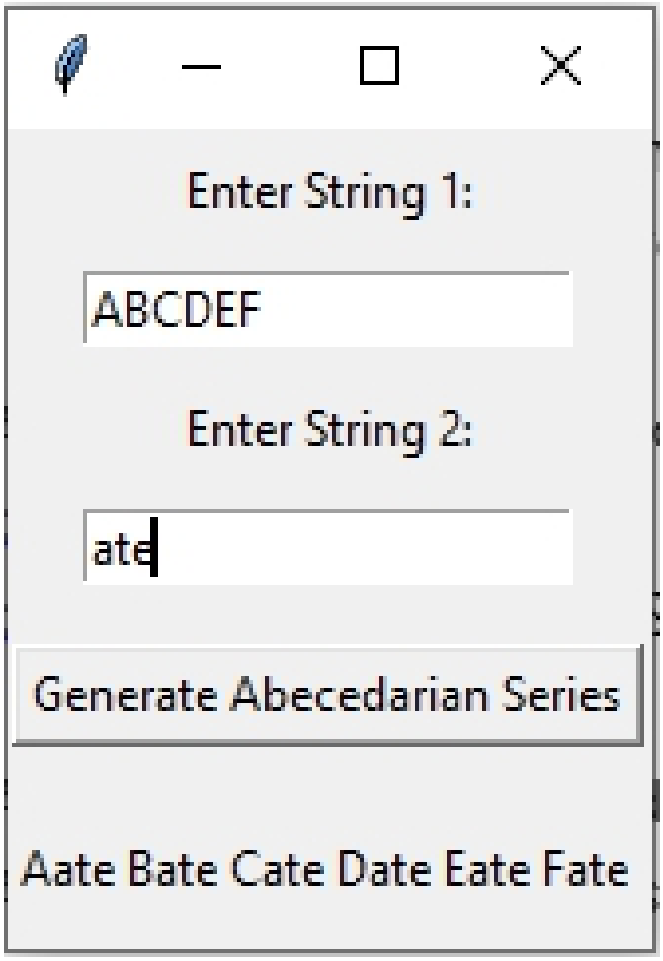
```
    if str1 and str2:
        result = ""
        # Iterate through each character in str1
        for char in str1:
            # Concatenate each character of str1 with str2 and add a
space
            result += char + str2 + " "
            # Update the result_label with the current result
            result_label.config(text=result)
    else:
        # Display an error message if either input string is empty
        result_label.config(text="Both input strings are required.")

# Create the main window
root = tk.Tk()
root.title("Abecedarian Series Generator")

# Create and place widgets
# Label to instruct the user to enter a first string
label_str1 = tk.Label(root, text="Enter String 1:")
label_str1.pack(pady=5)
# Entry widget for user to input a first string
entry_str1 = tk.Entry(root)
entry_str1.pack(pady=5)
# Label to instruct the user to enter a second string
label_str2 = tk.Label(root, text="Enter String 2:")
label_str2.pack(pady=5)
# Entry widget for user to input a second string
entry_str2 = tk.Entry(root)
entry_str2.pack(pady=5)
# Button to generate Abecedarian Series
generate_button = tk.Button(root, text="Generate Abecedarian
Series", command=generate_abecedarian_series)
generate_button.pack(pady=10)
# Label to display the result
result_label = tk.Label(root, text="")
result_label.pack(pady=10)

# Start the Tkinter event loop
```

root.mainloop()

Output:
After running the above code, the following GUI will be displayed on your screen. If the user does not enter both String 1 and String 2, a message saying 'Both input strings are required' will be displayed on the main window.



If the user enters String 1 and String 2 properly, the following window will display the appropriate output on the main window.



Simple Character Occurrence Counter Application

Project #4

Design a simple GUI form using Tkinter to count and print the number of occurrences of each character in a dictionary.

Introduction:

In this Program, we'll make a simple GUI form to count characters. Our goal is to create a form where you can type in some text, click a button, and see how many times each letter appears. Counting characters is important for many text tasks. With Tkinter, we'll make a form that's easy to use. You'll be able to type in your text and get the count with just one click. We'll use Entry widget for typing, Button widget for counting, and Python dictionaries to keep track of the counts. If you've followed our previous programs, you'll know about labels, entry fields, and buttons already. Now, let's build this helpful form together!

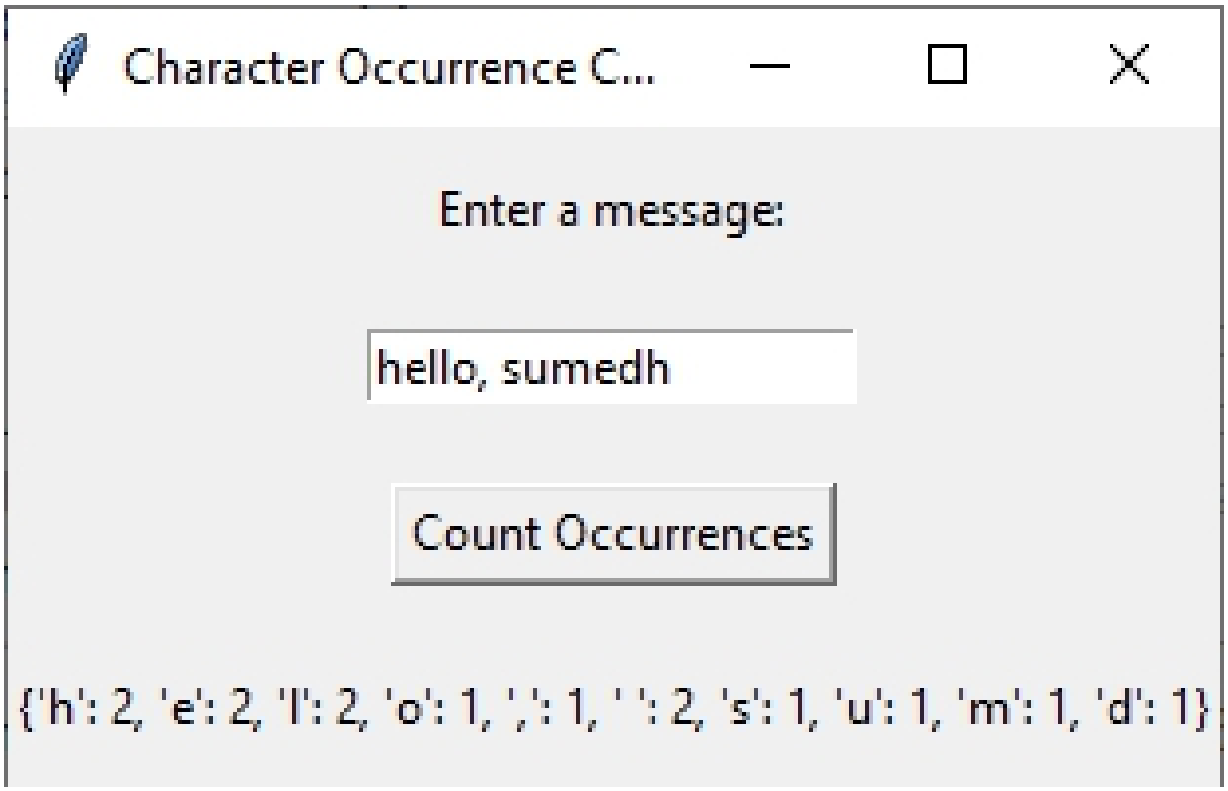
Solutions:

```
import tkinter as tk

def count_occurrences():
    # Get user input from the entry widget
    user_input = entry.get()

    # Count occurrences of each character in the input string
    char_count = {} # Initialize an empty dictionary to store the count of occurrences for each character
    # Iterate through each character in the user_input string
    for char in user_input:
        # Check if the character is already in the dictionary, if not, default to 0
        current_count = char_count.get(char, 0)
        # Increment the count for the current character
```

```
            char_count[char] = current_count + 1

        # Display the result in the label
        result_label.config(text=str(char_count))

# Create the main window
root = tk.Tk()
root.title("Character Occurrence Counter")

# Create and place widgets
label = tk.Label(root, text="Enter a message:")
label.pack(pady=10)

entry = tk.Entry(root)
entry.pack(pady=10)

count_button = tk.Button(root, text="Count Occurrences",
command=count_occurrences)
count_button.pack(pady=10)

result_label = tk.Label(root, text="")
result_label.pack(pady=10)

# Start the Tkinter event loop
root.mainloop()
```

Output :

After running above code, following GUI will be displayed on your screen.



This program creates a simple GUI form using Tkinter. It consists of a label prompting the user to enter a string, an entry field to input the string, and a button to trigger the character counting process. The count_occurrences () function is called when the button is clicked. It retrieves the input string from the entry field, iterates over each character in the string, and counts the occurrences of each character using a dictionary. Finally, it prints the dictionary containing the character counts.

Simple GCD and LCM Calculator

Project#5

Create a graphical user interface (GUI) application using Tkinter in Python for calculating both the Greatest Common Divisor (GCD) and the Least Common Multiple (LCM) of two given integers entered by the user.

Introduction:

Welcome to our Tkinter application! Today, we'll be building a graphical user interface (GUI) to calculate both the Greatest Common Divisor (GCD) and the Least Common Multiple (LCM) of two integers. These calculations are common in mathematics and are often needed in various applications. With Tkinter, we'll create a simple and easy-to-use interface. Users can enter two integers, and with a click of a button, they'll get the calculated GCD and LCM displayed in a messagebox. This application will be helpful for anyone who needs to find the GCD and LCM quickly without using a calculator or performing manual calculations. So, let's dive in and create this handy tool together!

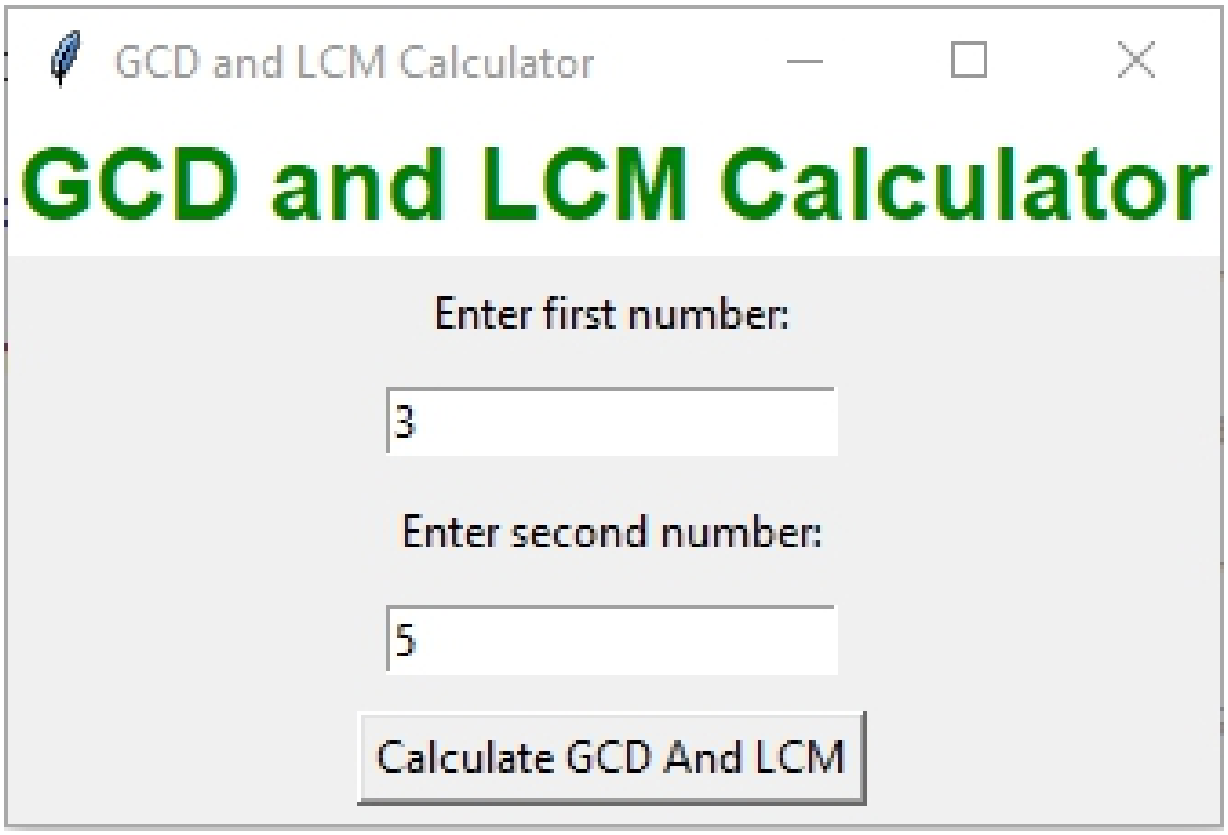
Solution:

```
import tkinter as tk
from tkinter import messagebox
def gcd(a, b):
    # Continuously loop until b becomes zero
    while b != 0:
        # Store the current value of b in a temporary variable c
        c = b
        # Update the value of b to be the remainder of a divided by the current value of b
        b = a % b
        # Assign the original value of b (stored in c) to a, effectively swapping the values
        a = c
```

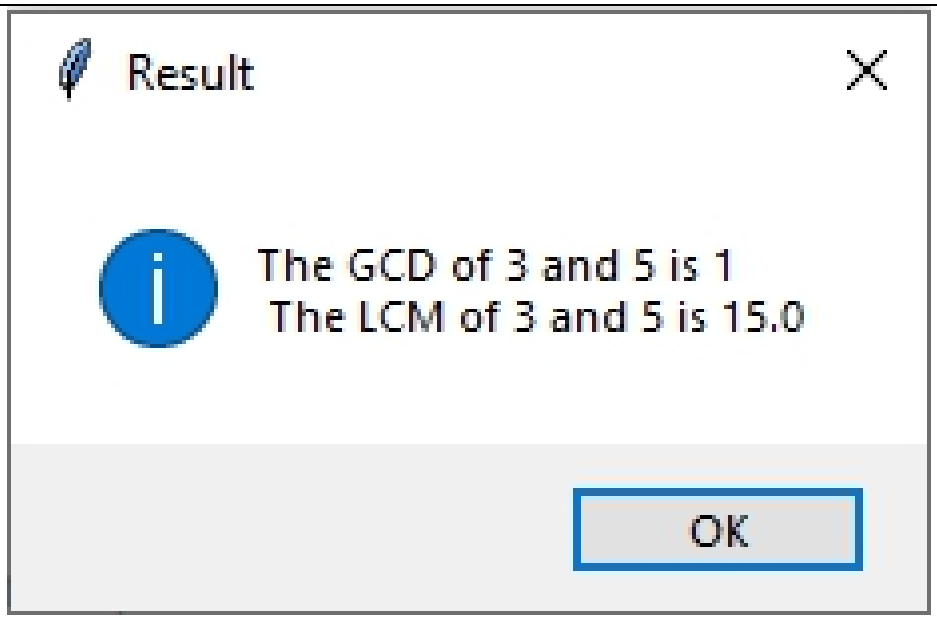
```
    # When b becomes zero, return the current value of a, which is the GCD
    return a
def calculate():
    try:
        # Attempt to retrieve the values entered in the entry fields and convert them to integers
        num1 = int(entry_num1.get())
        num2 = int(entry_num2.get())

        # Calculate the GCD using the previously defined gcd function
        result = gcd(num1, num2)

        # Calculate the LCM using the formula: LCM = (num1 * num2) / GCD
        lcm = (num1 * num2) / result

        # Display the result in a message box, including both the GCD and LCM
        messagebox.showinfo("Result", f"The GCD of {num1} and {num2} is {result}\n "f"The LCM of {num1} and {num2} is {lcm}")
    except ValueError:
        # If the user entered invalid input (e.g., non-integer values), display an error message
        messagebox.showerror("Error", "Please enter valid integers for both numbers.")
# Creating the main tkinter window
root = tk.Tk()
root.title("GCD and LCM Calculator")
# Setting font for title label
bold_font = ("Arial", 20, "bold")
# Creating label for title
title_label = tk.Label(root, text="GCD and LCM Calculator", bg="white", fg="Green", font=bold_font)
title_label.pack()

# Creating labels and entry fields
label_num1 = tk.Label(root, text="Enter first number:")
label_num1.pack(padx=5, pady=5)
```

```
entry_num1 = tk.Entry(root)
entry_num1.pack(padx=5, pady=5)

label_num2 = tk.Label(root, text="Enter second number:")
label_num2.pack(padx=5, pady=5)

entry_num2 = tk.Entry(root)
entry_num2.pack(padx=5, pady=5)

# Creating a button to trigger calculation
calculate_button = tk.Button(root, text="Calculate GCD And LCM",
command=calculate)
calculate_button.pack(padx=5, pady=5)

# Running the main tkinter event loop
root.mainloop()
```

Output:
After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.



After clicking on the 'Calculate GCD and LCM' button, the program displays a messagebox showing the calculated GCD (Greatest Common Divisor) and LCM (Least Common Multiple) of the input numbers.



Battery Level Monitor and Status Alert GUI

Project #6

Design a graphical user interface (GUI) application using Tkinter in Python to monitor battery levels and display alerts when the battery level is low.

Introduction:

In this program, we will design a simple GUI to monitor battery levels and receive alert message when the battery level is low. Our goal is to develop an application that monitors battery levels and issues alerts when the battery level is low, all triggered by the click of a button. Battery monitoring and alerting are essential functionalities, especially in systems that rely on battery power, such as laptops, smartphones, and IoT devices. With Tkinter, we'll create an intuitive interface where users can easily check the current battery level and receive timely alerts when the battery is running low. We wil use different type of message boxes to keep you inform about your device power status.

Requirement:

We will use button widget and messagebox to design this simple GUI form. Lets talk about different types of messagebox and use them in code.

Messagebox :

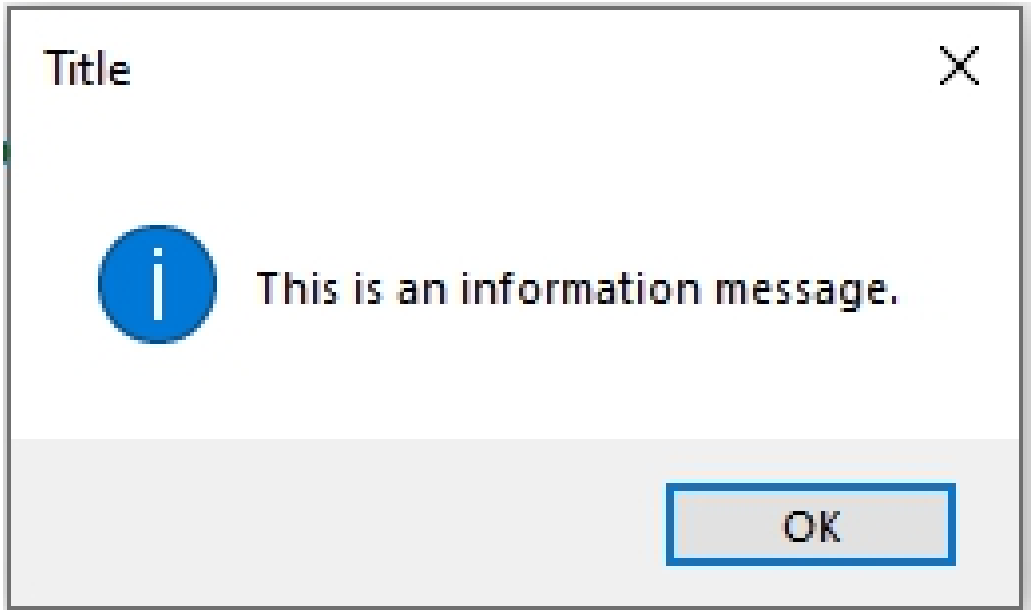
Tkinter provides a convenient module called messagebox to create different types of message boxes within your GUI applications. These boxes are useful for displaying information, warnings, errors, and even asking questions to users.

Informative Boxes:

showinfo(title, message, **options): Displays a message box with an information icon.

showwarning(title, message, **options): Displays a message box with a warning icon.
showerror(title, message, **options): Displays a message box with an error icon.
Asking Questions:
askquestion(title, message, **options): Displays a message box with "Yes" and "No" buttons. Returns either 'yes' or 'no'.
askokcancel(title, message, **options): Displays a message box with "OK" and "Cancel" buttons. Returns either 'ok' or 'cancel'.
askyesno(title, message, **options): Displays a message box with "Yes" and "No" buttons. Returns either True for "Yes" or False for "No".
askretrycancel(title, message, **options): Displays a message box with "Retry", "Cancel", and "Ignore" buttons. Returns either 'retry', 'cancel', or 'ignore'.
Common Options:
default: Sets the default button (e.g., 'ok', 'yes').
icon: Specifies the icon to display (e.g., 'info', 'warning', 'error').
parent: Defines the parent window of the message box.
How to create messagebox?
First, import the messagebox module from Tkinter.
from tkinter import messagebox
Show Information Message Box: Use the showinfo() function to display an information message box.

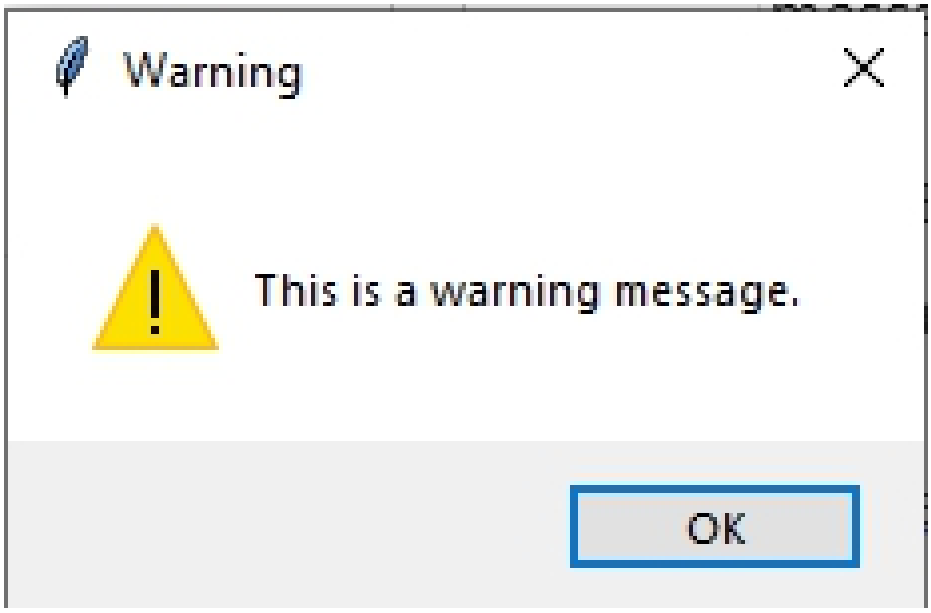
messagebox.showinfo("Title", "This is an information message.")



Show Warning Message Box: Use the showwarning() function to display a warning message box. Here's how you can use it:

Code:

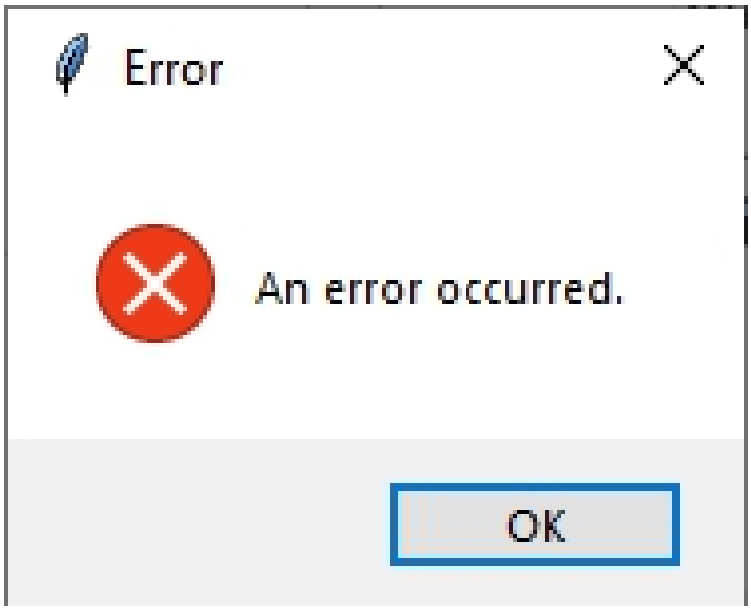
```
messagebox.showwarning("Warning", "This is a warning message.")
```



Show Error Message Box: Use the showerror() function to display an error message box. Here's how you can use it:

Code:

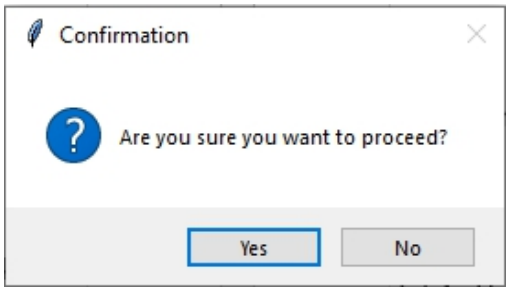
```
messagebox.showerror("Error", "An error occurred.")
```



Ask for Confirmation: Use the askquestion() function to ask for user confirmation with "Yes" and "No" buttons. Here's how you can use it

Code:

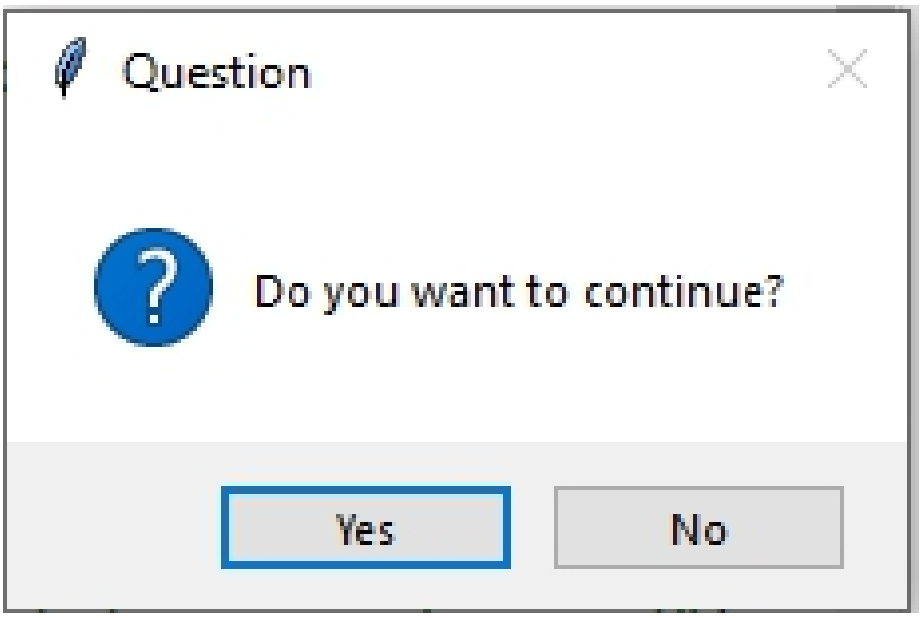
```
messagebox.askquestion("Confirmation", "Are you sure you want to proceed?")
```



Ask for yes or no : Use the askyesno() function to ask for a yes or no response. Here's how you can use it:

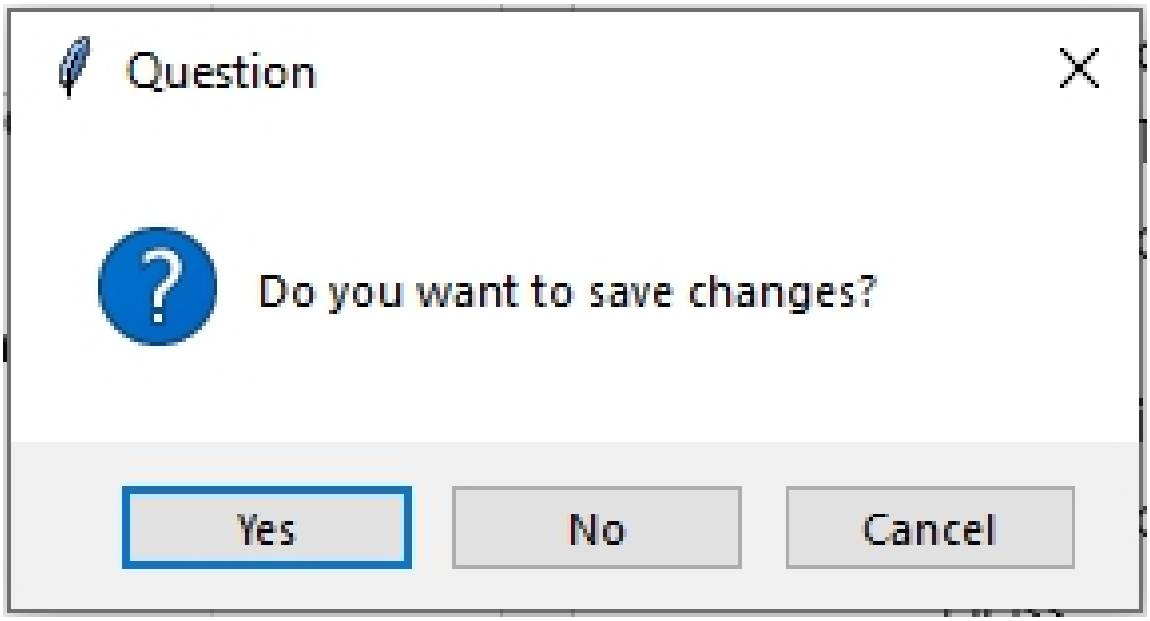
Code :

```
result = messagebox.askyesno("Question", "Do you want to continue?")
if result:
    # User clicked Yes
    pass
else:
    # User clicked No
    Pass
```



Ask for Yes, No, or Cancel: Use the askyesnocancel() function to ask for a yes, no, or cancel response. Here's how you can use it
Code:

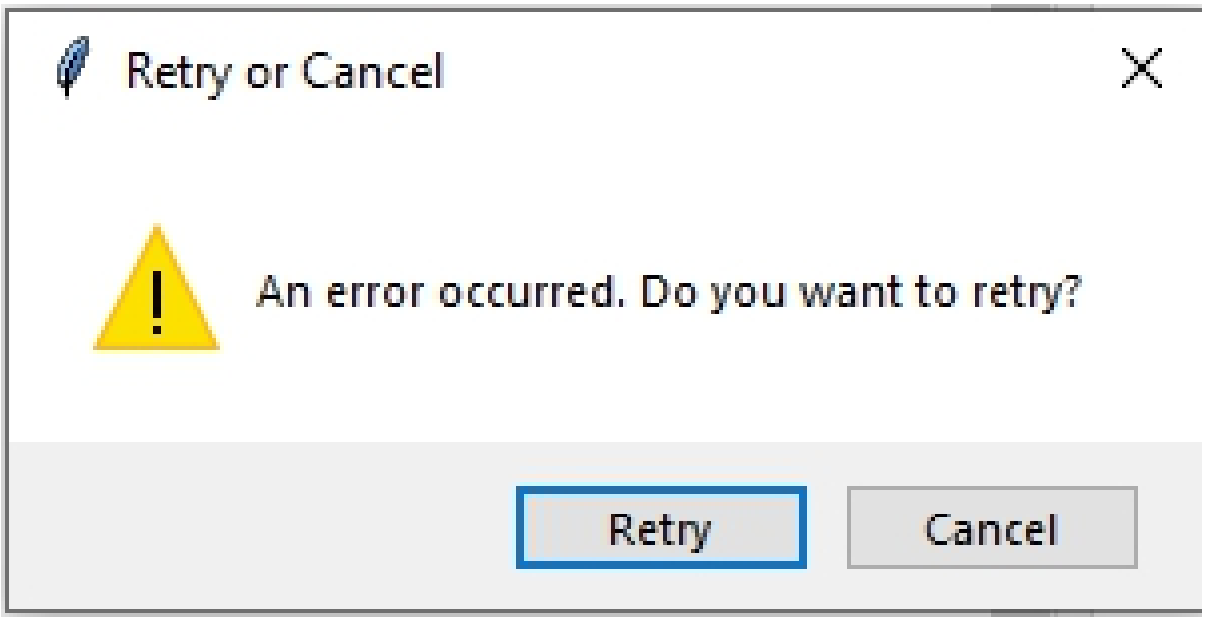
```
result = messagebox.askyesnocancel("Question", "Do you want to save changes?")
if result is True:
    # User clicked Yes
    pass
elif result is False:
    # User clicked No
    pass
else:
    # User clicked Cancel
    Pass
```



Show askretrycancel Messagebox : Use askretrycancel() function to ask the user whether to retry an operation or cancel it. It returns True if the user chooses to retry and False if the user chooses to cancel. Here's how you can use it:

Code:

```
    result = messagebox.askretrycancel("Retry or Cancel", "An error occurred. Do you want to retry?")
    if result:
        print("User chose to retry.")
else:
        print("User chose to cancel.")
```



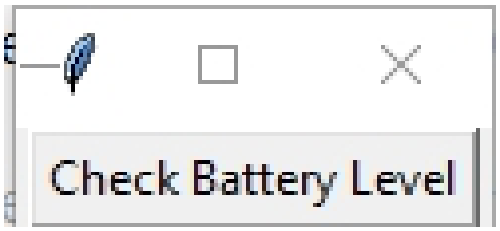
Solutions:

```
import tkinter as tk
from tkinter import messagebox

# Function to check the battery level and display appropriate
messages
def check_battery_level():
    # Dummy battery level for testing
    battery_level = 10

    # Check if battery level is low
    if battery_level < 20:
        # Display warning message about low battery
        messagebox.showwarning("Low Battery", "Battery level is low!
Please plug in")

        # Ask user if they want to continue
        response = messagebox.askyesno("Yes/No", "Do you want to
continue?")

        # If user chooses No or closes the messagebox, abort the
operation
        if not response:
            messagebox.showinfo("Info", "Operation aborted.")
```

```
            return

    # If user chooses Yes, or battery level is above 20%, continue the
operation
    messagebox.showinfo("Info", "Operation continues...")

# Create the main Tkinter window
root = tk.Tk()
root.title("Battery Level Alert")

# Create a button to trigger battery level checking
check_button = tk.Button(root, text="Check Battery Level",
command=check_battery_level)
check_button.pack()

# Start the Tkinter event loop
root.mainloop()
```

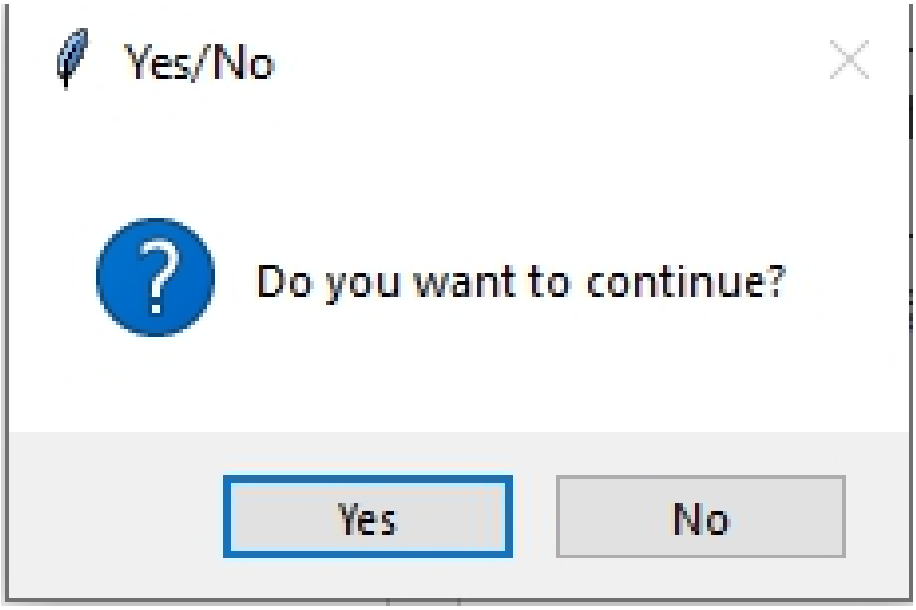
Output :

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.

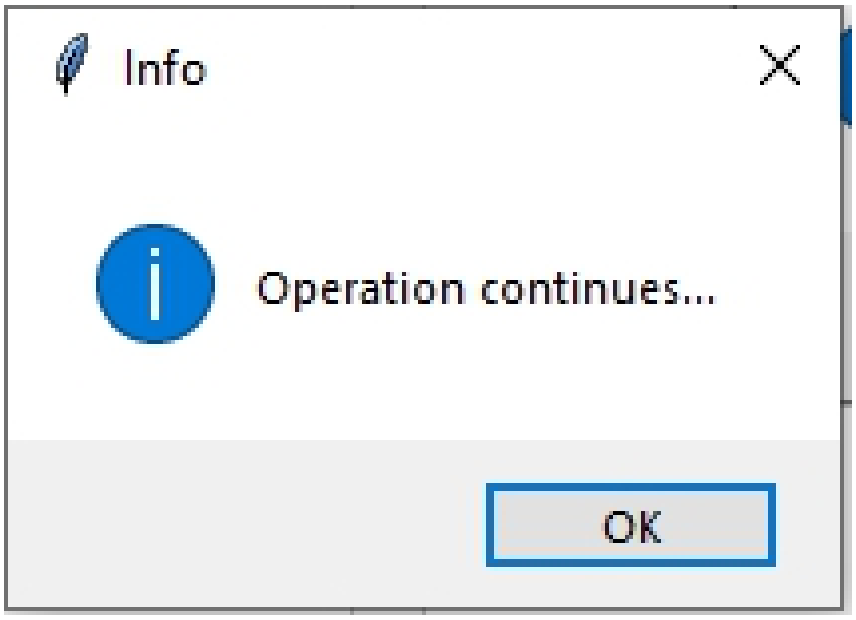


When user click on Check Battery Level button, following Warning message is displayed indicating that low battery and ask user to plug in device

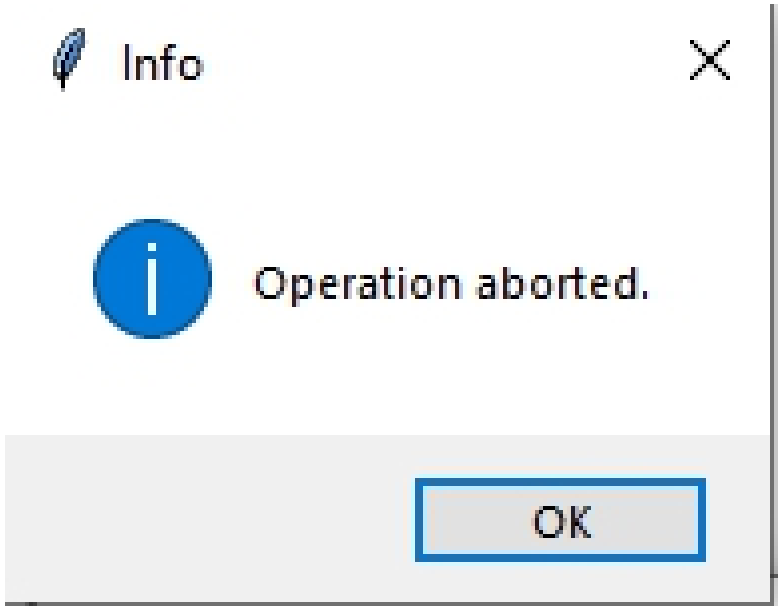
When user click on OK button, following message is displayed



If user click on Yes button, following information message is shown.



If user click on NO button, following information message is displayed



Simple Color Changer Application

Project #7

Design and implement a graphical user interface (GUI) application using Tkinter in Python that allows users to change the background color of a form dynamically by clicking on color buttons.

Introduction:

In this problem, we'll design and implement a Graphical User Interface (GUI) application using Tkinter in Python. Our objective is to create an application that allows users to dynamically change the background color of a form by clicking on color buttons.
Dynamic background color changing is a common feature in GUI applications. It provide users with customization options to personalize their interface experience. With Tkinter, we'll build an intuitive interface where users can interact with color buttons to modify the background color of the form in real-time.

Solutions:

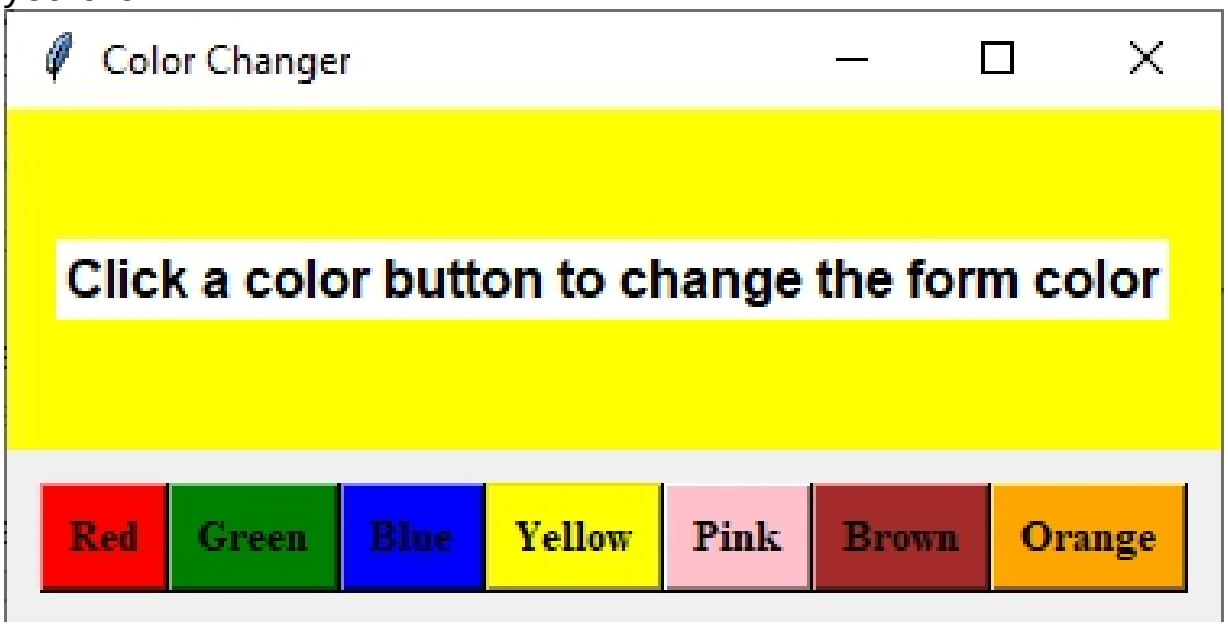
```
import tkinter as tk
# Create the main window
root = tk.Tk()
root.title("Color Changer")
# Define a bold font style and font name for color_label
bold_font = ("Arial", 12, "bold")
# Create a label to provide instructions
color_label = tk.Label(root, text="Click a color button to change the
form color",bg = 'white',font = bold_font)
color_label.pack(pady=40)
# Create a frame to contain color buttons
color_frame = tk.Frame(root, padx=10, pady=10)
color_frame.pack()
# Define a bold font style
```

```
bold_font = ("Times new Roman", 10, "bold")
# Create color buttons, each changing the background color when clicked
redButton = tk.Button(color_frame, text='Red', bg='red', padx=5, pady=5, command=lambda: root.config(bg='red'),font = bold_font)
greenButton = tk.Button(color_frame, text='Green', bg='green', padx=5, pady=5, command=lambda: root.config(bg='green'),font = bold_font)
blueButton = tk.Button(color_frame, text='Blue', bg='blue', padx=5, pady=5, command=lambda: root.config(bg='blue'),font = bold_font)
yellowButton = tk.Button(color_frame, text='Yellow', bg='yellow', padx=5, pady=5, command=lambda: root.config(bg='yellow'),font = bold_font)
pinkButton = tk.Button(color_frame, text='Pink', bg='pink', padx=5, pady=5, command=lambda: root.config(bg='pink'),font = bold_font)
brownButton = tk.Button(color_frame, text='Brown', bg='brown', padx=5, pady=5, command=lambda: root.config(bg='brown'),font = bold_font)
orangeButton = tk.Button(color_frame, text='Orange', bg='orange', padx=5, pady=5, command=lambda: root.config(bg='orange'),font = bold_font)
# Pack buttons into the frame, placing them in one line horizontally
redButton.pack(side='left')
greenButton.pack(side='left')
blueButton.pack(side='left')
yellowButton.pack(side='left')
pinkButton.pack(side='left')
brownButton.pack(side='left')
orangeButton.pack(side='left')
# Start the Tkinter event loop
root.mainloop()
```

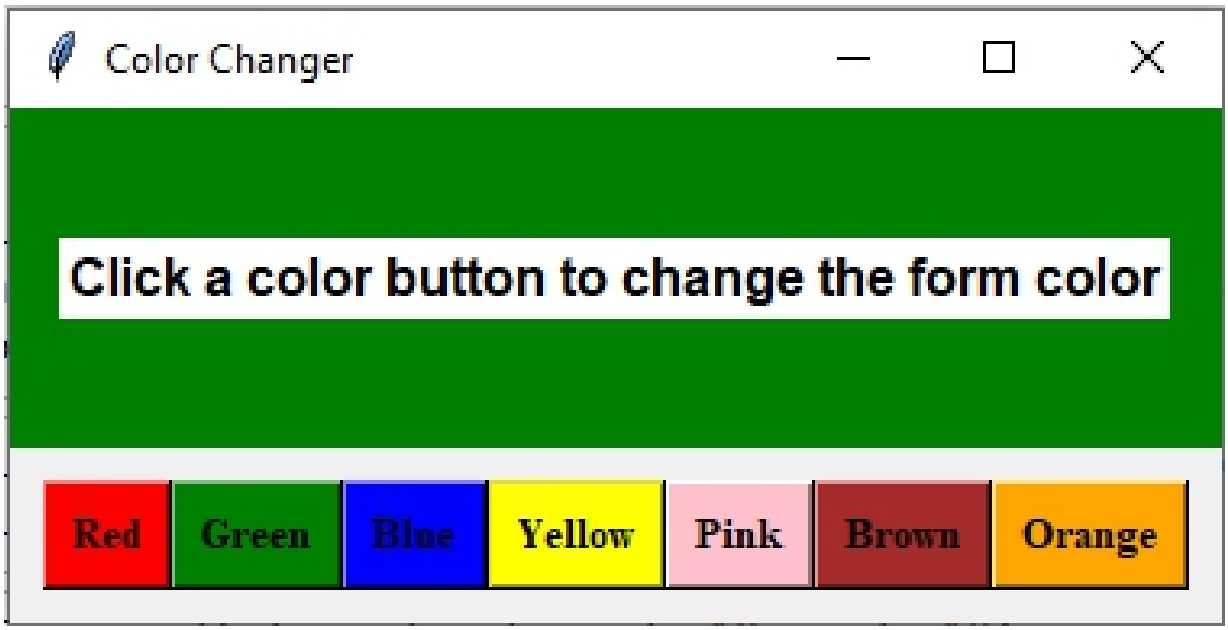
Output:

After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This GUI window serves as the interface for the Dynamic Background Color Changer application. Here's what you'll see:

Title: Positioned at the top of the window, you'll find a title reading "Color Changer". This title indicates the purpose of the application.
Label: Beneath the form title, there is an instructional label that reads, "Click a color button to change the form color."
Color Buttons: Below the title, there are seven color buttons: Red, Green, Blue, Yellow, Pink, Brown, and Orange. These buttons allow you to select the desired background color for the form.
Form: The main area of the window serves as the form, where the background color dynamically changes according to the color button you click.



When the user clicks on the Yellow button, the background of the window changes to yellow, as shown in the output.



After clicking on the Green button, the window's background changes to green, as demonstrated in the output.

Secure Download Manager

Project #8

Develop a Graphical User Interface (GUI) application using Tkinter in Python to facilitate the downloading of files from the internet.

Introduction:

Downloading files from the internet is something many of us do regularly, but it's important to be cautious because there can be risks like viruses or malware hiding in those files. With Tkinter, we'll create a simple program that warns users about these risks before they start downloading. We'll use buttons and message boxes to make it easy to understand and interact with. This program will help users make informed decisions about downloading files, keeping their devices safe from potential threats. Let's work together to build this user-friendly and security-conscious program!

Requirement:

In our previous program, we learned about different types of message boxes and how they work. In this program, we'll use Tkinter's showinfo and showwarning message boxes to warn users about the security risks of downloading files from the internet. We'll also add a button to start the download process. When users click the download button, the program will give them helpful messages about the security risks before starting the download. This way, users can make smart decisions about their online safety.

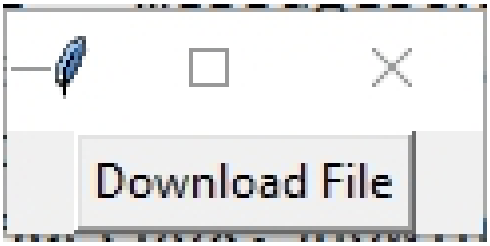
Solutions:

```
import tkinter as tk
from tkinter import messagebox
# Function to show the warning message before downloading the file
def show_download_warning():
    # Prompt a warning message using messagebox
    messagebox.showwarning( "Warning", " Suspicious File is downloading .....")
```

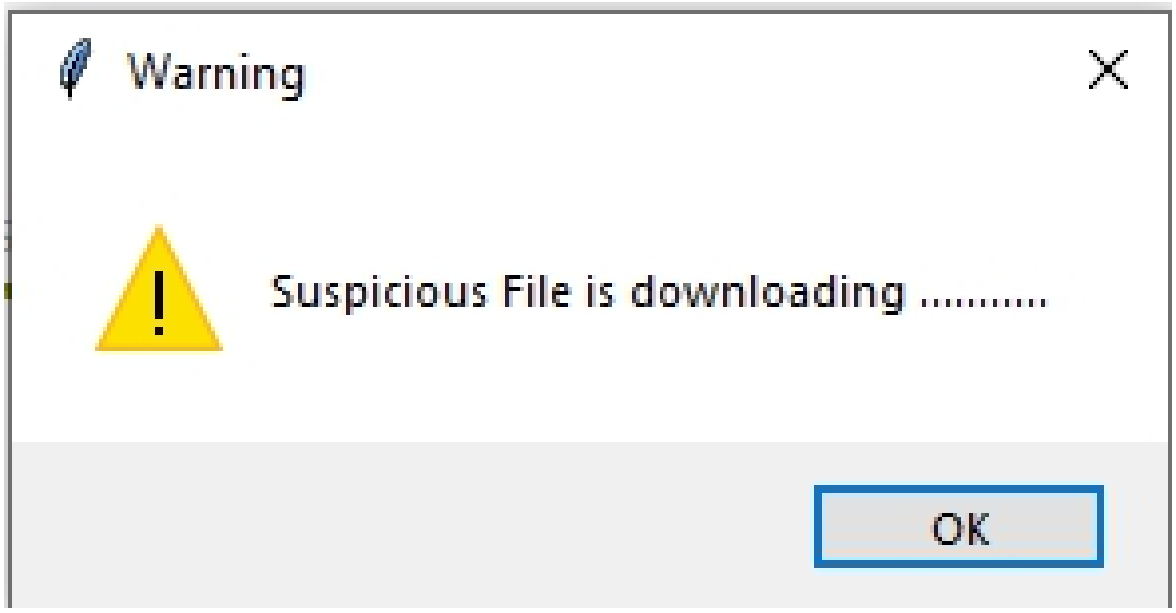
```
    response = messagebox.askokcancel("Virus Detected",
"Downloading files from the internet can be unsafe. Do you want to
continue?")
    # If the user chooses to continue, continue the operation
    if response:
        messagebox.showinfo("Download", "File download complete!")
    # If the user chooses to cancel, cancel the operation
    else:
        messagebox.showinfo("Cancelled", "File download
cancelled.")

# Create the main Tkinter window
root = tk.Tk()
root.title("File Download")
# Create a button to trigger the download warning message
download_button = tk.Button(root, text="Download File",
command=show_download_warning)
download_button.pack() #pack the button into window
# Start the tkinter event loop
root.mainloop()
```

Output:

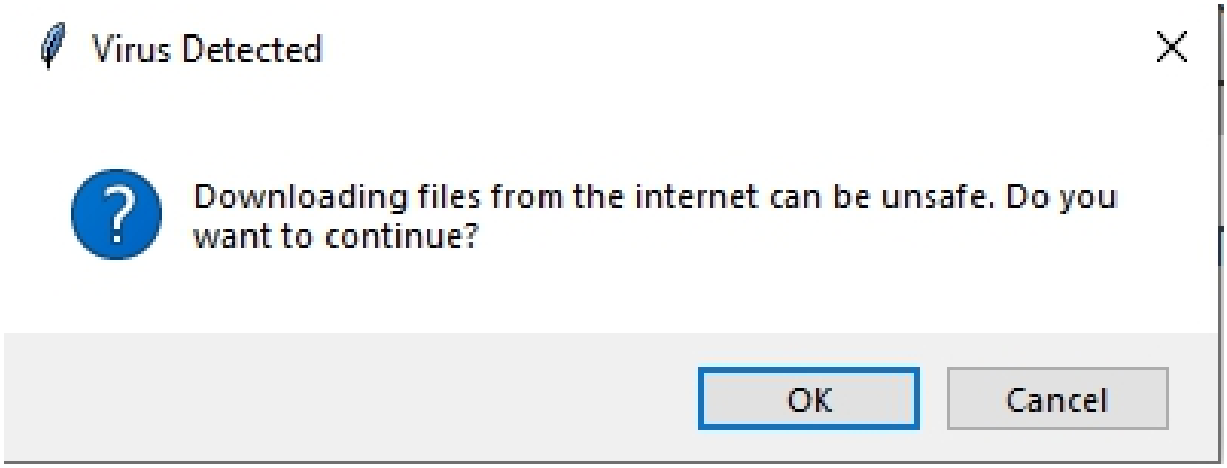
Upon executing the code, following GUI window is displayed



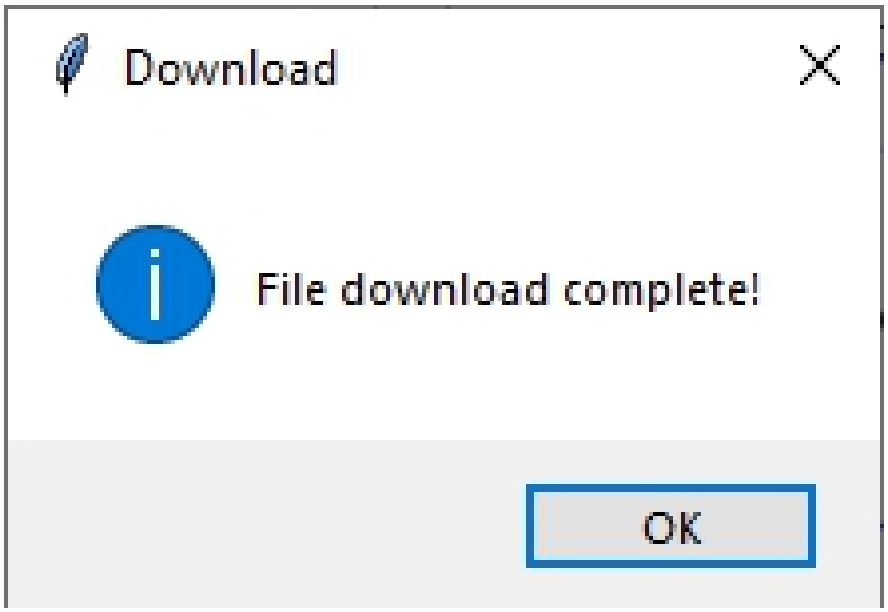
Following warning message is displayed when Download file button clicked



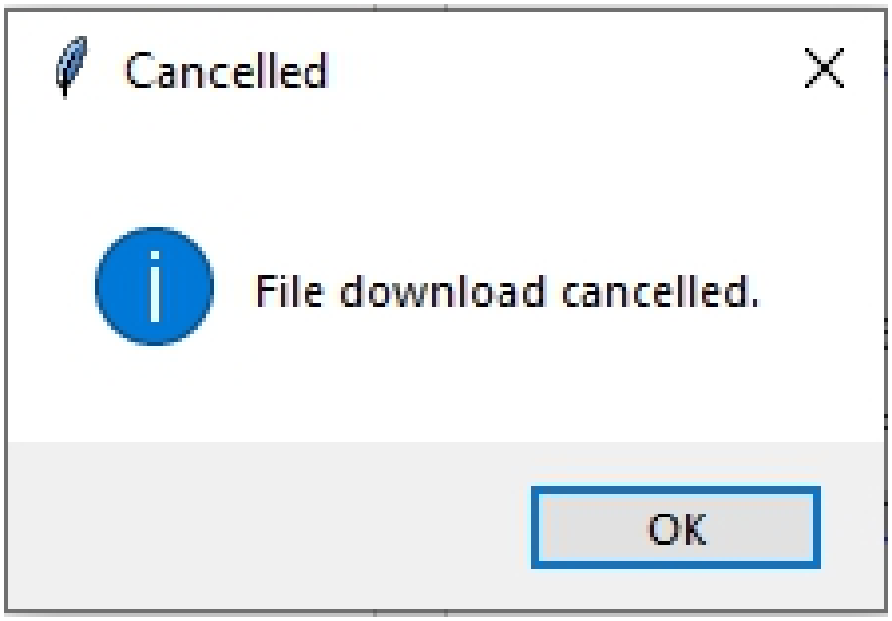
When User click on the ok button, following warning message is displayed informing about the risk of downloading files from internet and ask the user whether they want to continue with download



If user choose OK, then following information message is displayed



If user choose CANCEL, then following information message is displayed



Simple OTP Generator

Project #9

Design simple OTP generator GUI using Tkinter and Python

Introduction:

In this program, we'll design a simple yet effective Graphical User Interface (GUI) application using Tkinter in Python to generate One-Time Passwords (OTPs). The primary functionality of this application is to generate a unique OTP upon clicking the "Generate OTP" button, which will be displayed in a label on the application's main window.

One-Time Passwords (OTPs) are widely used for secure authentication purposes. It offers an additional layer of security by providing a temporary and unique code for each login attempt. Using Tkinter widgets such as Labels and Buttons, we'll guide you through the process of building this OTP Generator application. Upon clicking the "Generate OTP" button, the application will dynamically generate a random OTP and display it in a label on the main window. Let's start building this OTP Generator together!

Solution:

```
import tkinter as tk
import tkinter as tk
import random

def generate_otp():
    # Generate a random 6-digit OTP
    otp = ''.join(random.choices('0123456789', k=4))

    # Display the generated OTP in the label
    otp_label.config(text="Generated OTP: " + otp)

# Create the main window
root = tk.Tk()
```

```
root.title("OTP Generator")

# Set font style for title label
font_bold = ("Arial", 20, "bold")

# Create a Lable for title
title_label = tk.Label(root, text="OTP Generator", fg="blue",
bg="white", font=font_bold)
title_label.pack()

# Create and place widgets
generate_button = tk.Button(root, text="Generate OTP",
command=generate_otp)
generate_button.pack(pady=10)
otp_label = tk.Label(root, text="Generated OTP: ")
otp_label.pack(pady=10)

# Start the Tkinter event loop
root.mainloop()
```

Output :

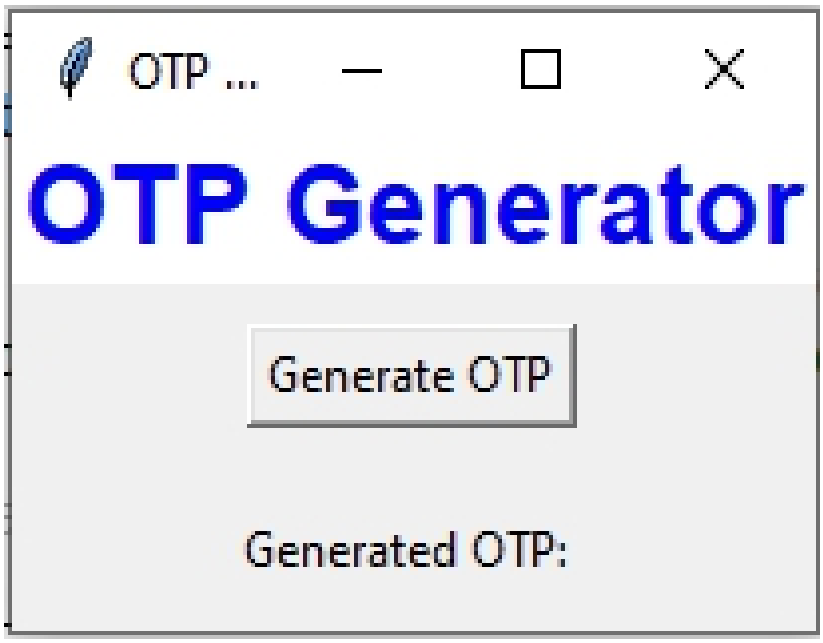
After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This GUI window serves as the interface for the Simple OTP Generator application. Here's what you'll see:

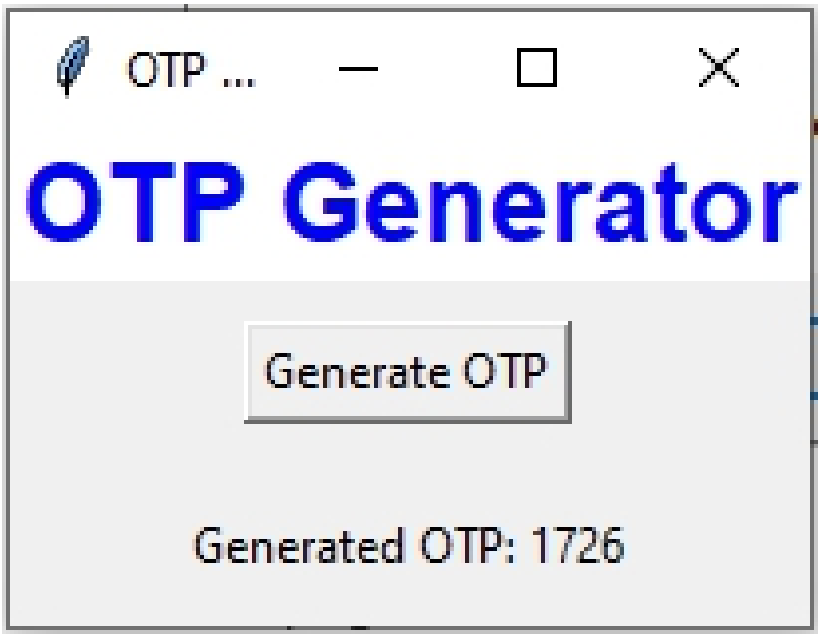
Title: Positioned at the top of the window, you'll find a title reading "OTP Generator". This title indicates the purpose of the application.

Generate OTP Button: Below the title, there is a button labeled "Generate OTP". Clicking this button initiates the generation of a One-Time Password (OTP).

Label: Beneath the "Generate OTP" button, there is a label that reads "Generated OTP:".

Generated OTP: Upon clicking the "Generate OTP" button, the generated OTP is dynamically displayed next to the label "Generated OTP:".





Simple BMI Calculator

Project #10

Design a simple BMI calculator using Python tkinter

Introduction:

Welcome to our BMI (Body Mass Index) Calculator program! In this application, we'll create a simple tool to help you calculate your BMI based on your height and weight. BMI is a measure of body fat based on your height and weight, and it's often used to assess whether you're underweight, normal weight, overweight, or obese. With Tkinter, we'll design an interface where you can input your height and weight in entry widgets. Then, with a click of the "Calculate BMI" button, you'll see your BMI displayed in a messagebox. Additionally, you'll receive a message indicating whether you're underweight, normal weight, overweight, or obese based on your BMI. This program will be helpful for anyone interested in monitoring their weight and overall health. So, let's get started and build this useful BMI Calculator together!

Solution:

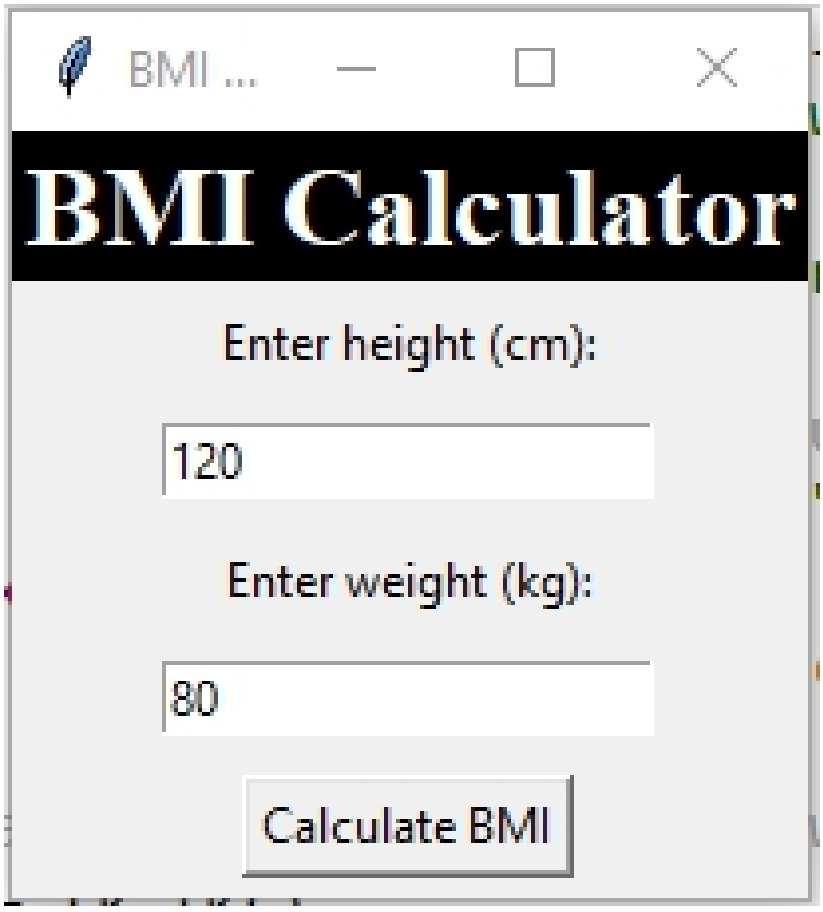
```
import tkinter as tk
from tkinter import messagebox

def calculate_bmi():
    try:
        height = float(entry_height.get())
        weight = float(entry_weight.get())

        # Calculating BMI
        bmi = weight / ((height/100) ** 2)

        # Determining BMI category
        if bmi < 16:
            category = " Severely Underweight"
        elif bmi >= 16 and bmi < 18.5:
            category = "Underweight"
```

```
        elif bmi >= 18.5 and bmi < 25:
            category = "Healthy"
        elif bmi >=25 and bmi < 30:
            category = "Overweight"
        elif bmi >= 30:
            category = "Severely Overweight"

        # Displaying the result
        messagebox.showinfo("Result", f"Your BMI is: {bmi:.2f}\n You
are {category}")
    except ValueError:
        messagebox.showerror("Error", "Please enter valid numeric
values for height and weight.")

# Creating the main tkinter window
root = tk.Tk()
root.title("BMI Calculator")

# Setting up bold font for title label
bold_font = ("Times new roman", 22, "bold")

# Creating lable for title
title_label = tk.Label(root, text="BMI Calculator", bg="black",
fg="white", font= bold_font)
title_label.pack()
# Creating labels and entry fields
label_height = tk.Label(root, text="Enter height (cm):")
label_height.pack(padx=5, pady=5)

entry_height = tk.Entry(root)
entry_height.pack(padx=5, pady=5)

label_weight = tk.Label(root, text="Enter weight (kg):")
label_weight.pack(padx=5, pady=5)

entry_weight = tk.Entry(root)
entry_weight.pack(padx=5, pady=5)

# Creating a button to trigger calculation
calculate_button = tk.Button(root, text="Calculate BMI",
command=calculate_bmi)
```

```
calculate_button.pack(padx=5, pady=5)

# Running the main tkinter event loop
root.mainloop()
```

Output:

After running above code, following GUI will be displayed on your screen. Let’s see what you will see in GUI:
Title is positioned at the top of the window which serve the purpose of application. The application features two entry fields labeled "Enter Height" and "Enter Weight" where users can input their height in centimeters and weight in kilograms, respectively. Below these entry fields, there is a button labeled "Calculate BMI" which, when clicked, computes the BMI based on the provided inputs.



After providing your height and weight measurements and clicking the 'Calculate BMI' button, you'll receive a prompt displaying your Body Mass Index (BMI) result.

Decimal to Binary, Octal and Hexadecimal Convertor

Project #11

Create a graphical user interface (GUI) application using Tkinter in Python for converting a decimal number to its binary, octal, and hexadecimal representations.

Introduction:

Welcome to our Decimal to Binary, Octal, and Hexadecimal Converter! In this Tkinter application, we'll design a simple tool to help you convert a decimal number into its binary, octal, and hexadecimal representations. Decimal numbers are commonly used in everyday life, but sometimes it's useful to convert them into different number systems for various purposes. With Tkinter, we'll create an interface where you can enter any decimal number. Then, with a click of the "Convert" button, you'll see the calculated binary, octal, and hexadecimal representations displayed in labels on the main window. This program will be handy for anyone who needs to perform these conversions quickly and easily. So, let's dive in and build this useful Decimal to Binary, Octal, and Hexadecimal Converter together!

In this program, we will use built in function bin(),hex() and oct() to convert integers to their binary, hexadecimal, and octal representations, respectively.

Here's a brief overview of each function:

bin(): This function takes an integer as input and returns its binary representation as a string prefixed with '0b'.

```
>> bin(10)
'0b1010'
```

hex(): This function takes an integer as input and returns its hexadecimal representation as a string prefixed with '0x'.

```
>>> hex(10)
'0xa'
```

oct(): This function takes an integer as input and returns its octal representation as a string prefixed with '0o'.

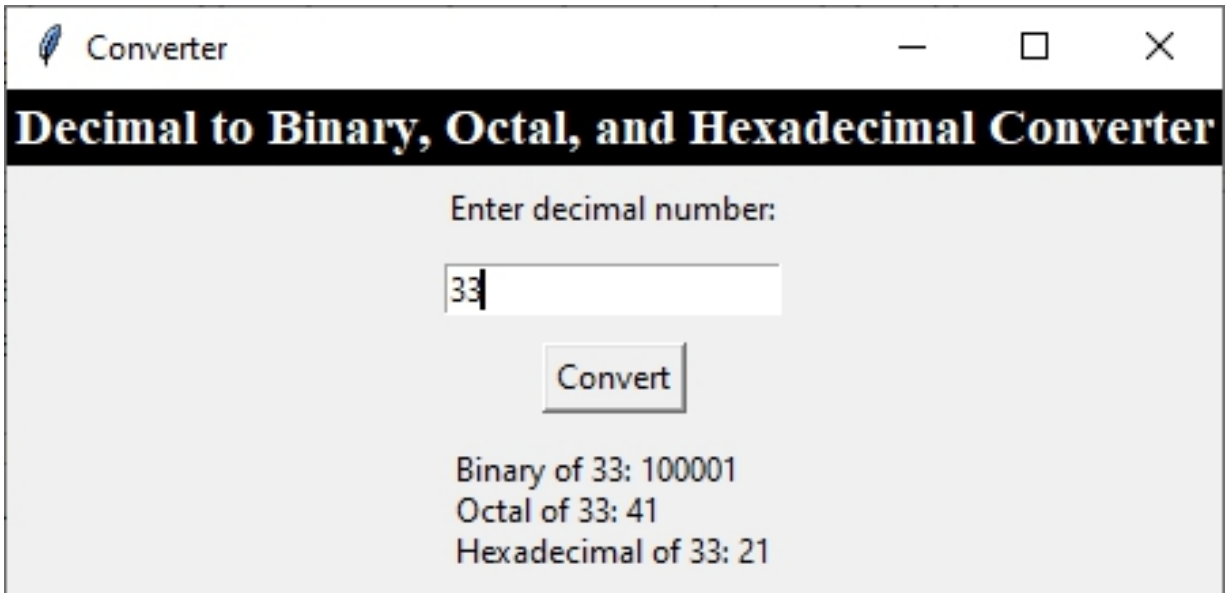
```
>>> oct(10)
'0o12'
```

Solution:

```
# conversion programs
import tkinter as tk
from tkinter import messagebox

def convert():
    try:
        # Retrieve the decimal number entered by the user
        decimal_number = int(entry_decimal.get())

        # Convert the decimal number to binary, octal, and
hexadecimal
        binary_number = bin(decimal_number)[2:]
        octal_number = oct(decimal_number)[2:]
        hexadecimal_number = hex(decimal_number)[2:].upper()

        # Clear previous result, if any
        result_label.config(text="")

        # Display the conversion results on the window
        result_label.config(text=f"Binary of {decimal_number}:
{binary_number}\n"
                            f"Octal of {decimal_number}:
{octal_number}\n"
                            f"Hexadecimal of {decimal_number}:
{hexadecimal_number}")
    except ValueError:
        # Display an error message if the input is not a valid decimal
number
        result_label.config(text="Please enter a valid decimal
number.")

# Creating the main tkinter window
root = tk.Tk()
root.title("Converter")
```

```
# Setting up bold font for title label
bold_font = ("Times new roman", 14, "bold")
# Creating lable for title
title_label = tk.Label(root, text="Decimal to Binary, Octal, and
Hexadecimal Converter", bg="black", fg="white", font= bold_font)
title_label.pack()
# Creating labels and entry field
label_decimal = tk.Label(root, text="Enter decimal number:")
label_decimal.pack(padx=5, pady=5)

entry_decimal = tk.Entry(root)
entry_decimal.pack(padx=5, pady=5)

# Creating a button to trigger conversion
convert_button = tk.Button(root, text="Convert", command=convert)
convert_button.pack(padx=5, pady=5)
# Label to display the result
result_label = tk.Label(root, text="", justify="left")
result_label.pack(padx=5, pady=5)

# Running the main tkinter event loop
root.mainloop()
```

Output: After running the above code, following GUI will be displayed on screen.



After initiating the conversion process by clicking the 'Convert' button, the main window dynamically presents the computed binary, octal, and hexadecimal representations corresponding to the entered decimal number.

&#

Simple ListBox Movement Application

Project #12

Design tkinter GUI application for ListBox Move

Introduction:

Welcome to our List Box Movement Application! In this Tkinter program, we'll create a simple tool to help you manage data between two listboxes. Listboxes are useful for displaying lists of items, and sometimes you need to move items between lists for organization or processing. With Tkinter, we'll design an interface with two listboxes and a set of buttons in between them. These buttons, such as "Move Left", "Move Right", "Move All Left", and "Move All Right", will allow you to move data items from one listbox to the other. This way, you can easily organize your data according to your needs. This application will be handy for anyone who needs to manage and manipulate lists of items efficiently. So, let's start building this useful List Box Movement Application together! First we will study about the listbox and its key features and properties in brief.

ListBox Widget

A Tkinter Listbox is a widget used to display a list of items from which users can select one or more items. It provides a simple way to present data in a list format within a graphical user interface.

Here are some key features and properties of Tkinter Listbox widget:
Displaying Items: The Listbox widget can display a list of items vertically.
Selection: Users can select one or more items from the list using mouse clicks or keyboard navigation.
Scrollbars: If the list is too long to fit within the available space, Tkinter automatically adds vertical and/or horizontal scrollbars to allow users to navigate through the list.
Binding Events: You can bind events to the Listbox widget to perform specific actions when items are selected or interacted with.

Item Configuration: Each item in the Listbox can have its own configuration, such as font, color, or background color.
Inserting and Deleting Items: You can insert new items into the Listbox or delete existing items dynamically.
Selection Modes: Listbox supports different selection modes, such as single selection, multiple selection, or extended selection.

Tkinter Listbox Properties:

Background (bg): Sets the background color of the Listbox.
Border Width (bd): Sets the width of the border around the Listbox.
Font (font): Sets the font style and size for the text displayed in the Listbox.
Foreground (fg): Sets the color of the text displayed in the Listbox.
Height (height): Sets the height of the Listbox in terms of the number of displayed items.
Width (width): Sets the width of the Listbox in terms of characters or pixels.
Relief: Determines the appearance of the border around the Listbox. It can be set to "flat", "raised", "sunken", "groove", or "ridge".
Select Mode (selectmode): Sets the selection mode of the Listbox. Options include "single" for single selection, "browse" for single selection with mouse button 1, "multiple" for multiple selection with Shift and Ctrl keys, and "extended" for extended selection with the Shift key. **yscrollcommand:** Attaches a scrollbar for vertical scrolling.
xscrollcommand: (optional) Attaches a scrollbar for horizontal scrolling.
activestyle: Sets the style for highlighting active items (none, dotbox, underline).

Tkinter Listbox Methods:

insert(index, *elements): Inserts one or more elements into the Listbox at the specified index.
delete(0, END): Deletes all items from the listbox.
delete(index1, index2): Deletes items within the specified range (index1 to index2).
index(item): Returns the index of the first occurrence of an item.

get(index1, index2): Retrieves a list of items within the specified range.
size(): Returns the total number of items in the listbox.
activate(index): Selects the item at the specified index.
select_set(index1, index2): Selects a range of items.
select_clear(): Deselects all items.
selection_includes(index): Checks if an item is selected.
curselection(): Returns a list of indices of currently selected items.
see(index): Ensures the item at the specified index is visible by scrolling if needed.
first(): Returns the index of the first item.
last(): Returns the index of the last item.

Solutions:

```
import tkinter as tk

def insert_list():
    data = entry_item.get()
    # Populate the first listbox with some sample items
    listbox1.insert(tk.END, f"{data}\n")

def move_right():
    selected_indices = listbox1.curselection()
    for idx in selected_indices[::-1]:
        listbox2.insert(tk.END, listbox1.get(idx))
        listbox1.delete(idx)

def move_left():
    selected_indices = listbox2.curselection()
    for idx in selected_indices[::-1]:
        listbox1.insert(tk.END, listbox2.get(idx))
        listbox2.delete(idx)

def move_all_right():
    listbox2.delete(0, tk.END)
    for idx in range(listbox1.size()):
        listbox2.insert(tk.END, listbox1.get(idx))
    listbox1.delete(0, tk.END)
```

```
def move_all_left():
    listbox1.delete(0, tk.END)
    for idx in range(listbox2.size()):
        listbox1.insert(tk.END, listbox2.get(idx))
    listbox2.delete(0, tk.END)

# Create the main window
root = tk.Tk()
root.title("Listbox Move Example")

# Setting up bold font for title label
bold_font = ("Times new roman", 20, "bold")

# Creating lable for title
title_label = tk.Label(root, text="Listbox Move Application",
bg="yellow", fg="red", font= bold_font)
title_label.pack(padx=5, pady=5)

# Creating labels, entry fields, and buttons for simple interest
label_item = tk.Label(root, text="Enter list items one by one:")
label_item.pack(padx=5, pady=5)

entry_item = tk.Entry(root)
entry_item.pack(padx=5, pady=5)

add_button = tk.Button(root, text="Add list items",
command=insert_list)
add_button.pack(pady=10)

# Create listboxes
listbox1 = tk.Listbox(root, selectmode=tk.MULTIPLE)
listbox1.pack(side=tk.LEFT, padx=10, pady=10)


# Create a frame to contain color buttons
btn_frame = tk.Frame(root, padx=10, pady=10)
btn_frame.pack(side=tk.LEFT)

# Create buttons for moving items between listboxes
move_right_button = tk.Button(btn_frame, text=">",
command=move_right)
```

```
move_right_button.pack(side=tk.TOP, padx=10, pady=10)

move_left_button = tk.Button(btn_frame, text="<",
command=move_left)
move_left_button.pack(side=tk.TOP, padx=10, pady=10)
move_all_right_button = tk.Button(btn_frame, text=">>",
command=move_all_right)
move_all_right_button.pack(side=tk.TOP, padx=10, pady=10)

move_all_left_button = tk.Button(btn_frame, text="<<",
command=move_all_left)
move_all_left_button.pack(side=tk.TOP, padx=10, pady=10)

listbox2 = tk.Listbox(root, selectmode=tk.MULTIPLE)
listbox2.pack(side=tk.LEFT, padx=10, pady=10)

# Start the Tkinter event loop
root.mainloop()
```

Output:

After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This GUI window serves as the interface for the Listbox Move application. Here's what you'll see:

Title is positioned at the top of the window, you'll find a title reading "Listbox Move Application". This title indicates the purpose of the application.

Entry Field: Below the title, there's an entry field where users can input list items one by one.

Add list Item Button: Beneath to the entry field, there's a button labeled "Add Item". Clicking this button adds the item entered in the entry field into the first listbox.

Listboxes:

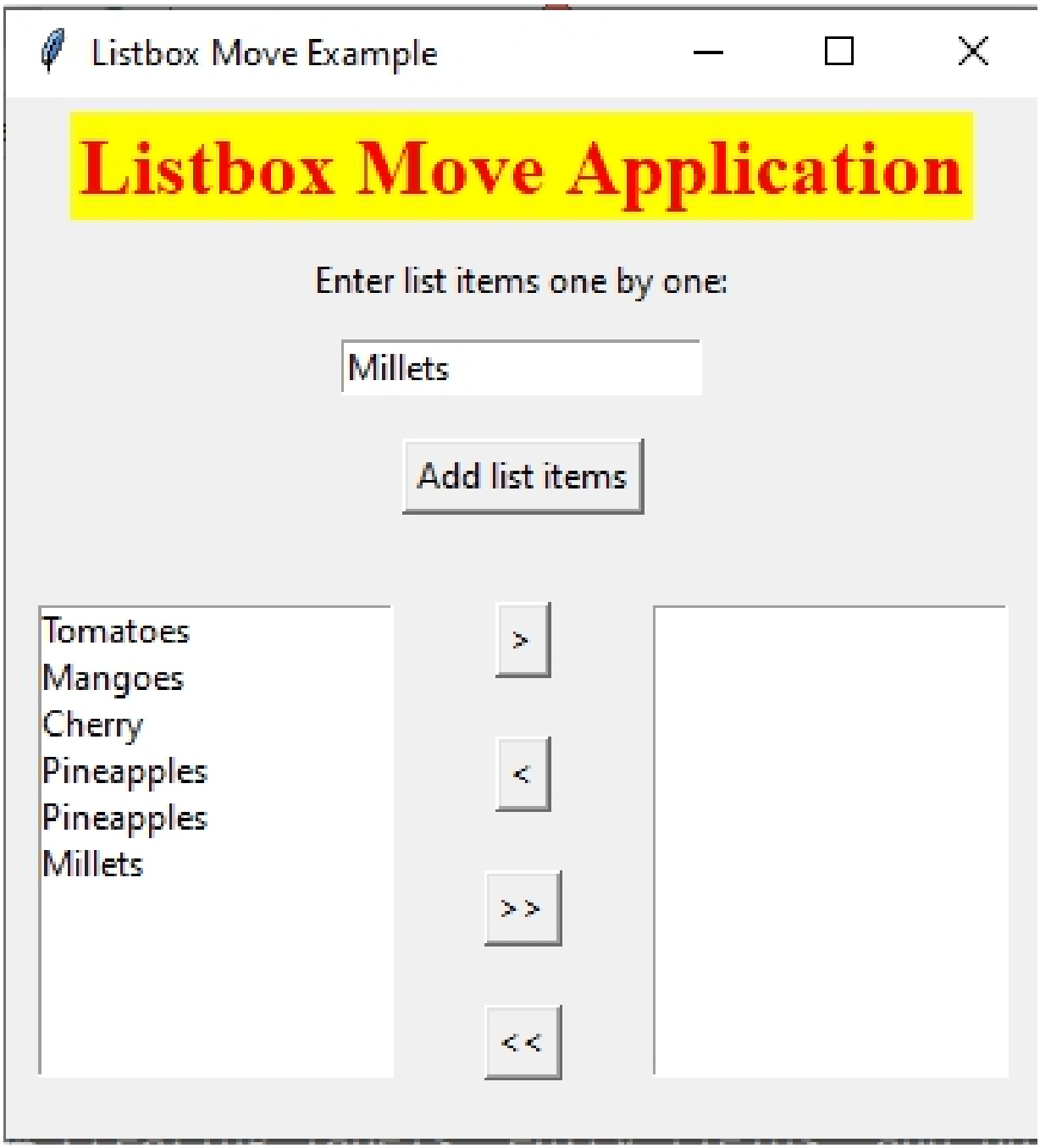
First Listbox: Below the "Add Item" button, there's the first listbox where the added items are displayed.

Second Listbox: Next to the first listbox, there's the second listbox. Initially, it will be empty.

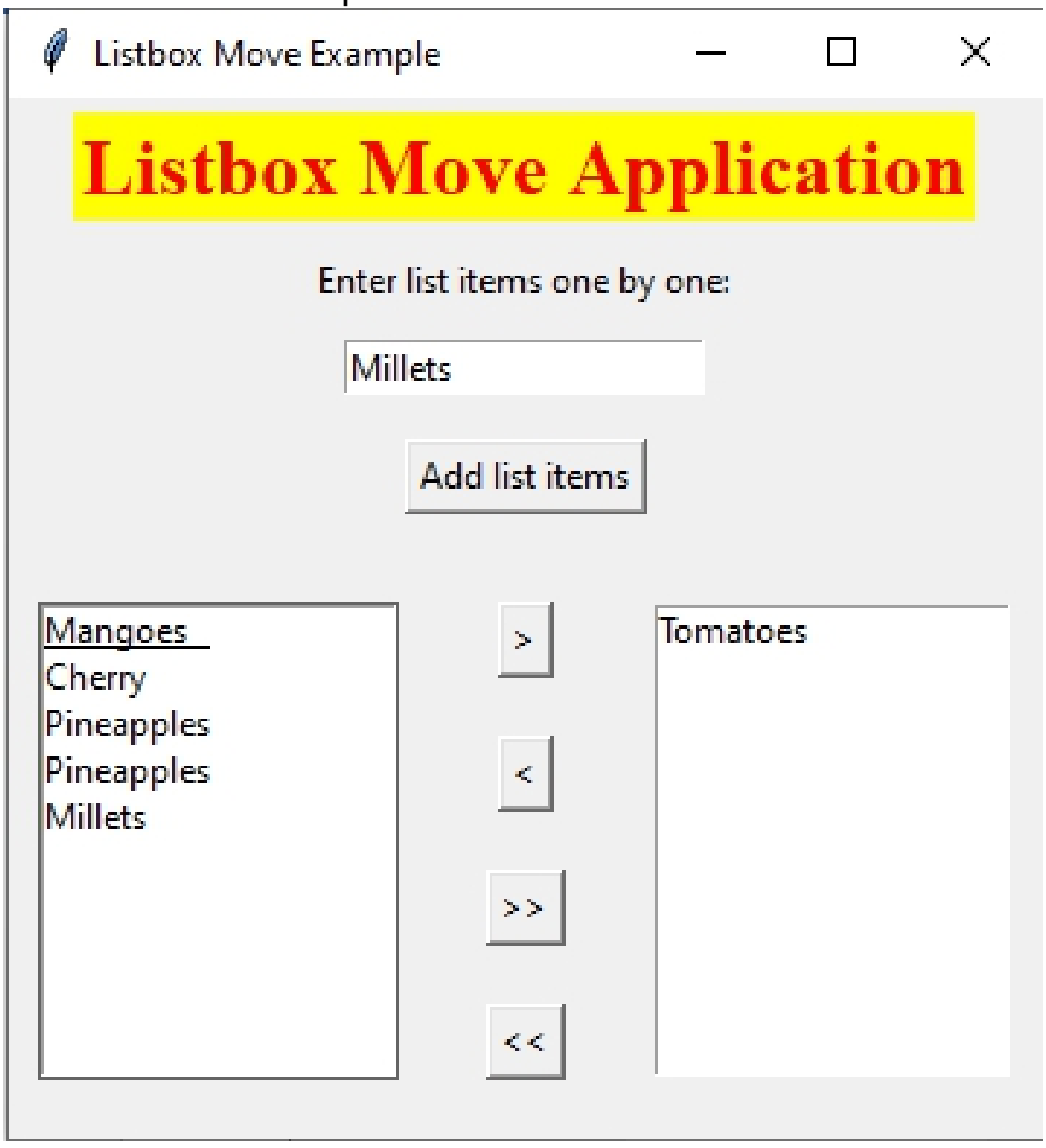
Buttons (positioned between the two listboxes):

Move Left Button: This button allows you to move the selected item from the second listbox back to the first listbox.

Move Right Button: This button enables you to move the selected item from the first listbox to the second listbox.
Move All Left Button: Clicking this button moves all items from the second listbox back to the first listbox.
Move All Right Button: This button moves all items from the first listbox to the second listbox.



User enters the list of items one by one into the entry field and clicks on the 'Add list items' button. With each click, the items are added to the first list box. All list items are then populated in the first list box, as shown above in output.



After selecting the list item “tomatoes” from first list box, and clicking on move right button, list item tomatoes is moved from first listbox to second list box like above.

After selecting the list item 'tomatoes' from the second listbox and clicking the 'Move Left' button, it is shifted from the second listbox back to the first listbox, where it is appended at the end of the existing elements, as shown above.

After clicking the 'Move All Right' button, all elements of the first listbox are transferred to the second listbox, as depicted above.

After clicking the 'Move All left' button, all elements of the second listbox are transferred to the first listbox, as depicted above.

Simple Font Style Customizer

Project #13

Design a GUI application using Tkinter in Python that allows users to customize the font style of a label dynamically.

Introduction:

Welcome to the Font Style Customizer! This easy-to-use program lets you change how text looks in a label on your screen. You can pick different font styles, like Times New Roman or Calibri, from a list. Plus, you can make the text bold, italic, or underline by checking boxes. Changing font styles and making text bold or italic can make your text look more interesting or easier to read. With the Font Style Customizer, you can do all of this quickly and easily. We'll guide you through using simple menus and checkboxes to change the font style and attributes. As you make choices, you'll see the label on the screen change right away to match your selections. By the end of this program, you'll be able to customize text on your screen just the way you like it. Let's get started with the Font Style Customizer!

First we will study about Checkbox widget and Dropdown menu. So we can use them in our code.

Checkbox :

Checkboxes are small boxes that you can click to select or deselect an option. They look like little squares that you can either tick or leave empty. When you tick a checkbox, it means you're choosing that option. If you untick it, you're saying you don't want that option. In Tkinter, checkboxes are used in graphical user interfaces (GUIs) to let users make choices. For example, if you have a list of options and you want users to be able to choose more than one, you can use checkboxes. Each checkbox represents one option, and users can click on them to select or deselect their choices.

Creating checkboxes :

Creating a checkbox in Tkinter involves a few simple steps:
1. Import Tkinter: import tkinter as tk
2. Create a Tkinter window:
root = tk.Tk()
window.title("My Checkbox Example")
3. Define a variable to store the checkbox state:
state = tk.IntVar() # Creates an integer variable, initially set to 0 (unchecked)
4. Create the checkbox:
checkbox = tk.Checkbutton(root, text="Click me!", variable=state,onvalue=1, offvalue=0 # Set onvalue to 1 when checked, offvalue to 0 when unchecked)
5. Customize the checkbox (optional):
Text: Change the text displayed with the text option.
Font: Change the font with the font option.
Color: Change the color with the fg (foreground) and bg (background) options.
Image: Use an image instead of text with the image option.
6. Place the checkbox in the window:
checkbox.pack() # Or use other layout managers like grid or place
7. (Optional) Add functionality:
Link the checkbox to an action: Use the command option to call a function when the checkbox is clicked.

Dropdown menu:

In Tkinter, a dropdown menu is a type of menu that allows users to choose from a list of options by clicking on a button. When you click on the button, a list of options appears, and you can select one of them. Once you select an option, it becomes the current choice displayed on the button.
Dropdown boxes are commonly used in graphical user interfaces (GUIs) to provide users with a convenient way to select from a list of options without taking up too much space on the screen. They're handy for giving users choices and making it easy for them to make selections from a list of options.
Creating a dropdown in Tkinter involves several steps. Below, I'll guide you through each step to create a simple dropdown menu:

Step 1: Import Tkinter

import tkinter as tk
Step 2: Create the Main Window
root = tk.Tk()
Step 3: Create a Variable to Store the Selected Option

selected_option = tk.StringVar()
Step 4: Define the Options for the Dropdown

options = ["Option 1", "Option 2", "Option 3"]
Step 5: Create the Dropdown Menu

dropdown = tk.OptionMenu(root, selected_option, *options)
Step 6: Place the Dropdown Menu in the Window

dropdown.pack()
Step 7: Start the Tkinter Event Loop

root.mainloop()

Solution:

```
import tkinter as tk
from tkinter import font

# Create the main window
root = tk.Tk()
root.title("Font Style Changer")

# Define font styles
font_styles = ["Helvetica", "Arial", "Times New Roman", "Courier"]

# Variable to store the selected font style
current_font = tk.StringVar()
current_font.set("Helvetica")

# Variable to store the selected font size
current_font_size = tk.IntVar()
current_font_size.set(12)

# Function to update the font of the label
def update_label_font(*args):
```

```
    selected_font = font.Font(family=current_font.get(),
size=current_font_size.get(), weight=current_bold.get(),
slant=current_italic.get(), underline=current_underline.get(),
overstrike=current_strikethrough.get())
    label.config(font=selected_font)

# Create dropdown for font style selection
font_style_label = tk.Label(root, text="Font Style:")
font_style_label.grid(row=0, column=0, padx=5, pady=5)

font_style_dropdown = tk.OptionMenu(root, current_font,
*font_styles, command=update_label_font)
font_style_dropdown.grid(row=0, column=1, padx=5, pady=5)

# Create checkboxes for font style
current_bold = tk.StringVar(value="normal")
bold_check = tk.Checkbutton(root, text="Bold",
variable=current_bold, onvalue="bold", offvalue="normal",
command=update_label_font)
bold_check.grid(row=1, column=0, padx=5, pady=5)

current_italic = tk.StringVar(value="roman")
italic_check = tk.Checkbutton(root, text="Italic",
variable=current_italic, onvalue="italic", offvalue="roman",
command=update_label_font)
italic_check.grid(row=1, column=1, padx=5, pady=5)

current_underline = tk.IntVar(value=0)
underline_check = tk.Checkbutton(root, text="Underline",
variable=current_underline, onvalue=1, offvalue=0,
command=update_label_font)
underline_check.grid(row=1, column=2, padx=5, pady=5)

current_strikethrough = tk.IntVar(value=0)
strikethrough_check = tk.Checkbutton(root, text="Strikethrough",
variable=current_strikethrough, onvalue=1, offvalue=0,
command=update_label_font)
strikethrough_check.grid(row=1, column=3, padx=5, pady=5)

# Create label with initial font settings
```

```
label = tk.Label(root, text="Welcome to Python Programming Worlds!!!!", font=(current_font.get(), current_font_size.get()))
label.grid(row=2, columnspan=4, padx=5, pady=5)

# Start the Tkinter event loop
root.mainloop()
```

Output:

After launching the provided code, you'll encounter a Graphical User Interface (GUI) window on your screen, which serves as the Font Styler application interface. Here's an overview of the components:

Dropdown for Font Name: At the top of the window, there's a dropdown menu containing options for selecting the font name. Users can choose from options such as Arial, Times New Roman, and others.

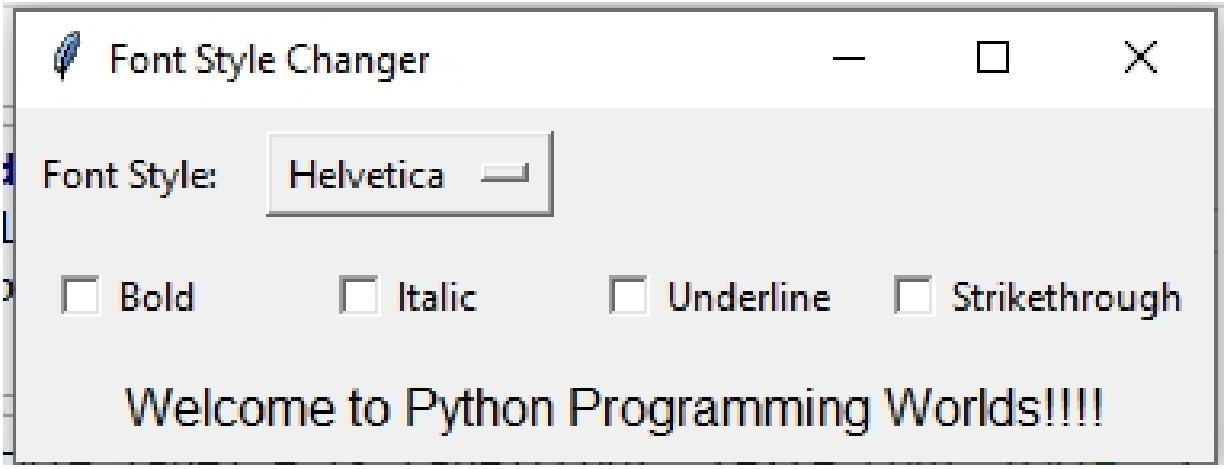
Checkboxes for Font Styles: Below the font name dropdown, there are four checkboxes:

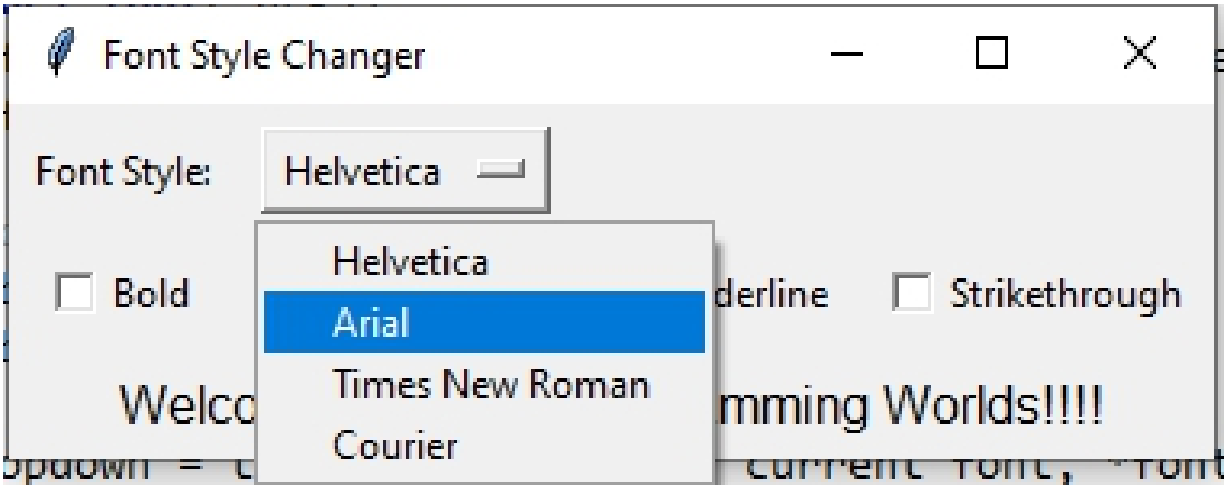
Bold: Allows users to apply bold formatting to the selected text.
Italic: Enables users to apply italic formatting to the selected text.
Underline: Permits users to apply underline formatting to the selected text.
Strike Through: Allows users to apply strike through formatting to the selected text.

Label: Positioned at the bottom of the checkboxes, there's a label that displays the current font style based on the checkboxes' selection and the font name chosen from the dropdown.

Functionality:

Users can select a font name from the dropdown menu.
They can then click on the checkboxes to apply various font styles such as bold, italic, underline, and strike through.
As users make selections, the label at the bottom dynamically updates to reflect the chosen font style and name.





After choosing the font name 'Times New Roman' from the dropdown menu, and activating the checkboxes for bold, italic, and underline, the label 'Welcome to Python Programming Worlds!!!' transforms as shown above.

Image Swapper Application

Project #14

Develop an Image Swapper Application that allows users to view two images and swap between them with the click of a button.

Introduction:

Welcome to the Image Swapper! This program helps you look at two pictures and switch between them easily. Just click a button, and the pictures will change.

Sometimes you need to compare two pictures or see how something looked before and after. With this program, you can do that quickly and without any trouble.

We'll use Tkinter, a tool for making buttons and windows in Python, to create this Image Swapper. It's easy to use, and you'll learn how to switch between pictures in no time.

By the end of this program, you'll know how to use a simple tool to swap images and make comparing pictures a breeze. Let's start swapping images!

PILLOW Library

In this program, we will use Tkinter pillow library for image processing. So let’s talk about Tkinter Pillow. Pillow (formerly known as PIL) is a popular library for image processing and manipulation in Python. It supports a wide range of image formats, including PNG, JPEG, GIF, and more.

You can use Pillow to open, resize, crop, convert, and perform other operations on images. Tkinter and Pillow complement each other perfectly. You can use Pillow to process and manipulate images, then use Tkinter to display them in your GUI.

To install the Pillow library, you can use Python's package manager, pip. Here's how you can do it:

pip install pillow

This command will download and install the Pillow library and all its dependencies. Once the installation is complete, we can start using Pillow in our Python projects for image processing tasks.
When you want to work with images using the Pillow library (PIL stands for Python Imaging Library, which was the old name for Pillow), you import the Image module from PIL. Similarly, if you're working with Tkinter and want to display images within Tkinter GUI applications, you import the ImageTk module from PIL. Here's the standard import statement:

from PIL import Image, ImageTk

With this import statement, you can use Image class to load image from file, resize it and apply filter and do many more tasks and `ImageTk` class to convert a Pillow Image object into a format compatible with Tkinter's `PhotoImage` widget, which you can then display in your GUI. Both Image and ImageTk classes are imported from Pillow Library. We will discuss what PhotoImage widget is later.

Solutions:

```
import tkinter as tk
from PIL import Image, ImageTk

# Define current_image_index variable
current_image_index = 0

def swap_images():
    # Use the global keyword to access and modify the global variable current_image_index
    global current_image_index

    # Update the current_image_index to the index of the next image in the images list
    current_image_index = (current_image_index + 1) % len(images)

    # Swap the positions of the images in the images list
    images[0], images[1] = images[1], images[0]

    # Update the image_label to display the first image in the list
    image_label.config(image=images[0])
```

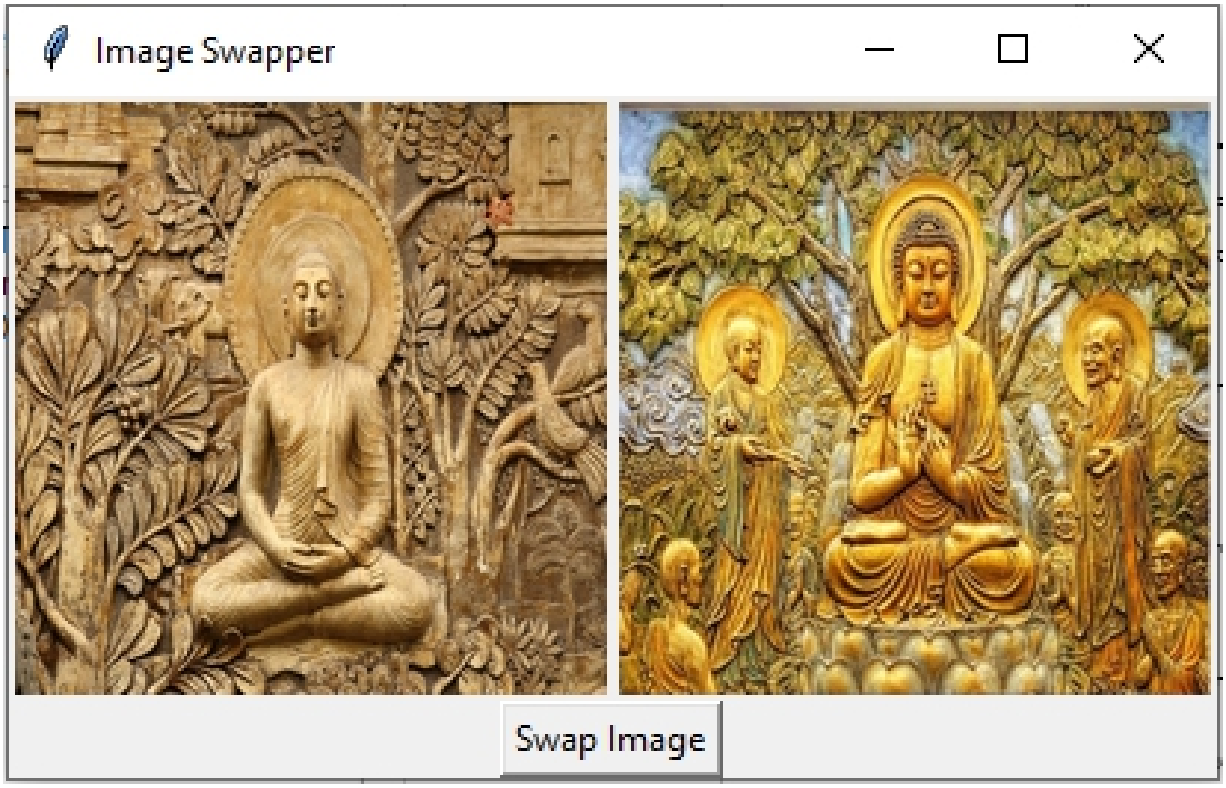
```
    # Update the image_label2 to display the second image in the list
    image_label2.config(image=images[1])

# Create a Tkinter window
root = tk.Tk()
# Set the title of the window to "Image Swapper"
root.title("Image Swapper")

# List of image file paths
image_files = ["image/image1.jpg", "image/image2.jpg"]

# Initialize an empty list to store resized and converted images
images = []

# Iterate over each image file path
for file in image_files:
    # Open the image file and resize it to the desired dimensions
(200x200 pixels)
    img = Image.open(file).resize((200, 200), Image.ANTIALIAS)

    # Convert the resized image to a Tkinter PhotoImage object
    photo_img = ImageTk.PhotoImage(img)

    # Append the PhotoImage object to the images list
    images.append(photo_img)

# Display original images
# Create a Label widget to display the first image
image_label = tk.Label(root, image=images[0])

# Place the image_label widget in the Tkinter window using the grid
geometry manager
# The widget is placed in row 0 and column 0
image_label.grid(row=0, column=0)

# Create another Label widget to display the second image
image_label2 = tk.Label(root, image=images[1])

# Place the image_label2 widget in the Tkinter window using the grid
geometry manager
# The widget is placed in row 0 and column 1
image_label2.grid(row=0, column=1)
```

```
# Create a button widget for swapping images
swap_button = tk.Button(root, text="Swap Image",
command=swap_images)

# Place the button in the Tkinter window using the grid geometry
manager
# The button will be placed in row 1 and will span across 2 columns
swap_button.grid(row=1, columnspan=2)


# Enter the Tkinter event loop

root.mainloop()
```

Output:

After executing the above code, you will see a graphical user interface (GUI) window displayed on your screen. This GUI window showcases two images side by side, with a 'Swap' button positioned below them. When clicked, the images interchange their positions. For instance, if the left image was initially displayed on the left side, clicking the 'Swap' button will move it to the right side, and vice versa.

Initially, the images are displayed as shown in the output above. However, after clicking the "Swap Image" button, the images are exchanged, resulting in the updated display depicted below.



Simple Progress Bar Application

Project #15

Develop a Graphical User Interface (GUI) application using Tkinter in Python that includes a progress bar.

Introduction:

Welcome to our new project! In this program, we're going to make something cool using Tkinter in Python. We'll create a special button that, when you click on it, makes a progress bar move forward. A progress bar is like a little bar that fills up as something happens. It's useful for showing how far along a task is, like when you're downloading a file or completing a job. Our goal is to make a program where there's a button called "Start Progress". When you click on this button, a progress bar will start filling up to show you that something is happening. We'll use Tkinter to make this program easy to use. By the end of our project, you'll know how to make a program where clicking a button makes a progress bar move. Let's get started!

ProgressBar widget

In Tkinter, the ProgressBar widget allows you to display a progress bar in your GUI applications.
Here are some common properties of the progress bar widget that you can modify:
maximum: This property sets the maximum value of the progress bar. By default, it is set to 100.
value: This property sets the current value of the progress bar. It should be between the minimum and maximum values.
mode: This property determines how the progress bar is displayed. It can be set to "determinate" (default), "indeterminate", or "buffer".
orient: This property sets the orientation of the progress bar. It can be set to "horizontal" (default) or "vertical".
length: This property sets the length of the progress bar in pixels or other units.

variable: This property can be used to link a Tkinter variable to the progress bar. Changes to the variable will automatically update the progress bar.

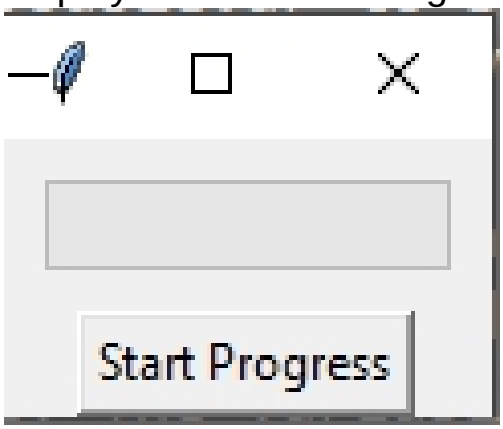
Solutions:

```
import tkinter as tk
from tkinter import ttk
import time

def start_progressbar():
    progress_var.set(0)  # Reset progress bar to start from 0
    for i in range(101):  # Iterate from 0 to 100 (inclusive)
        time.sleep(0.05)  # Simulate some task delay
        progress_var.set(i)  # Update progress bar value to current iteration
        root.update_idletasks()  # Update GUI to reflect changes
    tk.messagebox.showinfo("Progress", "Your Task is completed!")  # Display completion message

root = tk.Tk()
root.title("Progress Bar")

# Create a progress bar widget
# Create a DoubleVar to store the value of the progress bar
progress_var = tk.DoubleVar()

# Create a ttk.Progressbar widget with the specified options:
# - root: The parent window for the progress bar
# - variable: The variable associated with the progress bar to track its value
# - maximum: The maximum value of the progress bar (in this case, 100)
progress_bar = ttk.Progressbar(root, variable=progress_var, maximum=100)

# Pack the progress bar widget into the window with a padding of 10 pixels on the y-axis
progress_bar.pack(pady=10)

# Create a button to start the progress
```

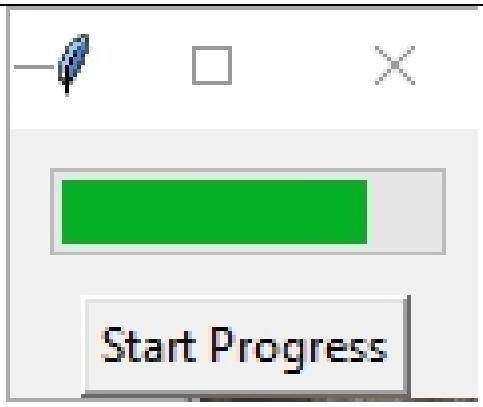
```
start_button = tk.Button(root, text="Start Progress",
command=start_progressbar)
start_button.pack()
root.mainloop()
```

Output:

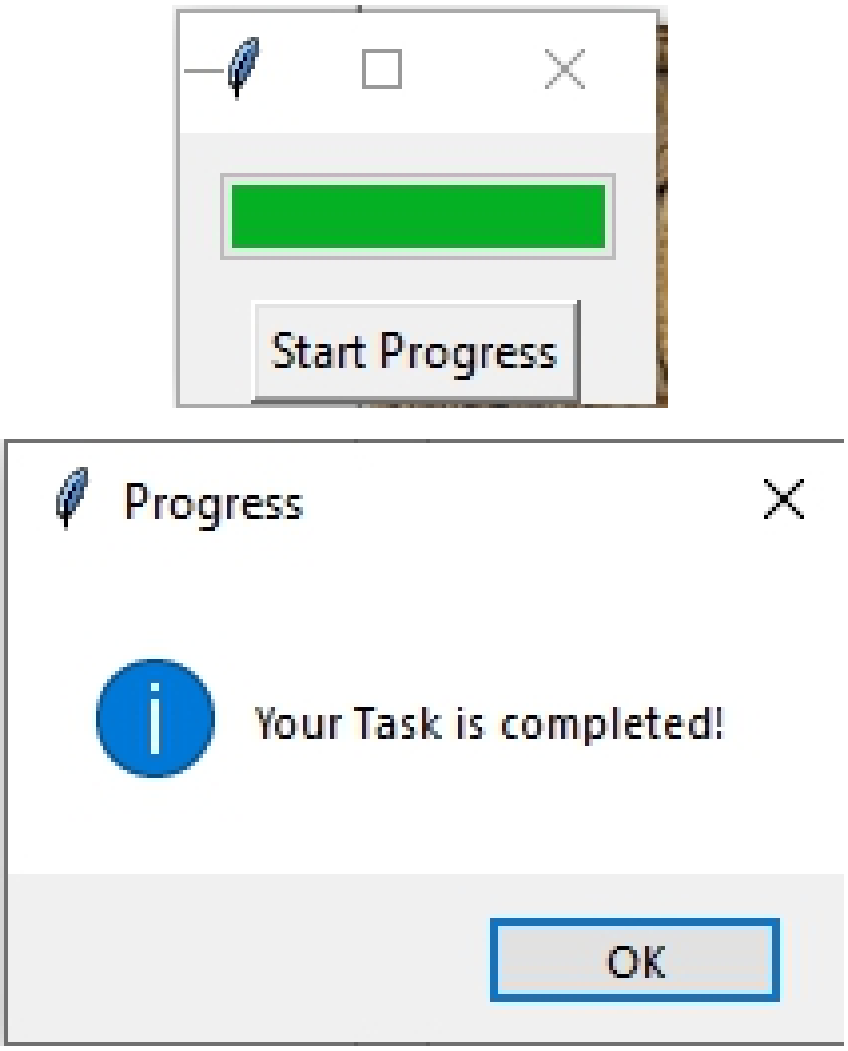
Following window is displayed after executing the code above



When the user clicks on the "Start Progress" button, the progress bar begins processing.



Following information message is displayed after completing the task.



Times Table Generator

Project #16

Develop a Python GUI application using Tkinter that generates a times table based on a number entered by the user.

Introduction:

In this project, we'll be creating a Python GUI application using Tkinter that helps users generate times tables based on a number they enter. The application will display the times table neatly in a text area on the click on generate times table button.

Times tables are useful for learning multiplication and understanding mathematical patterns. With our application, users can enter any number, and the program will generate the times table for that number, making it easy to practice multiplication and study math concepts.

Using Tkinter, a Python library for creating graphical user interfaces, we'll guide you through the process of building this application. You'll learn how to create input fields for users to enter numbers, how to generate times tables based on user input, and how to display the results in a text area. Lets get started :

First ,we will study about TextArea widget and then use it in our code

TextArea widget :

In Tkinter, a textarea is like a big box where you can write and display text. It's useful for showing a lot of text at once, like a paragraph or a list.

In a Tkinter application, you can use a textarea to show information, messages, or results to the user. It's a handy tool for displaying large amounts of text in a clear and organized way.

Creating a textarea in Tkinter involves several steps. Below, I'll guide you through each step to create a simple textarea:

Step 1: Import Tkinter

import tkinter as tk
Step 2: Create the Main Window
root = tk.Tk()
Step 3: Create a Text Area Widget

text_area = tk.Text(root)
Step 4: Add Text to the Text Area
text_area.insert(tk.END, "Hello, this is a textarea!")
Step 5: Place the Text Area in the Window

text_area.pack()
Step 6: Start the Tkinter Event Loop

root.mainloop()
General Properties:

width: Sets the width of the textarea in characters (default: 20).
height: Sets the height of the textarea in lines (default: 10).
textvariable: Links the textarea to a Tkinter variable to store the text content.
font: Defines the font family, size, and other font attributes (e.g., "Helvetica 12 bold").
foreground: Sets the text color (e.g., "black").
background: Sets the background color (e.g., "white").
wrap: Controls how text wraps within the textarea (e.g., "word", "char", "none").
padx and pady: Add horizontal and vertical padding around the text content.
relief: Sets the border style (e.g., "flat", "raised", "sunken").
cursor: Defines the cursor appearance (e.g., "xhair", "ibeam").
state: Controls the editable state of the textarea (e.g., "normal", "disabled", "readonly").
highlightthickness: Sets the thickness of the border when the textarea is focused.
insertofftime and insertontime: Control the blink rate of the text insertion cursor (in milliseconds).
Scrollbars:

xscrollcommand and yscrollcommand: Links the textarea to scrollbar widgets for horizontal and vertical scrolling.
Events:
Command: Calls a function when the user types or changes the text.
insert: Fires when text is inserted at a specific index.
delete: Fires when text is deleted from a specific index.
bind: Associates any event with a specific function (e.g., <Button-1> for clicking).

Solutions:

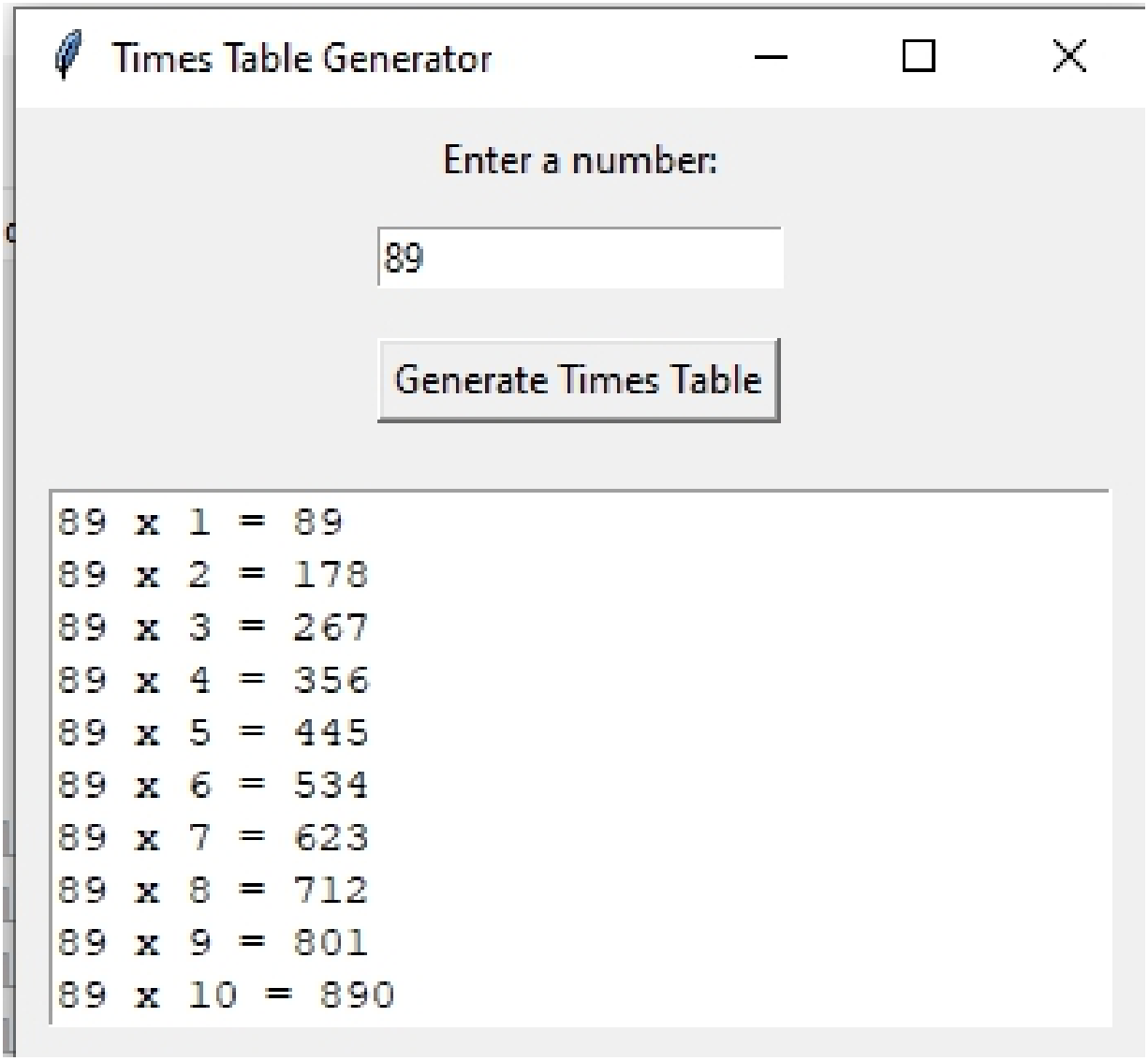
```
import tkinter as tk

# Function to generate the times table
def generate_times_table():
    try:
        # Get the number entered by the user
        number = int(number_entry.get())
        # Enable the text area for editing and clear its contents
        text_area.config(state=tk.NORMAL)
        text_area.delete("1.0", tk.END)

        # Generate and display the times table for the entered number
        # Iterate through numbers from 1 to 10 to generate the times table for the entered number
        for i in range(1, 11):
        # Calculate the result of multiplying the entered number by the current iteration value
            result = number * i
            # Insert the result into the text area at the end of the current content

            text_area.insert(tk.END, f"{number} x {i} = {result}\n")
        # Disable the text area to prevent editing
        text_area.config(state=tk.DISABLED)

    except ValueError:
```

```
        # If the entered value is not a valid integer, display an error message
        text_area.config(state=tk.NORMAL)
        # Clear the contents of the text area from the first character of the first line to the end of the text
        text_area.delete("1.0", tk.END)
        # Insert a message prompting the user to enter a valid number at the end of the text area
        text_area.insert(tk.END, "Please enter a valid number.")
        # Disable the text area to prevent editing
        text_area.config(state=tk.DISABLED)

# Create the main window
window = tk.Tk()
window.title("Times Table Generator")

# Create and place widgets
tk.Label(window, text="Enter a number:").pack(pady=5)
number_entry = tk.Entry(window)
number_entry.pack(pady=5)

generate_button = tk.Button(window, text="Generate Times Table", command=generate_times_table)
generate_button.pack(pady=10)

# Create a textarea (text widget)
text_area = tk.Text(window, height=10, width=40)
text_area.pack(padx=10, pady=10)

# Start the GUI event loop
window.mainloop()
```

Output:

After executing the above code, you will see a graphical user interface (GUI) window appear on your screen. At the top of the window, there's an input field where users can enter a number for which they want to generate the times table. Below the input field, there's a 'Generate' button. Users click this button to initiate the generation of the times table based on the number entered in the input field. Beneath the 'Generate' button, there's a text area. Once

the user clicks the 'Generate' button, the times table for the entered number will be displayed in this text area.



Simple ShapeDrawer Application

Project #17

Develop a Python program called ShapeDrawer that allows users to interactively select and display various shapes on a canvas.

Introduction:

Welcome to ShapeDrawer! This program is designed to help you explore various shapes interactively. On the main window, you'll find six different radio buttons representing different shapes: square, rectangle, circle, triangle, and diamond,pentagon.
With ShapeDrawer, you can select any shape you want to draw on the canvas. Once you've made your selection, simply click the "Display Shape" button, and voila! The chosen shape will be displayed dynamically on the canvas area.
Using radio buttons makes it easy for you to choose the shape you want to draw, and the "Display Shape" button ensures that your canvas stays clutter-free by clearing any previously drawn shapes before displaying the new one.

Canvas

In Tkinter, a canvas is like a blank sheet of paper where you can draw shapes, lines, and images. It's a space where you can create and display graphical elements.
Think of it as a drawing board where you can use your imagination to create anything you want. You can draw shapes like squares, circles, and triangles, or even make your own designs.
A canvas is useful for creating interactive graphics, such as games or diagrams. You can add different elements to the canvas and make them move or change based on user input.
Overall, a canvas in Tkinter is a versatile tool that allows you to create and display visual content in your Python applications.

In Tkinter, the Canvas widget is used to create a drawing area where you can display and manipulate graphical elements such as shapes, lines, and images.

Here are some common properties of the Canvas widget that you can modify:

bg (background): Sets the background color of the canvas.
width: Sets the width of the canvas in pixels.
height: Sets the height of the canvas in pixels.
bd (borderwidth): Sets the width of the border around the canvas.
relief: Sets the border style of the canvas. It can be set to "flat" (default), "raised", "sunken", "ridge", or "groove".
scrollregion: Sets the region of the canvas that can be scrolled. It should be specified as a tuple (x1, y1, x2, y2) representing the coordinates of the top-left and bottom-right corners of the region.
highlightbackground: Sets the color of the focus highlight when the canvas does not have focus.
highlightcolor: Sets the color of the focus highlight when the canvas has focus.
highlightthickness: Sets the width of the focus highlight.
cursor: Sets the cursor type when the mouse is over the canvas. It can be set to "arrow", "circle", "cross", "hand2", "fleur", and more.
These properties allow you to customize the appearance and behavior of the canvas in your Tkinter applications according to your needs.

Solutions:

```
import tkinter as tk
from tkinter import Canvas, Radiobutton, StringVar

def display_shape():
    # Clear the canvas
    canvas.delete("all")

    selected_shape = shape_var.get()

    if selected_shape == "Square":
        canvas.create_rectangle(50, 50, 150, 150, fill="blue")
```

```
    elif selected_shape == "Rectangle":
        canvas.create_rectangle(50, 50, 200, 150, fill="green")
    elif selected_shape == "Circle":
        canvas.create_oval(50, 50, 150, 150, fill="red")
    elif selected_shape == "Triangle":
        canvas.create_polygon(100, 50, 50, 150, 150, 150,
fill="orange")
    elif selected_shape == "Diamond":
        canvas.create_polygon(100, 50, 50, 100, 100, 150, 150, 100,
fill="purple")
    elif selected_shape == "Pentagon":
        canvas.create_polygon(100, 50, 150, 100, 125, 150, 75, 150,
50, 100, fill="pink", outline="red")

# Create the main window
root = tk.Tk()
root.title("Shape Display")

# Variables
shape_var = StringVar()
shape_var.set("Square")

# Define a bold font for the label
bold_font = ("Arial", 20, "bold")

# Create a Lable
label = tk.Label(root, text="Shape Selector Panel", bg="Blue",
fg="White", font=bold_font )
label.pack()

# Create frame
shape_frame = tk.Frame(root, padx=10, pady=10)
shape_frame.pack()

# Radiobuttons for selecting the shape
square_radio = Radiobutton(shape_frame, text="Square",
variable=shape_var, value="Square")
rectangle_radio = Radiobutton(shape_frame, text="Rectangle",
variable=shape_var, value="Rectangle")
```

```
circle_radio = Radiobutton(shape_frame, text="Circle",
variable=shape_var, value="Circle")
triangle_radio = Radiobutton(shape_frame, text="Triangle",
variable=shape_var, value="Triangle")
diamond_radio = Radiobutton(shape_frame, text="Diamond",
variable=shape_var, value="Diamond")
pentagon_radio = Radiobutton(shape_frame, text="Pentagon",
variable=shape_var, value="Pentagon")

# Canvas for displaying shapes
canvas = Canvas(root, width=250, height=200, bg="white")

# Button to display the selected shape
display_button = tk.Button(root, text="Display Shape",
command=display_shape)
# Pack the widgets
square_radio.pack(side='left')
rectangle_radio.pack(side='left')
circle_radio.pack(side='left')
triangle_radio.pack(side='left')
diamond_radio.pack(side='left')
pentagon_radio.pack(side='left')
display_button.pack()
canvas.pack()

# Start the Tkinter event loop
root.mainloop()
```

Output:

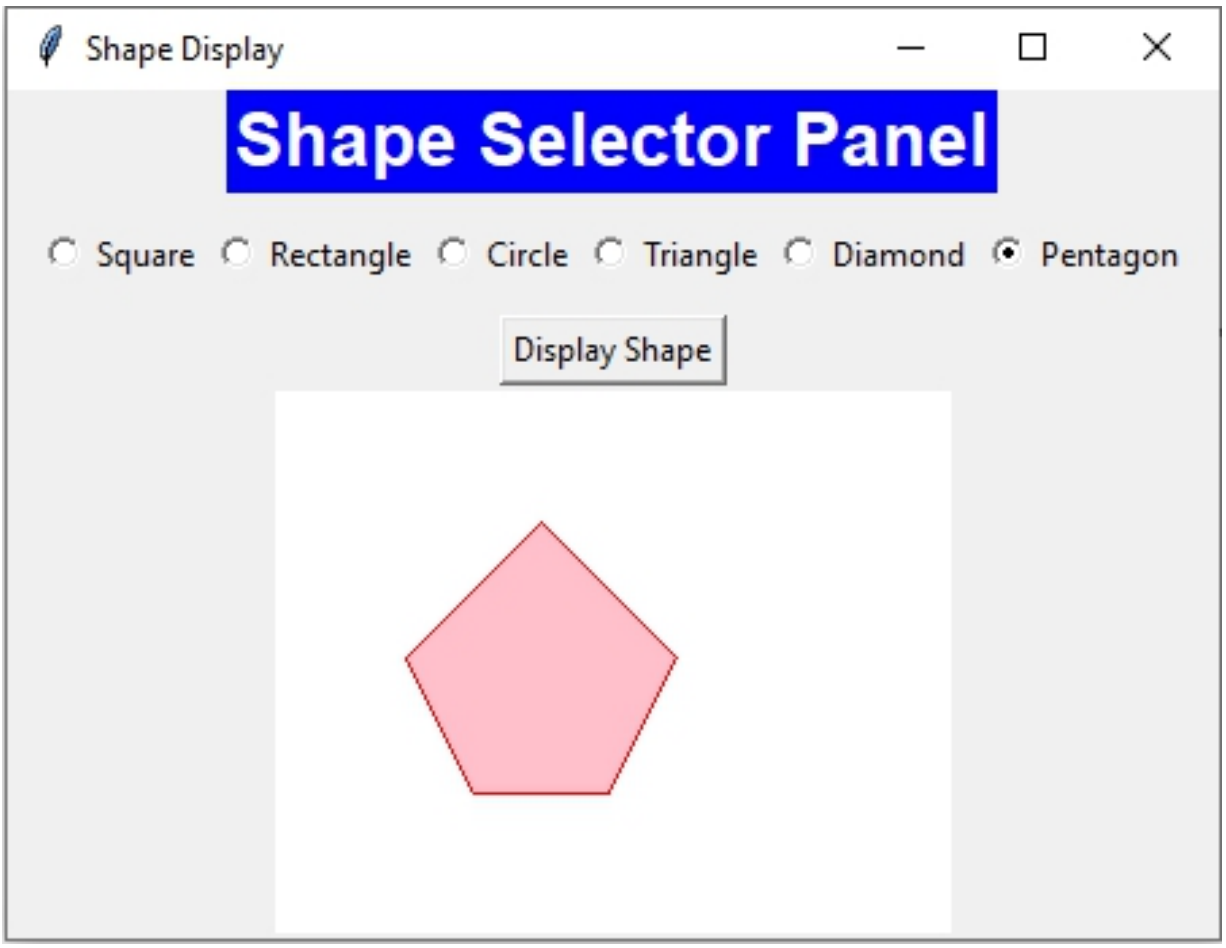
After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen. This GUI window serves as the interface for the Shape Drawer application. Here's what you'll see:

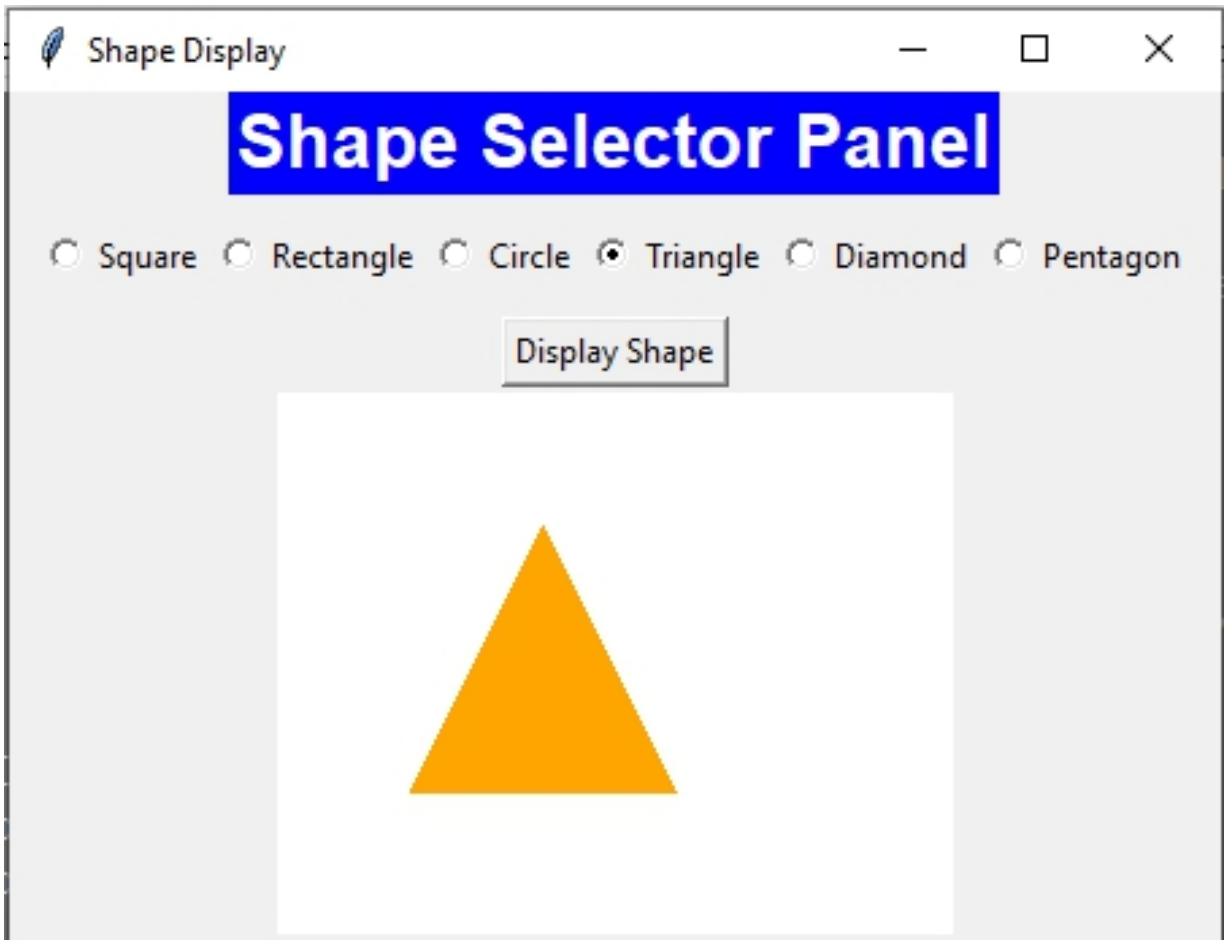
Radio Buttons:

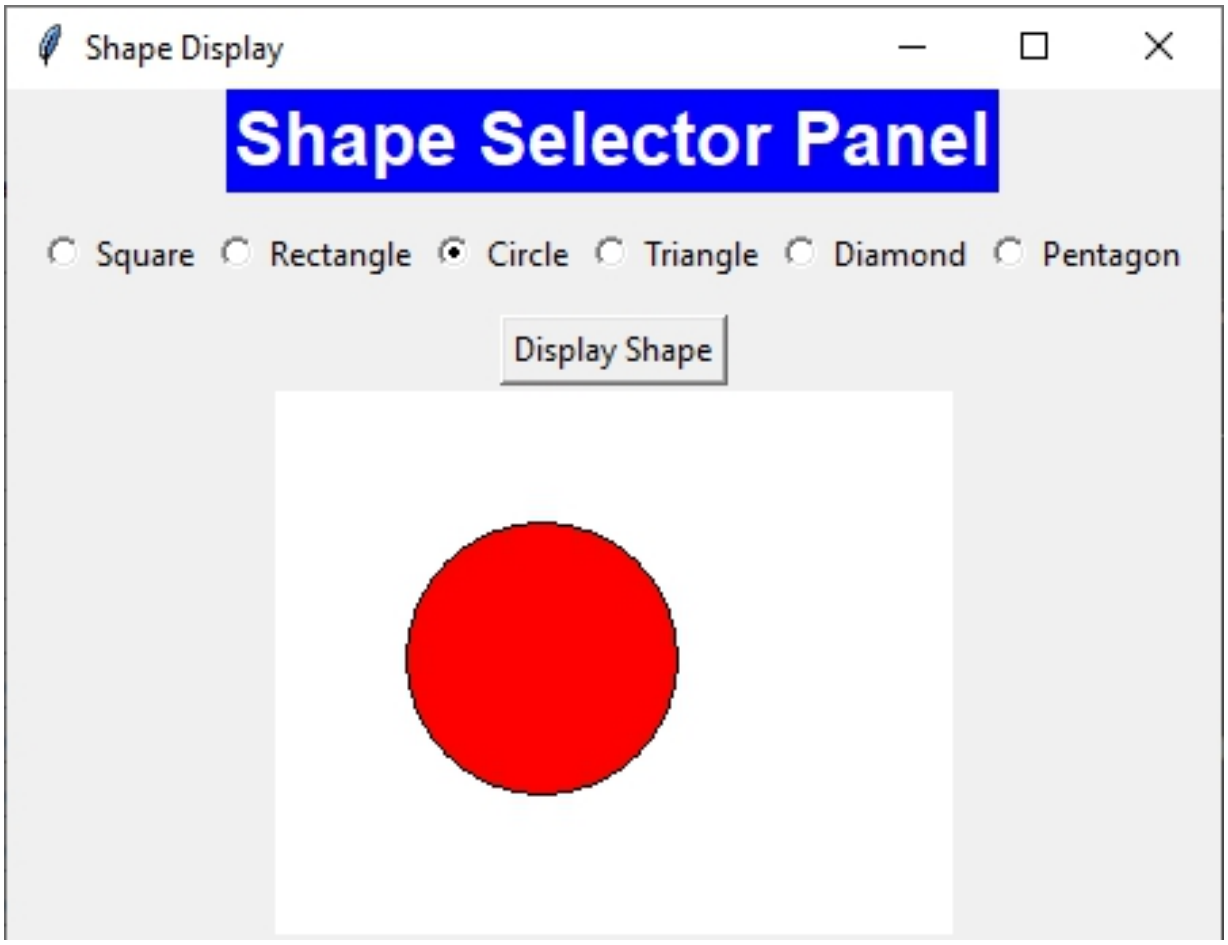
Positioned at the top of the window, you'll find six radio buttons labeled circle, rectangle, triangle, pentagon, diamond, and square. These buttons allow you to choose the shape you want to draw.

Display Shape Button:

Below the radio buttons, there's a 'Display Shape' button. Clicking this button will draw the selected shape on the canvas.
Canvas: In the center of the window, there's a canvas where the selected shape will be displayed.




Shape Display
Shape Selector Panel
Square
Rectangle
Circle
Triangle
Diamond
Pentagon
Display Shape



Simple WebPage Viewer

Project #18

Design python GUI for webpage viewer using tkinter

Introduction

Welcome to our WebPageViewer! This special program helps you see webpages in a separate window. You don't need to use your web browser. Just enter the webpage's address, click the "Open Webpage" button, and a new window will show the webpage you want. It's easy! No more switching between windows or tabs. With the WebPageViewer, you can quickly look at any webpage without any hassle. Let's start using the WebPageViewer and explore the web with ease!

Web browser

In Tkinter, a web browser is a tool that helps you look at web pages on your computer. Just like a regular browser, you can type in the address of a website, and then the browser will show you that website. You can click on links to go to different pages and do all the things you normally do on the internet. It's like having a small internet window inside your Tkinter program.

Methods:

webbrowser.open(url, new=0, autoraise=True): Opens the specified URL (e.g., https://www.google.com) in the default web browser.
new: Controls whether to open in a new window (0), new tab (1), or current window (2).
autoraise: Brings the browser window to the foreground (default: True).
This approach doesn't involve creating a web browser within your Tkinter application but instead launches the system's default browser.

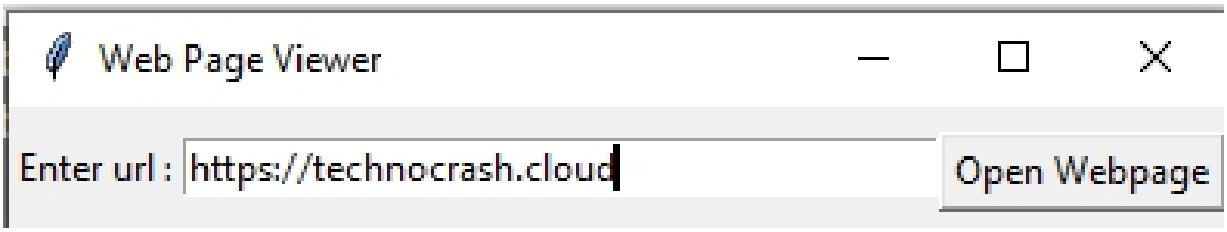
Solution:

```
import tkinter as tk
```

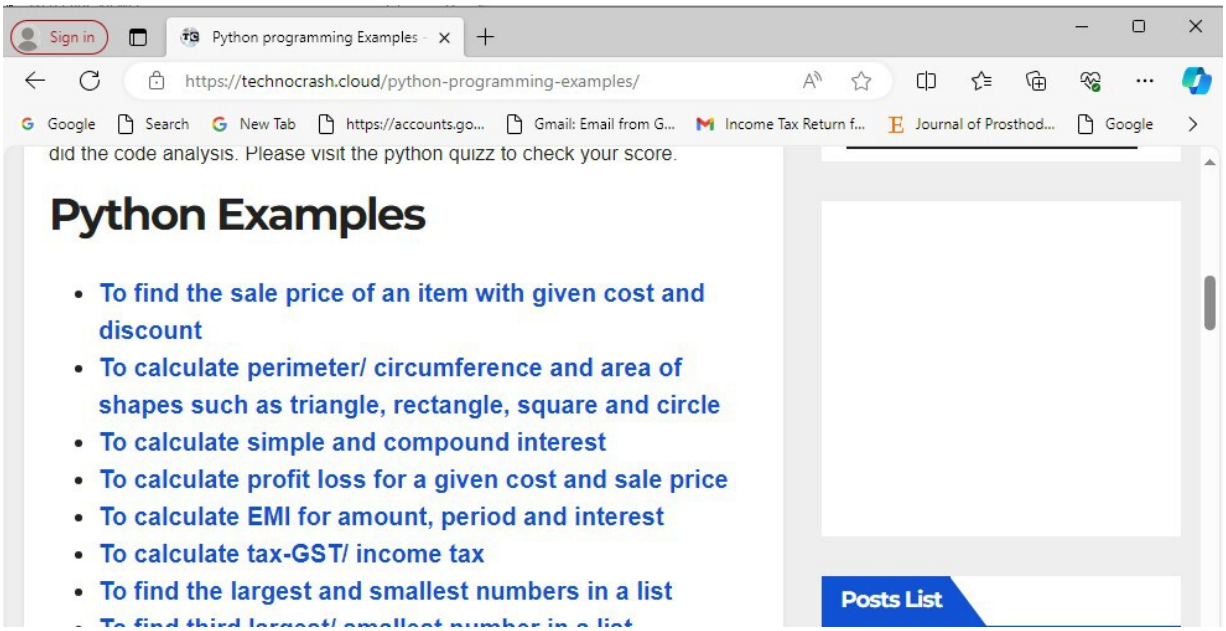
```
import webbrowser
def open_webpage():
    # Get the URL from the entry widget
    url = entry.get()
    # Open the default web browser with the specified URL
    webbrowser.open(url)
# Create the main window
root = tk.Tk()
root.title("Web Page Viewer")
# Create a Label
label = tk.Label(root, text = "Enter url :")
label.pack(side= 'left')
# Entry widget for entering URL
entry = tk.Entry(root, width=40)
entry.pack(side= 'left',pady=10)
# Button to open the webpage in the default browser
open_button = tk.Button(root, text="Open Webpage",
command=open_webpage)
open_button.pack(side= 'bottom',pady=5)
# Start the Tkinter event loop
root.mainloop()
```

Output:

After executing the above code, GUI will be displayed as below.



After entering the url of website and clicking on the ‘Open webpage’ button, following webbrowser is displayed in separate window.



Word Sorter Program

Project #19

Design Python GUI using Tkinter that sorts words in a sentence in decreasing order of their length and displays the sorted words along with their lengths

Introduction:

Welcome to our Word Sorter program! This Python GUI application, built using Tkinter, helps you sort words in a sentence based on their length. Simply enter any sentence into the provided entry field, and our program will sort the words in decreasing order of their length. Once you've entered the sentence and clicked the "Sort Words" button, the program will display the sorted words along with their respective lengths in the text box below.

Lets talk about Frame widget and see how to use it in our code

Frame widget

In Tkinter, a frame is like a box that holds other things, such as buttons or text boxes. It helps keep everything organized and neat in your window. You can think of it as a container that groups similar items together.

Frames are useful because they let you arrange your widgets, like buttons or labels, in a structured way. They make it easier to manage and organize different parts of your application. Frames can also have their own look and style, like a background color or border, to make your application more visually appealing.

In Tkinter, frames have several properties that you can customize to control their appearance and behavior.

Here are some common **frame properties:**

Background (bg): Sets the background color of the frame.
Borderwidth (bd): Sets the width of the border around the frame.
Relief: Determines the appearance of the border. It can be set to "flat", "raised", "sunken", "groove", or "ridge".
Width (width): Sets the width of the frame.
Height (height): Sets the height of the frame.

Padding (padx, pady): Adds padding around the inside edges of the frame.
Cursor: Sets the mouse cursor appearance when it is over the frame.
Highlightbackground: Sets the color of the focus highlight when the frame does not have focus.
Highlightcolor: Sets the color of the focus highlight when the frame has focus.
Highlightthickness: Sets the width of the focus highlight.

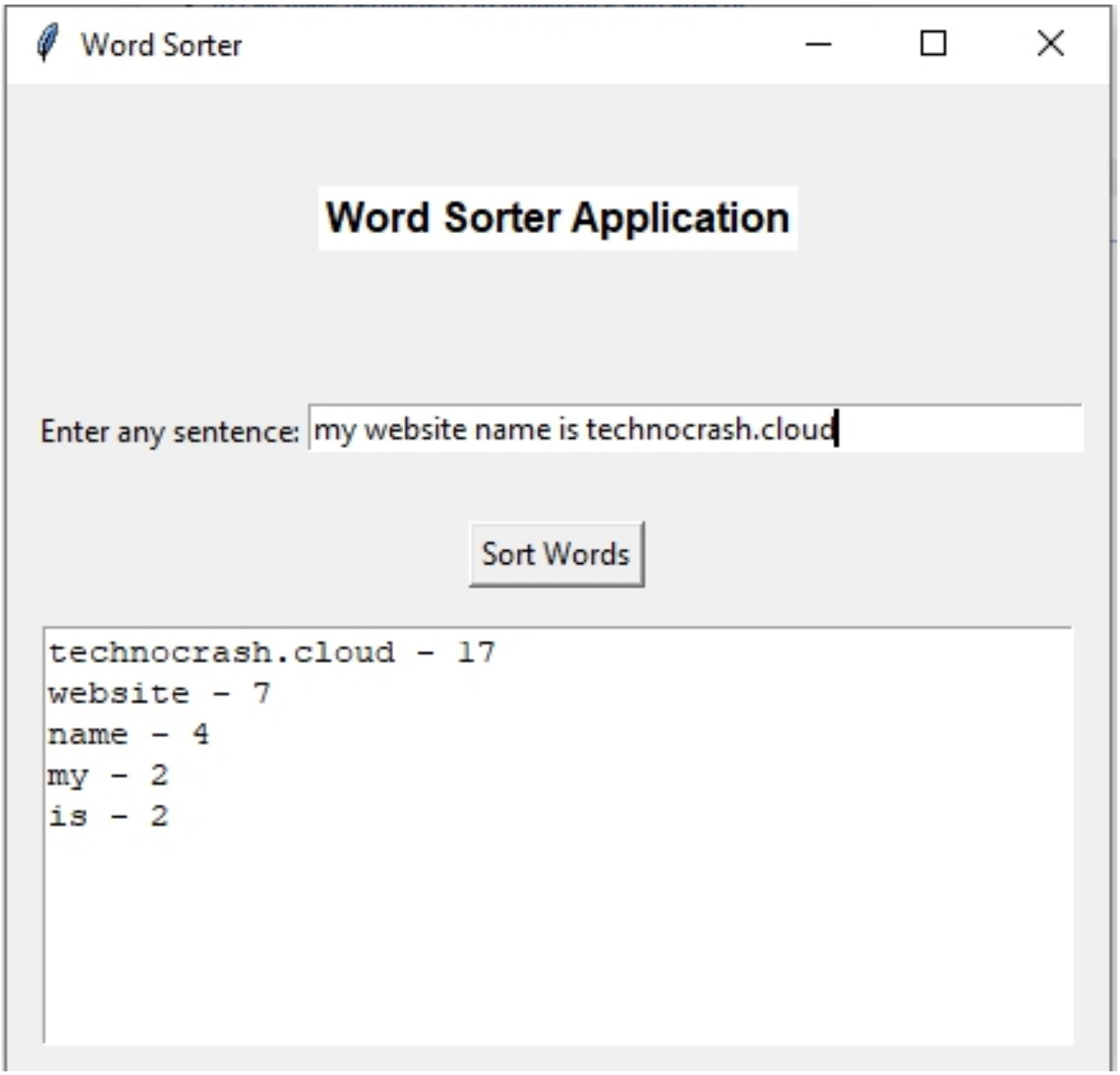
Solutions:

```
import tkinter as tk
def sort_and_display():
    sentence = entry.get()
    words = sentence.split()
    # Sort words by length in decreasing order
    sorted_words = sorted(words, key=lambda x: len(x),
reverse=True)
    # Display sorted words and their lengths
    result_text.delete(1.0, tk.END)  # Clear previous result
    for word in sorted_words:
        result_text.insert(tk.END, f"{word} - {len(word)}\n")

# Create the main window
root = tk.Tk()
root.title("Word Sorter")

# Define a bold font style and font name for color_label
bold_font = ("Arial", 12, "bold")

# Create a label to provide instructions
title_label = tk.Label(root, text="Word Sorter Application",bg =
'white',font = bold_font)
title_label.pack(pady=40)

# Create a frame to contain color buttons
label_frame = tk.Frame(root, padx=10, pady=10)
label_frame.pack()

# Create a Lable to provide instructions
```

```
label = tk.Label(label_frame, text="Enter any sentence:")
label.pack(side= 'left',pady=10)

# Entry widget for entering a sentence
entry = tk.Entry(label_frame, width=50)
entry.pack(pady=10)

# Button to trigger sorting and displaying
sort_button = tk.Button(root, text="Sort Words",
command=sort_and_display)
sort_button.pack( pady=5)

# Text widget to display the result
result_text = tk.Text(root, width=50, height=10)
result_text.pack(side='bottom',pady=10,padx=10)

# Start the Tkinter event loop
root.mainloop()
```

Output:

After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This window features an input field where you can enter any sentence, a 'Sort words' button to trigger the word sorting based on their lengths, and a text box below where the sorted words along with their respective lengths will be displayed.



Bank Account Simulator

Project #20

Design Python GUI using Tkinter to simulate a bank account with support for depositing money, withdrawing money, and checking the balance.

Introduction:

Welcome to our Bank Account Simulator! This easy-to-use Python GUI application, powered by Tkinter, puts you in control of your virtual bank account. With just a few clicks, you can deposit money, withdraw money, and check your remaining balance. Our Bank Account Simulator features three simple buttons: "Deposit", "Withdraw", and "Show Balance". When you deposit or withdraw money, the application instantly updates the label to show your remaining balance. It's like having your own personal bank right on your computer screen! Lets get started.

Solutions:

```
import tkinter as tk
from tkinter import messagebox
# Initial balance
balance = 500
def deposit():
    global balance
    try:
        amount = float(amount_entry.get())
        if amount > 0:
            balance += amount
            update_balance_label()
        else:
            messagebox.showerror("Error", "Please enter a
valid    positive amount.")
    except ValueError:
```

```
        messagebox.showerror("Error", "Please enter a valid numeric
amount.")
def withdraw():
    global balance
    try:
        amount = float(amount_entry.get())
        if amount > 0 and amount <= balance:
            balance -= amount
            update_balance_label()
        elif amount <= 0:
            messagebox.showerror("Error", "Please enter a valid
positive amount.")
        else:
            messagebox.showerror("Error", "Insufficient funds.")
    except ValueError:
        messagebox.showerror("Error", "Please enter a valid numeric
amount.")
def show_balance():
    messagebox.showinfo("Balance", f"Your balance is: ${balance}")
def update_balance_label():
    balance_label.config(text=f"Balance: ${balance}")
# Create the main window
root = tk.Tk()
root.title("Bank Account Simulator")
# Define bold font for title label
font_bold = ("Times New Roman",25,"bold")
# Create title Label
title_label = tk.Label(root, text ="World Bank", font = font_bold)
title_label.pack()
# Label to display balance
balance_label = tk.Label(root, text=f" Your Remaining Balance:
${balance}")
balance_label.pack(pady=10)
# Create a frame to contain label and entry widget
label_frame = tk.Frame(root, padx=10, pady=10)
label_frame.pack()
# Label to provide instruction
```

```
label = tk.Label(label_frame, text= "Enter amount to deposit or
withdraw :")
label.pack(side= 'left')
# Entry widget for amount
amount_entry = tk.Entry(label_frame, width=20)
amount_entry.pack(pady=5)
# Buttons for deposit, withdraw, and show balance
deposit_button = tk.Button(root, text="Deposit", command=deposit)
deposit_button.pack(side=tk.LEFT, padx=5)
withdraw_button = tk.Button(root, text="Withdraw",
command=withdraw)
withdraw_button.pack(side=tk.LEFT, padx=5)
balance_button = tk.Button(root, text="Show Balance",
command=show_balance)
balance_button.pack(side=tk.LEFT, padx=5)
# Start the Tkinter event loop
root.mainloop()
```

Output:

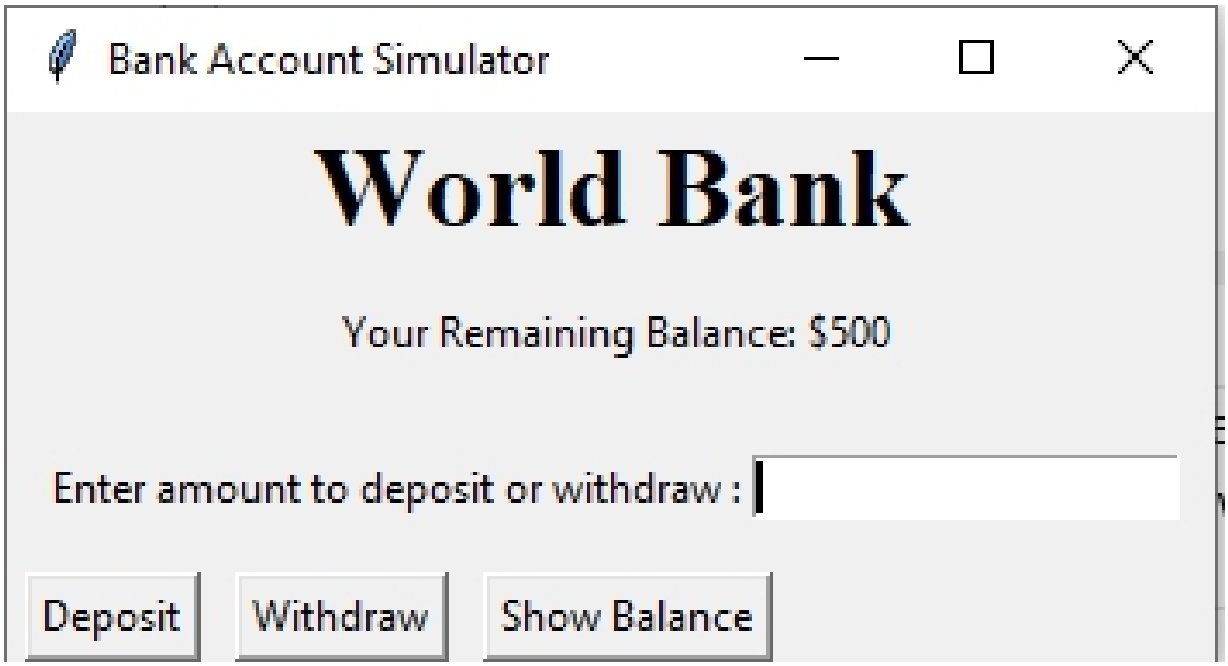
Upon executing the above code, GUI for bank simulator will be displayed on your screen. This window comprise the label which is positioned at the top of the window, indicating the current balance. This label dynamically updates to reflect changes in the account balance resulting from deposit or withdrawal transactions. Below the label, there's an input field where users can enter the amount they wish to deposit or withdraw.

Buttons:

Deposit Button: Beneath the input field, there's a 'Deposit' button. Users can click this button to deposit the entered amount into their account.

Withdraw Button: Following the deposit button, there's a 'Withdraw' button. Clicking this button allows users to withdraw the entered amount from their account.

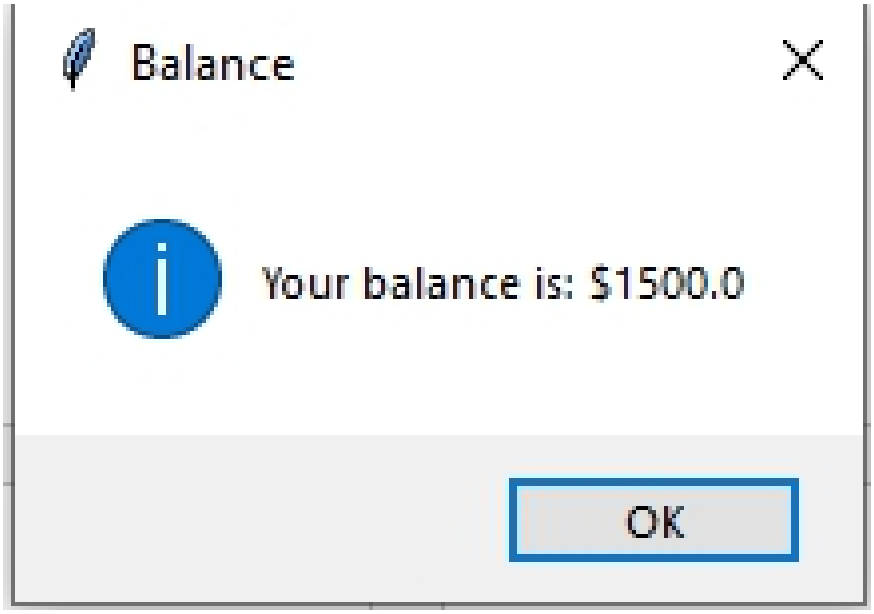
Show Balance Button: Positioned next to the withdraw button, there's a 'Show Balance' button. Clicking this button triggers a messagebox displaying the current balance of the account.



Above window shows remaining bank balance of user

After clicking the ‘deposit’ button, current balance label is automatically changes as above.

After clicking the withdraw button, left over bank balance of user is shown as above.



After clicking the “Show balance” button, above messaebox is displayed.

Yearly Calender Design Program

Project #21

Project: Design python GUI application for calendar of specific year using tkinter

Introduction:

Welcome to our Yearly Calendar Design program! In this Tkinter application, we'll create a tool to help you display a calendar for any given year. Calendars are useful for planning events, scheduling appointments, and organizing your time effectively. With Tkinter, we'll design an interface where you can enter the year for which you want to see the calendar. Then, with a click of the "Show Yearly Calendar" button, you'll see the calendar displayed in a text box. We'll automatically add horizontal and vertical scrollbars to the text box to ensure you can easily view the entire calendar. This application will be helpful for anyone who needs to quickly reference a calendar for planning purposes. So, let's get started and build this convenient Yearly Calendar Design tool together!

Calender module

To design calendar, we need to import calendar module in our program using import statement. Calendar module is part of Python standard library and it is easily available to use in program without needing to install external packages. This module provides various function for displaying and manipulating calenders. It does not have direct interaction with Tkinter, so you can use this calendar module to generate text calendar that can be then displayed within Tkinter labels, text areas, or other widgets.

Solution:

```
import calendar
import tkinter as tk
from tkinter import *

def printcalendar():
```

```
    year = int(year_entry.get())
    cal_text = calendar.TextCalendar().formatyear(year, 2, 1, 1, 3)
    result_text.delete(1.0, END)
    result_text.insert(INSERT, cal_text)

# Driver program
root = tk.Tk()
root.title("Yearly Calendar")
root.geometry("600x400")

# Calendar frame
calendar_frame = tk.Frame(root)
calendar_frame.pack(pady=10)

# Year dropdown
year_label = tk.Label(calendar_frame, text="Year:")
year_label.grid(row=0, column=0, padx=5)

year_entry = tk.Entry(calendar_frame)
year_entry.grid(row=0, column=1, padx=5)

# Button to show the calendar
show_button = tk.Button(calendar_frame, text="Show Yearly
Calendar", command=printcalendar)
show_button.grid(row=0, column=2, padx=5)

# Result text widget with scrollbar
result_text_frame = tk.Frame(root)
result_text_frame.pack(pady=10)

result_text = tk.Text(result_text_frame, height=20, width=55,
wrap="none")
result_text.grid(row=0, column=0)

# Scrollbar for the text widget
scrollbar = tk.Scrollbar(result_text_frame,
command=result_text.yview)
scrollbar.grid(row=0, column=1, sticky='nsew')
result_text['yscrollcommand'] = scrollbar.set
scrollbar = tk.Scrollbar(result_text_frame,
command=result_text.xview, orient="horizontal")
```

```
scrollbar.grid(row=1, column=0, sticky='nsew')
result_text['xscrollcommand'] = scrollbar.set

root.mainloop()
```

Output:

Upon executing the Yearly Calendar application, you will encounter a graphical user interface (GUI) window designed for displaying calendars of a specified year. Here's you will see

Input Field:

At the top of the window, there's an input field where users can enter the year for which they want to view the calendar.

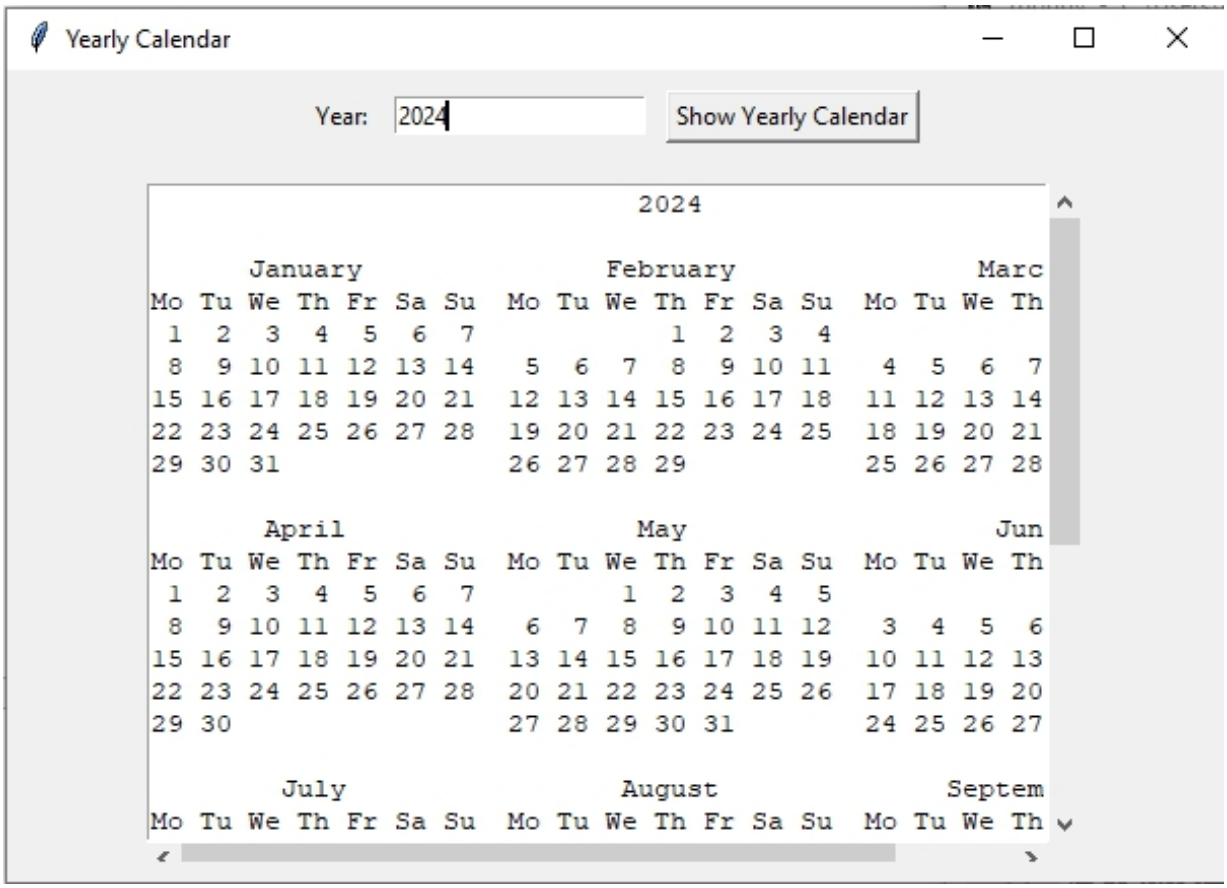
Show Yearly Calendar Button:

Following the input field, there's a button labeled 'Show Yearly Calendar'. Clicking this button triggers the display of the calendar for the entered year.

Text Widget:

Below the 'Show Yearly Calendar' button, there's a text widget. Once the user clicks the button, the yearly calendar for the specified year will be displayed in this text widget.

The text widget is equipped with both horizontal and vertical scrollbars to facilitate navigation through the calendar.



Above GUI shows the yearly calendar for 2024 as above.

Captcha Generator and Verifier

Project #22

Design Python GUI application for captcha generator and verifier using tkinter

Introduction:

Welcome to our Captcha Generator and Verifier! This Python GUI application, built with Tkinter and PILLOW, provides a simple solution for generating and verifying CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart). With just a click of a button, our application generates a unique CAPTCHA image using advanced techniques from Pillow's ImageDraw, ImageFilter, and ImageFont modules. These modules allow us to create captivating CAPTCHA images with intricate patterns and text, ensuring that they are challenging for automated bots to decipher but easy for humans to read. Once the CAPTCHA is generated, users can view the image and enter the displayed text into the verification field. Our application instantly verifies the entered CAPTCHA, distinguishing between human users and automated bots with remarkable accuracy.

To create this program, first we need to install pillow library if you haven't already:
pip install pillow

When you want to add cool effects like drawing shapes or text on images, or make them look different, you use Pillow with Tkinter. Here's what each part does:

Here's how you can use Pillow's ImageDraw, ImageFont, and ImageFilter modules with Tkinter:

ImageDraw: The ImageDraw module allows you to draw shapes and text on images. You can create an ImageDraw object and then use methods like line(), rectangle(), text(), etc., to draw on the image.
ImageFont: The ImageFont module provides tools for loading and using fonts in images. You can load a font file using the truetype()

function and then use the loaded font to draw text on images with the ImageDraw module.
ImageFilter: The ImageFilter module contains various image filters that you can apply to images. Filters like blur, contour, sharpen, etc., can be applied to enhance or modify the appearance of images.
If you're using ImageDraw, ImageFilter, and ImageFont from Pillow to manipulate images and add text or effects to them, you'll need to import these specific modules along with Pillow:

from PIL import Image, ImageDraw, ImageFilter, ImageFont

Photoimage: PhotoImage is a class in the Tkinter module that represents images. It allows you to display images within your Tkinter GUI applications. PhotoImage supports GIF, PGM, PPM, and PNG image file formats. It does not support JPEG images directly, but you can use the PIL (Python Imaging Library) module to convert JPEG images to other supported formats like GIF or PNG. You can't directly pass an image path to a Tkinter widget like a label. Instead, you need to convert the image into a PhotoImage object first.

now, you can create this application

Solution

```
import tkinter as tk
import random
from PIL import Image, ImageDraw, ImageFont, ImageFilter
from PIL import ImageTk
from tkinter import messagebox

def generate_captcha():
    # Generate a random 4-digit CAPTCHA code
    captcha_code =
''.join(random.choices('0123456789ABCD$@#%^&EFGHIJKLMNOP
+_)(*&qsertyuhgt', k=4))

    # Create an image with the CAPTCHA code
    image = Image.new('RGB', (150, 50), color='white')
    draw = ImageDraw.Draw(image)
```

```
    font = ImageFont.truetype('arial.ttf',22)
    draw.text((10, 10), captcha_code, fill='blue', font=font)

    # Apply a slight blur to the image
    image = image.filter(ImageFilter.EMBOSS)

    # Convert the image to a Tkinter PhotoImage
    tk_image = ImageTk.PhotoImage(image)

    # Update the label to display the CAPTCHA image
    captcha_label.config(image=tk_image)
    captcha_label.image = tk_image  # Keep a reference to avoid
garbage collection

    # Set the global variable to store the correct CAPTCHA code
    global correct_captcha
    correct_captcha = captcha_code

#
def verify_captcha():
    # Get the entered CAPTCHA code
    entered_captcha = captcha_entry.get()

    # Check if the entered CAPTCHA is correct
    if entered_captcha == correct_captcha:
        messagebox.showinfo("Success", "CAPTCHA verification
successful!")
    else:
        messagebox.showerror("Error", "CAPTCHA verification failed.
Try again.")

# Create the main window
root = tk.Tk()
root.title("CAPTCHA Generator")

# Set font
bold_font = ("Times New Roman", 20, "bold")

# Create a Label
title_label = tk.Label(root, text = "Captcha Generator and Verifier",
font=bold_font, bg="Yellow", fg="red")
title_label.pack()
```

```
# Create and place widgets
captcha_label = tk.Label(root)
captcha_label.pack(pady=10)

generate_button = tk.Button(root, text="Generate CAPTCHA",
command=generate_captcha)
generate_button.pack(pady=5)

default_text = "Enter captcha here....."
captcha_entry = tk.Entry(root, fg="grey")
captcha_entry.insert(0, default_text)
captcha_entry.pack(pady=5)

verify_button = tk.Button(root, text="Verify CAPTCHA",
command=verify_captcha)
verify_button.pack(pady=10)

# Initialize the correct_captcha variable
correct_captcha = ""

# Start the Tkinter event loop
root.mainloop()
```

Output:

Upon launching the Captcha Generator and Verifier application, you'll encounter a graphical user interface (GUI) window designed for generating and verifying captcha images. Here you wiil see:

Generate Captcha Button:

Positioned at the top of the window, there's a button labeled 'Generate Captcha'. Clicking this button triggers the generation of a captcha image.

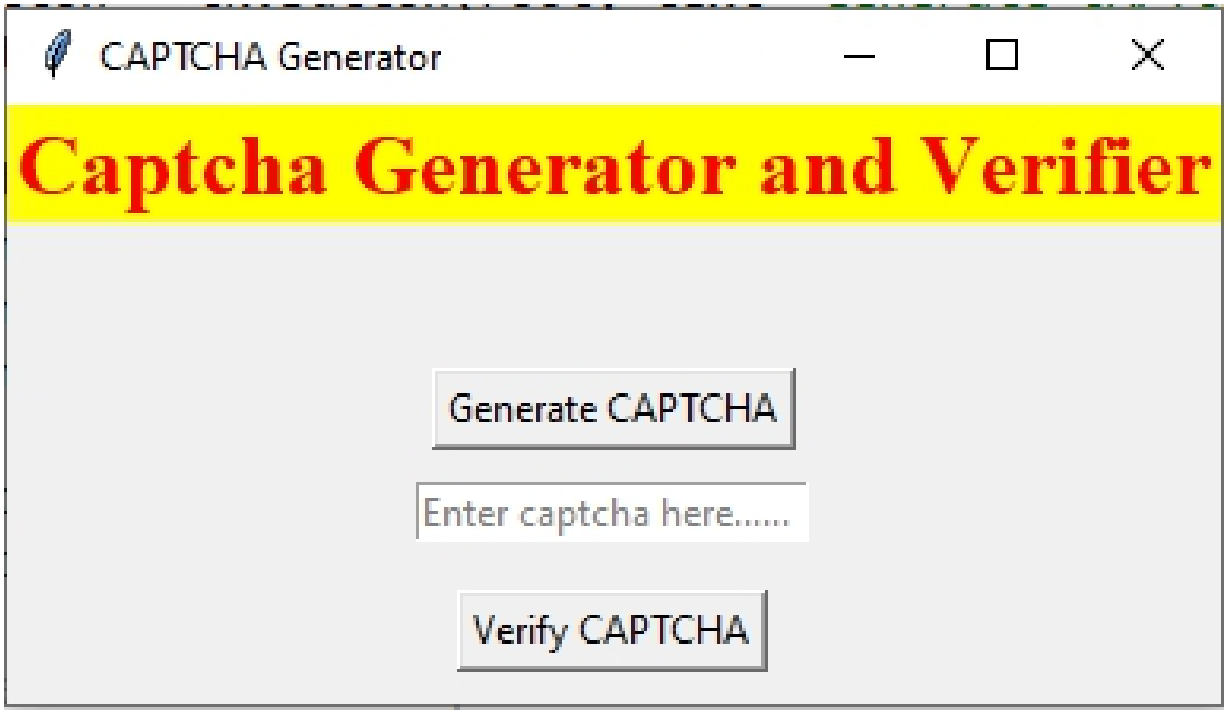
Captcha Image:

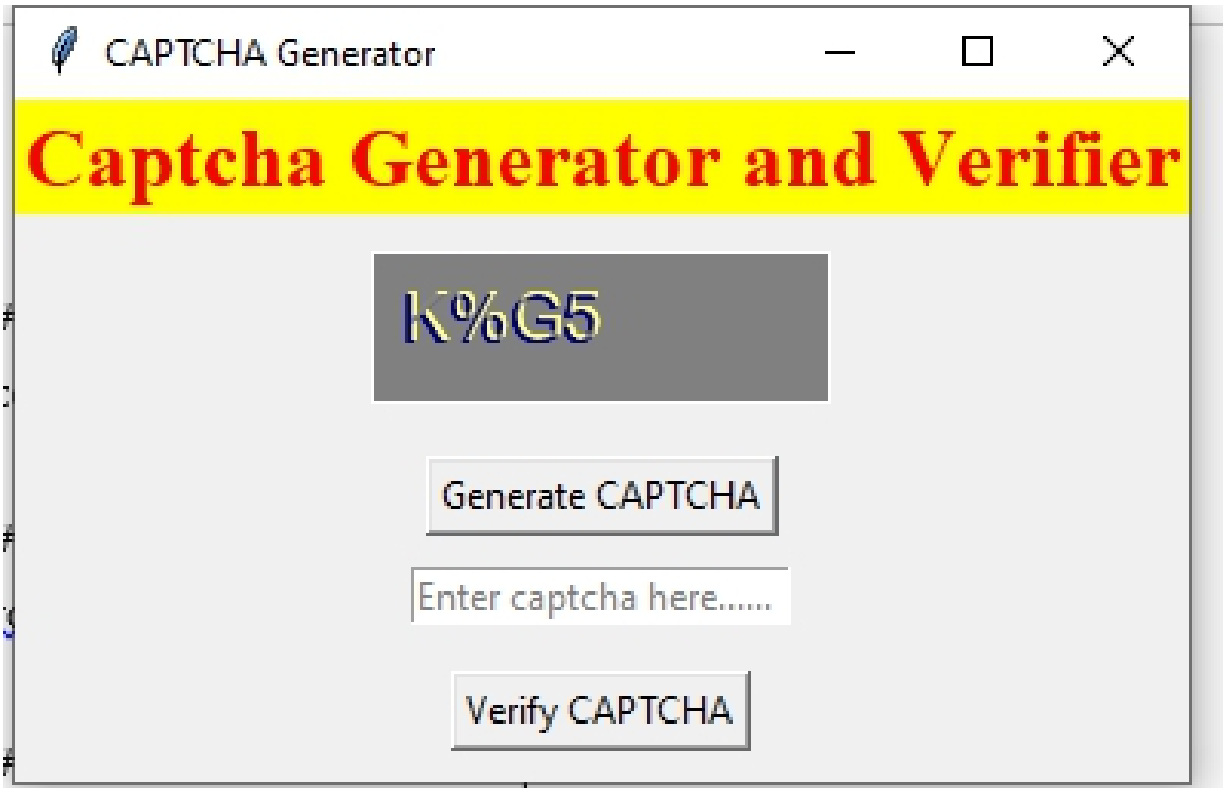
Above the 'Generate Captcha' button, there's an area where the generated captcha image is displayed. The image appears directly above the button.

Entry Field:

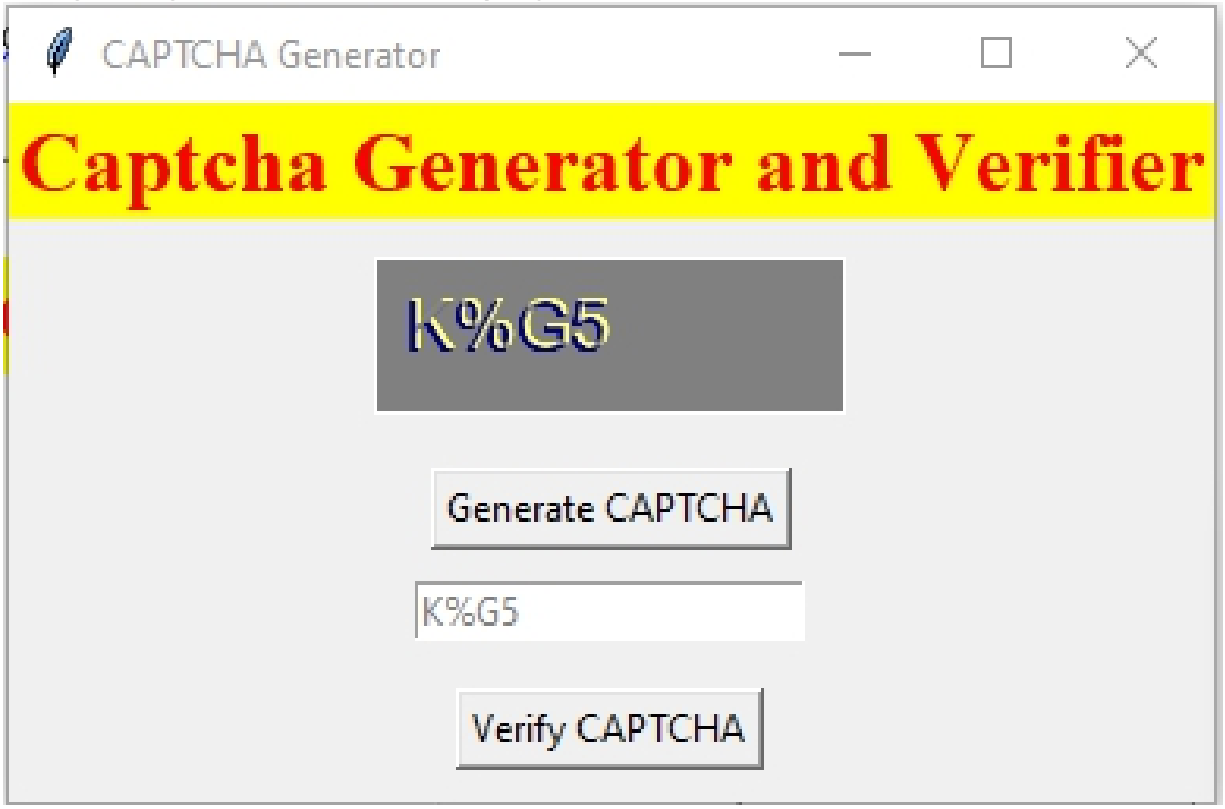
Following the captcha image, there's an entry field where users can enter the captcha text they see in the generated image. This entry field allows users to input their captcha response.

Verify Captcha Button:
Below the entry field, there's a button labeled 'Verify Captcha'. Clicking this button initiates the verification process to check if the entered captcha text matches the generated captcha. The verification result is then displayed to the user.

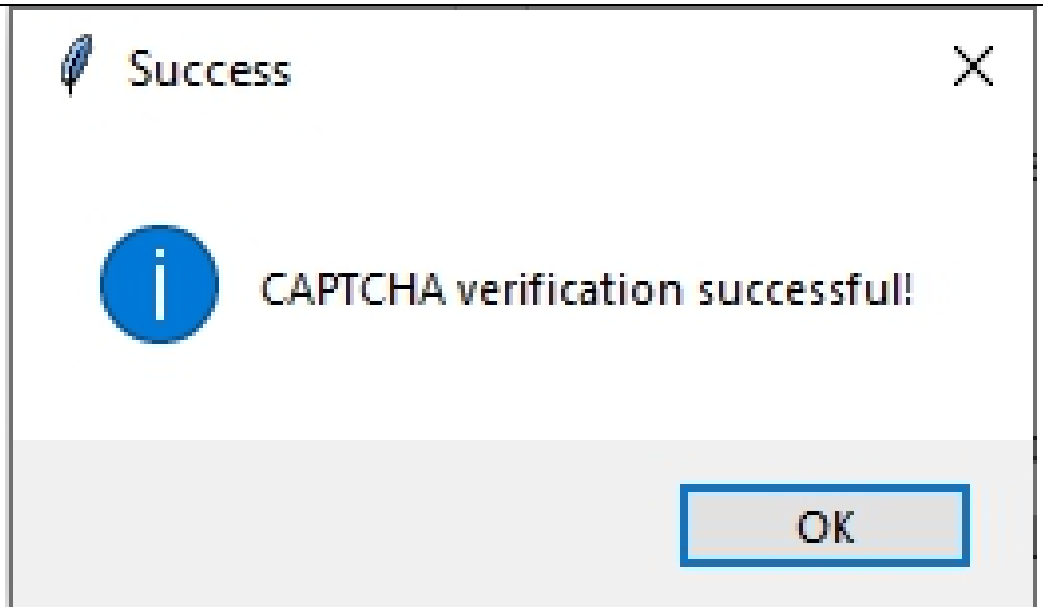




After clicking the “Generate CAPTCHA “button, above captcha image is generated and displayed above the button



After entering the correct captcha text into the entry field, click on the 'Verify Captcha' button. Subsequently, a messagebox will appear, indicating the result of the verification process.



Book QR Code Generator

Project #23

Design Pythn GUI application for Book QR Code Generator using tkinter

Introduction

Welcome to the Book QR Code Generator! This tool helps you make special codes for your books. Whether you're a librarian, a bookstore owner, or just someone who loves books, this app makes it easy to create QR codes for your books.

With the Book QR Code Generator, you can make QR codes by typing in the book's title, the author's name, and the ISBN number. Then, just click a button, and the QR code will appear.

QR codes are like digital shortcuts. They let you store information in a small picture that you can scan with a smartphone or QR code reader. So, when you scan the QR code we make, you'll quickly see details about the book.

To generate QR code for our project, we need to install qrcode module.

pip install qrcode

With the qrcode module, you can easily create QR codes programmatically in Python.

qrcode module

The qrcode module in Python is a popular library used for generating QR codes. QR codes, short for Quick Response codes, are two-dimensional barcodes that can store various types of information, such as text, URLs, contact information, or other data.

We will use qrcode module methods to generate qr code for our program. Lets see what are they

1. qrcode.QRCode(version, error_correction, box_size, border, data_mask):

This is the core function used to create a QR code object.

version: Controls the size of the code (1 being the smallest, 40 the largest).
error_correction: Specifies the level of error tolerance (L, M, Q, H).
box_size: Defines the size of each module (pixel size) in the code.
border: Adds a white border around the code (default is 4 modules).
data_mask: Applies a bit mask for further error correction (optional).
2. add_data(data):
This method adds the data you want to encode in the QR code. It supports various data types like text, URLs, and binary data.
3. make(fit=True):
This method generates the QR code image based on the provided data and settings.
Fit: If True, ensures the entire data fits even if a smaller version could be used (default is True).
make_image(fill_color, back_color):
This method converts the QR code data into an image object.
Fill_color: Defines the color of the data modules (default is black).
Back_color: Defines the background color of the image (default is white).
4.clear():
This method removes all previously added data from the QR code object.

Solutions:

```
#Book QR code generator using Python Tkinter
import tkinter as tk
from tkinter import messagebox
import qrcode
def generate_qr_code():
    # Get book information
    title = book_title_entry.get()
    author = author_entry.get()
    isbn = isbn_entry.get()
    # Combine book information into a single string
    book_info = f" Book Title: {title}\nAuthor: {author}\nISBN: {isbn}"
    # Generate QR code
    qr = qrcode.QRCode(version=1,
error_correction=qrcode.constants.ERROR_CORRECT_L,
```

```
box_size=10, border=4)
    qr.add_data(book_info)
    qr.make(fit=True)
    # Display QR code
    qr_img = qr.make_image(fill_color="black", back_color="white")
    qr_img.show()
# Create tkinter window
root = tk.Tk()
root.title("Book QR Code Generator")
# Create title label for form
title_label = tk.Label(root, text="Book QR Code Generator", font=
("Helvetica", 20, "bold"), bg="Yellow", fg="Navy blue")
title_label.grid(row=0, column=0, columnspan=2, padx=5, pady=5 )
# Create labels and entry fields for book information
book_title_label = tk.Label(root, text="Book Title:", font=20)
book_title_label.grid(row=1, column=0, padx=5, pady=5)
book_title_entry = tk.Entry(root)
book_title_entry.grid(row=1, column=1, padx=5, pady=5)
author_label = tk.Label(root, text="Author:", font=20)
author_label.grid(row=2, column=0, padx=5, pady=5)
author_entry = tk.Entry(root)
author_entry.grid(row=2, column=1, padx=5, pady=5)
isbn_label = tk.Label(root, text="ISBN:", font=20)
isbn_label.grid(row=3, column=0, padx=5, pady=5)
isbn_entry = tk.Entry(root)
isbn_entry.grid(row=3, column=1, padx=5, pady=5)
# Create generate button
generate_button = tk.Button(root, text="Generate QR Code", font=
("Arial",18,"bold"), command=generate_qr_code)
generate_button.grid(row=4, column=0, columnspan=2, padx=5,
pady=5)
# Run the tkinter event loop
root.mainloop()
```

Output:

After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This GUI window serves as the

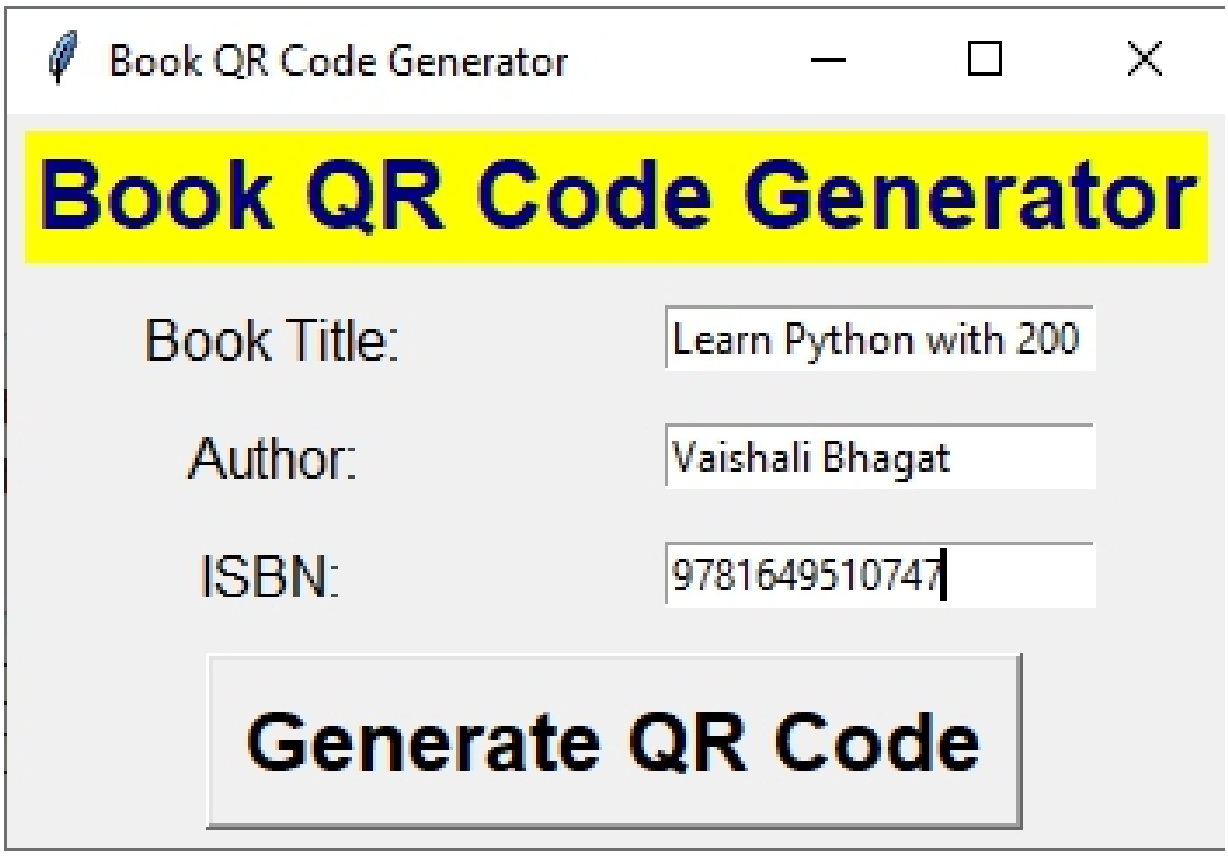
interface for the Book QR Code Generator application. Here's what you can expect to see:

Input Fields:

At the top of the window, there are input fields where users can enter the book title, author name, and ISBN number.

Generate QR Code Button:

Below the input fields, there's a button labeled 'Generate QR Code'. Clicking this button initiates the process of generating a QR code for the entered book information. Users can scan this QR code to access information about the book using a QR code reader.



Here is the generated QR code

Book barcode generator

Project #24

Design Pythn GUI application for Book barcode Generator using tkinter

Introduction

Welcome to the Book Barcode Generator! This tool helps you create special codes for your books. With the Book Barcode Generator, you can make barcodes by typing in the book's ISBN number. Then, just click a button, and the barcode will appear.
Barcodes are like digital tags. They store information about the book, making it easy to keep track of it. So, when you use the barcode we make, you'll quickly identify your book.

To generate barcodes for book, we need to install barcode module in our program.
pip install python-barcode

The barcode module in Python is a versatile library used for generating various types of barcodes. With this module, you can create barcodes for different purposes such as product labeling, inventory management, or document tracking.
The barcode module supports various barcode types such as Code 39, Code 128, EAN-13, UPC-A, and many others.
IN this program, we will use EAN-13 barcode type. The EAN-13 barcode is a widely used barcode standard for encoding unique identifiers for products. It is commonly found on retail products worldwide. An EAN-13 barcode is a special code made up of 13 numbers. People use it for different reasons, like keeping track of things in a store, identifying products, and making sales at the checkout. These barcodes are easy for barcode scanners to read, which helps stores quickly and correctly collect information.

To generate image file, we need to import ImageWriter from barcode.writer.
from barcode.writer import ImageWriter

This line allows you to use the ImageWriter class in your code to save generated barcodes as image files.
The "ImageWriter" is a special tool in the barcode module for Python. It helps you save barcodes you make as picture files. With the ImageWriter, you can choose things like what type of picture file to save (like PNG or JPEG), how clear the picture should be, and other details about how the barcode looks.

Solution:

```
import tkinter as tk
from tkinter import messagebox
# import EAN13 from barcode module
from barcode import EAN13
# import ImageWriter to generate an image file
from barcode.writer import ImageWriter

def generate_barcode():
    # Get ISBN number from user
    isbn = entry.get()
    # Check length of ISBN
    if len(isbn) != 13:
        messagebox.showerror("Error", "ISBN must be 13 digits
long!")
        return

    try:
        # Now, let's create an object of EAN13 class and
        # pass the number with the ImageWriter() as the writer
        ean = EAN13(isbn, writer=ImageWriter())
        # Our barcode is ready. Let's save it.
        barcode = ean.save('barcoder')
        messagebox.showinfo("Success", f"Barcode generated
successfully! Saved as {barcode}")
    except Exception as e:
        messagebox.showerror("Error", str(e))

# Create tkinter window
root = tk.Tk()
root.title("Book Barcode Generator")
```

```
# Create title label for form
title_label = tk.Label(root, text="Book Barcode Generator", font=
("Arial",20,"bold"), bg="black", fg="white")
title_label.pack()
# Create labels and entry fields for book ISBN
label = tk.Label(root, text="Enter ISBN", font=("Arial", 12))
label.pack()

entry = tk.Entry(root)
entry.pack(pady=5)
# Create generate button
generate_button = tk.Button(root, text="Generate Barcode", font=
("Arial", 12),command=generate_barcode)
generate_button.pack()
# Run the tkinter event loop
root.mainloop()
```

Output:

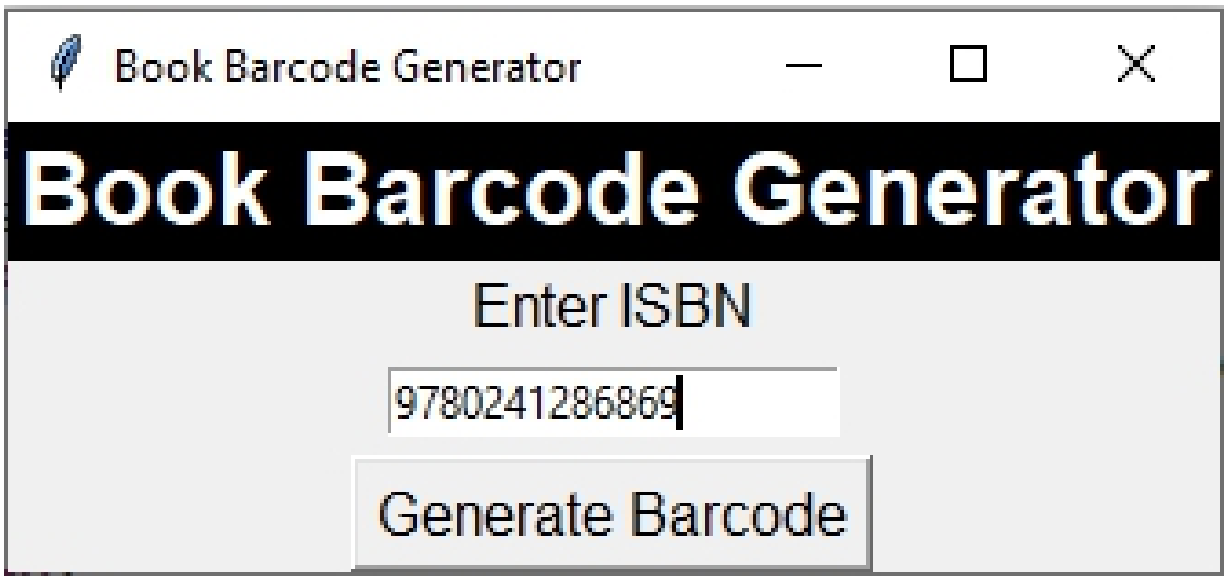
After executing the provided code, a graphical user interface (GUI) window will appear on your screen. This GUI window serves as the interface for the Book Barcode Generator application. Here's what you can expect to see:

Input Field:

At the top of the window, there's an input field where users can enter the ISBN number of the book.

Generate Barcode Button:

Below the input field, there's a button labeled 'Generate Barcode'. Clicking this button initiates the process of generating a barcode for the entered ISBN number. The generated barcode for the book's ISBN number will be displayed in separate window on the click of button. Users can scan this barcode to access information about the book using a barcode scanner.



Here is the generated barcode for book:

Currency converter

Project #25

Design Pythn GUI application for Currency Convertor using tkinter

Introduction:

Welcome to the Currency Converter! This program simplifies the task of converting currency amounts from one currency to another. With the Currency Converter, you can easily enter the amount you want to convert, select the source currency, choose the target currency from the dropdown menu, and with a simple click of the "Convert" button, the program will fetch the latest exchange rate from an API and display the converted amount. Using the request module, our converter always gets the newest exchange rates. This means you can trust the converted amount to be accurate. The program has an easy layout, so you can use it without any trouble. Once you convert, you'll see the result in a popup message.

To fetch newest currency rate, we need to install requests library. We can install this via pip

pip install requests

With the requests library, you can easily interact with web servers and retrieve data from web APIs to perform tasks such as web scraping, data retrieval, or API integration in your Python projects.

Solution:

```
import tkinter as tk
from tkinter import*
from tkinter import ttk
from tkinter import messagebox
import requests

def convert_currency():
    try:
        # Get the amount to convert and the selected currencies
        amount = float(amount_entry.get())
```

```
        from_currency = from_currency_combobox.get()
        to_currency = to_currency_combobox.get()

        # Make a request to the API to get the exchange rate
        url = f"https://api.exchangerate-
api.com/v4/latest/{from_currency}"
        response = requests.get(url)
        data = response.json()

        # Convert the amount to the target currency
        exchange_rate = data['rates'][to_currency]
        converted_amount = amount * exchange_rate

        # Display the converted amount
        messagebox.showinfo("Conversion Result", f"{amount}
{from_currency} is equal to {converted_amount:.2f} {to_currency}")

    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")

# Function for clearing the Entry field
def clear_all():
    amount_entry.delete(0, END)

# Create the main window
root = tk.Tk()
root.title("Currency Converter")

# Create label for title  to display
title_label = tk.Label(root, text="Currency Convertor", font=
("Helvetica", 20,"bold"), bg="black", fg="white")
title_label.grid(row=0, column=0, columnspan=2, padx=5, pady=5)
# Label and Entry for the amount to convert
amount_label = ttk.Label(root, text="Amount:", font=8)
amount_label.grid(row=1, column=0, padx=5, pady=5, sticky="w")
amount_entry = ttk.Entry(root)
amount_entry.grid(row=1, column=1, padx=5, pady=5, sticky="ew")

# Label and Combobox for the currency to convert from
from_currency_label = ttk.Label(root, text="From Currency:", font=8)
```

```
from_currency_label.grid(row=2, column=0, padx=5, pady=5,
sticky="w")
from_currency_combobox = ttk.Combobox(root, values=["USD",
"EUR", "GBP", "JPY", "INR", "CAD", "CNY", "DKK"], width=10)
from_currency_combobox.grid(row=2, column=1, padx=5, pady=5,
sticky="ew")
from_currency_combobox.current(0)

# Label and Combobox for the currency to convert to
to_currency_label = ttk.Label(root, text="To Currency:", font=8)
to_currency_label.grid(row=3, column=0, padx=5, pady=5,
sticky="w")
to_currency_combobox = ttk.Combobox(root, values=["USD",
"EUR", "GBP", "JPY", "INR", "CAD", "CNY", "DKK"], width=10)
to_currency_combobox.grid(row=3, column=1, padx=5, pady=5,
sticky="ew")
to_currency_combobox.current(1)

# Button to trigger the conversion
convert_button = ttk.Button(root, text="Convert", width=20,
command=convert_currency)
convert_button.grid(row=4, column=0, padx=5, pady=5)

# Function for clearing the Entry field
clear_button = ttk.Button(root, text="Clear", width=20,
command=clear_all)
clear_button.grid(row=4, column=1, padx=5,pady=5)

# Run the Tkinter event loop
root.mainloop()
```

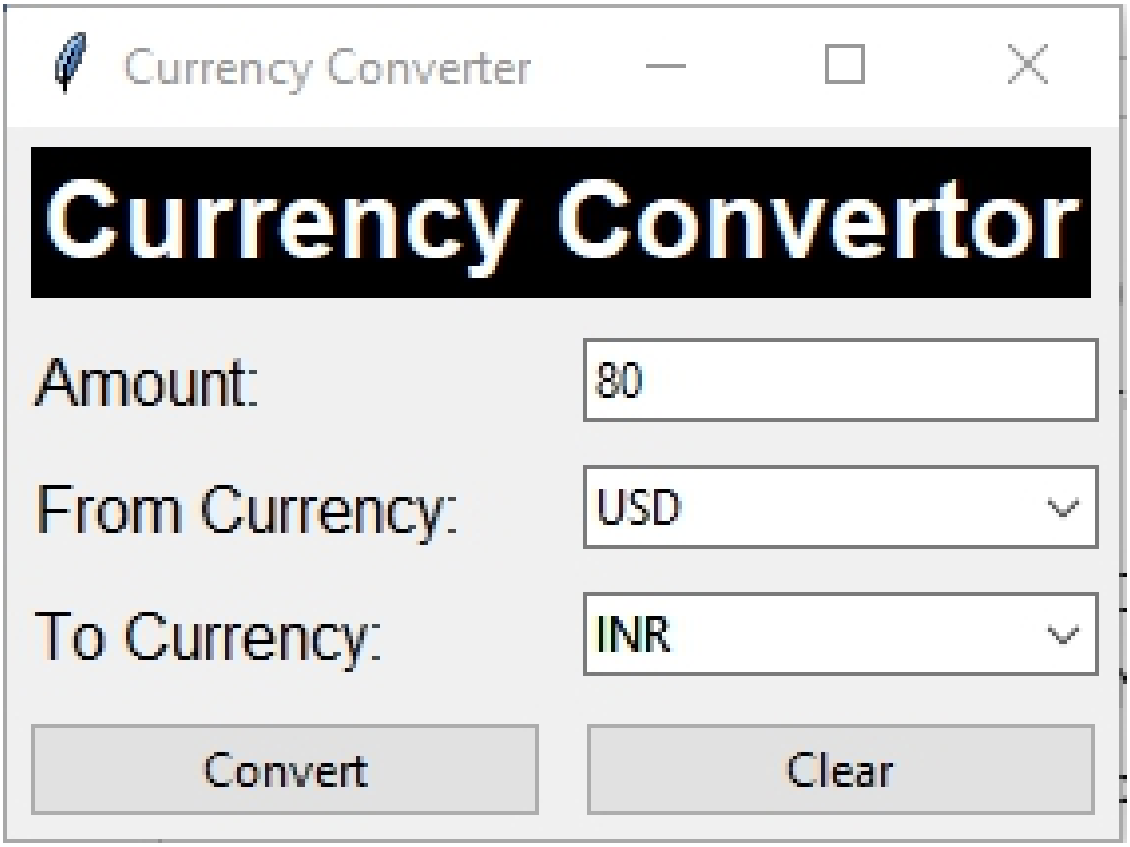
Output:

Upon executing the above program, you will see a GUI for Currency Converter. Here what will you see in GUI :

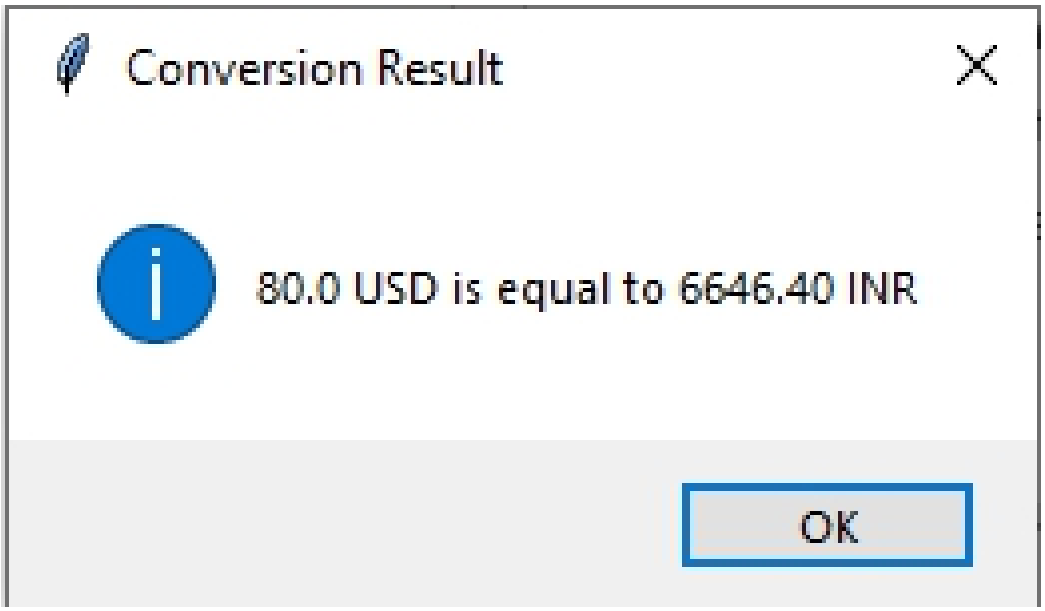
Amount Entry Field: A text entry field where users can input the amount they wish to convert.

From Currency Dropdown: A dropdown widget allowing users to select the source currency from a list of available options.

To Currency Dropdown: Another dropdown widget enabling users to select the target currency to which the amount will be converted.
Convert Button: A button that, when clicked, triggers the conversion process, displaying the converted amount based on the provided input.
Clear Button: A button that, when clicked, resets all input fields, allowing users to start over or make adjustments to their selections.



In the above window, an amount of 80 is converted from USD (United States Dollar) to INR (Indian Rupee), and the result of the conversion is displayed in a message box.



Simple and Compound Interest

Project #26

Design Pythn GUI application for simple and compound interest calculator using tkinter

Introduction:

Welcome to our Simple and Compound Interest Calculator! This tool helps you figure out how much money you can earn or owe over time. It has two buttons: one for simple interest and one for compound interest.

Simple interest is when you earn or owe money based only on the original amount you started with.

Formula:

```
Simple Interest = (Principal * Rate * Time) / 100
```

Compound interest is when you earn or owe money based on the original amount plus the interest that adds up over time. Using our calculator is straightforward. Simply input the principal amount, the rate of interest (expressed as a percentage), and the duration in years. Then, select the appropriate button - either "Calculate Simple Interest" or "Calculate Compound Interest" - and let our tool do the rest. Within seconds, you'll have the precise figures at your fingertips.

Formula:

```
compound_interest = principal * ((1 + rate) ** time - 1)
```

Solution:

```
import tkinter as tk
from tkinter import messagebox

def calculate_simple_interest():
    try:
        principal = float(entry_principal.get())
        rate = float(entry_rate.get()) / 100  # Converting percentage to
decimal
```

```
        time = float(entry_time.get())

        simple_interest = (principal * rate * time) / 100

        result_label.config(text=f"Simple Interest for principal amount {principal}: {simple_interest}")
    except ValueError:
        messagebox.showerror("Error", "Please enter valid numeric values.")

def calculate_compound_interest():
    try:
        principal = float(entry_principal.get())
        rate = float(entry_rate.get()) / 100  # Converting percentage to decimal
        time = float(entry_time.get())

        compound_interest = principal * ((1 + rate) ** time - 1)

        result_label.config(text=f"Compound Interest for principal amount{principal}: {compound_interest:.2f}")
    except ValueError:
        messagebox.showerror("Error", "Please enter valid numeric values.")

# Creating the main tkinter window
root = tk.Tk()
root.title("Interest Calculator")

# Setting up bold font for title label
bold_font = ("Times new roman", 14, "bold")

# Creating lable for title
title_label = tk.Label(root, text="Simple and Compound Interest Calculator", bg="yellow", fg="black", font= bold_font)
title_label.grid(row=0, column=0, columnspan=2,padx=5, pady=5)

# Creating labels, entry fields, and buttons for simple interest
label_principal = tk.Label(root, text="Principal amount:")
label_principal.grid(row=1, column=0, padx=5, pady=5)

entry_principal = tk.Entry(root)
```

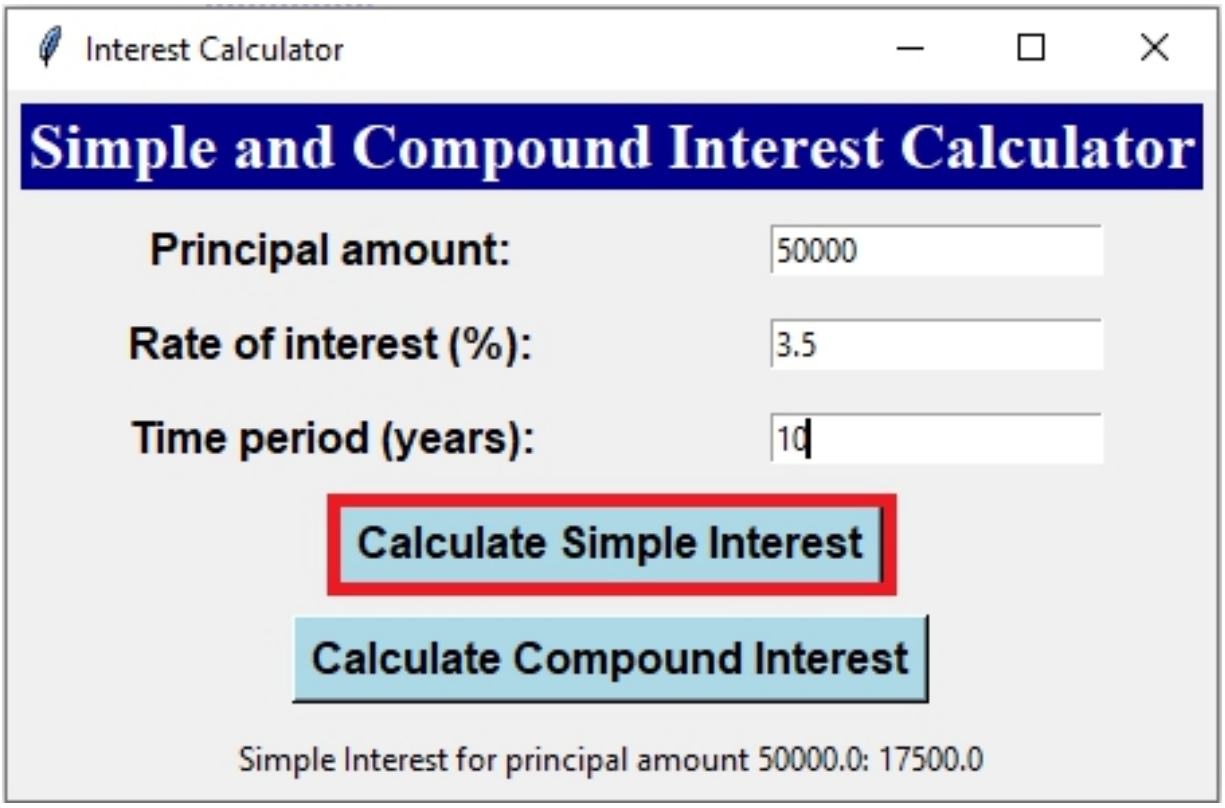
```
entry_principal.grid(row=1, column=1, padx=5, pady=5)

label_rate = tk.Label(root, text="Rate of interest (%):")
label_rate.grid(row=2, column=0, padx=5, pady=5)

entry_rate = tk.Entry(root)
entry_rate.grid(row=2, column=1, padx=5, pady=5)

label_time = tk.Label(root, text="Time period (years):")
label_time.grid(row=3, column=0, padx=5, pady=5)

entry_time = tk.Entry(root)
entry_time.grid(row=3, column=1, padx=5, pady=5)

calculate_simple_button = tk.Button(root, text="Calculate Simple
Interest", command=calculate_simple_interest)
calculate_simple_button.grid(row=4, column=0, columnspan=2,
padx=5, pady=5)

# Creating a button for compound interest calculation
calculate_compound_button = tk.Button(root, text="Calculate
Compound Interest", command=calculate_compound_interest)
calculate_compound_button.grid(row=5, column=0, columnspan=2,
padx=5, pady=5)

# Label to display the result
result_label = tk.Label(root, text="")
result_label.grid(row=6, column=0, columnspan=2, padx=5, pady=5)

# Running the main tkinter event loop
root.mainloop()
```

Output:

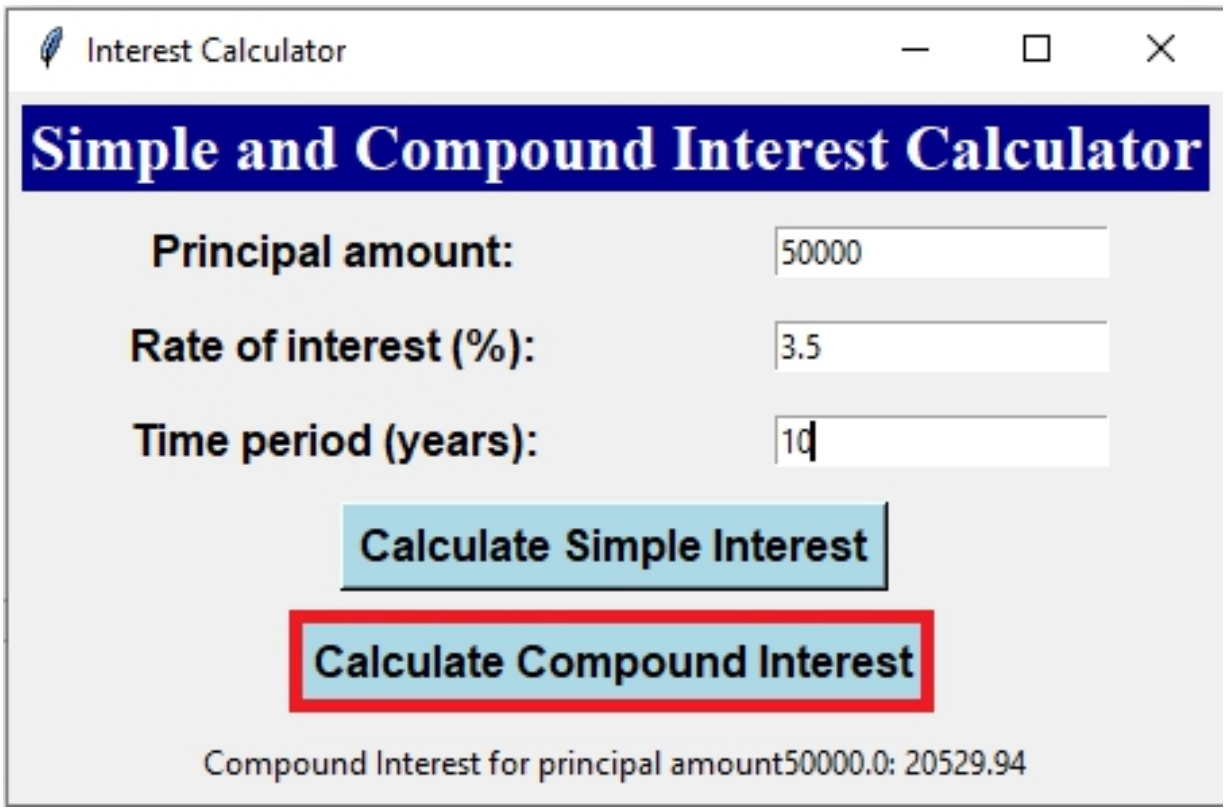
After executing the above code, you will see the GUI for simple and compound interest calculator. Lets see what will you find in GUI :
Input Fields: At the top of the window, there are input fields where users can enter the principal amount, rate of interest, and time period in years. These fields allow users to input the necessary parameters for calculating both simple and compound interest.
Calculate Simple Interest Button: Below the input fields, there's a button labeled 'Calculate Simple Interest'. Clicking this button

triggers the computation of the simple interest based on the provided inputs. The calculated simple interest result will be displayed in the main window below the buttons, providing users with the outcome of the computation.

Calculate Compound Interest Button: Beneath to the 'Calculate Simple Interest' button, there's another button labeled 'Calculate Compound Interest'. Clicking this button initiates the computation of compound interest using the entered parameters. Similar to the simple interest calculation, the result of the compound interest computation will be displayed in the main window below the buttons.



Above window shows the result of simple interest for principal amount 50000.



Above window shows the result of compound interest for principal amount 50000.

Word to PDF converter

Project #27

Design Pythn GUI application for word to PDF converter using tkinter

Introduction:

Welcome to the Word to PDF Converter program! This tool helps you change your Word documents into PDF files. It's easy to use and makes managing your documents simpler.

Features:

Browse: Click this button to find and choose the Word file you want to convert.

Convert: After you've chosen your Word file, click "Convert" to change it into a PDF file. Once you've picked the file, its location will be shown on the screen. The program does this quickly and keeps all your text and layout just the way it was in the Word file.

Open: Once the conversion is done, you can use the "Open" button to easily open the PDF file. This saves you time because you don't have to look for the file on your computer.

Error Handling:

If you forget to choose a Word file before trying to convert, the program will let you know with a message. This helps you avoid problems and makes sure everything goes smoothly while you're using the program.

This Word to PDF Converter is made to be simple and helpful for anyone who needs to manage their documents.

Solution:

```
import tkinter as tk
from tkinter import filedialog,messagebox
from docx import Document
from fpdf import FPDF
import os
```

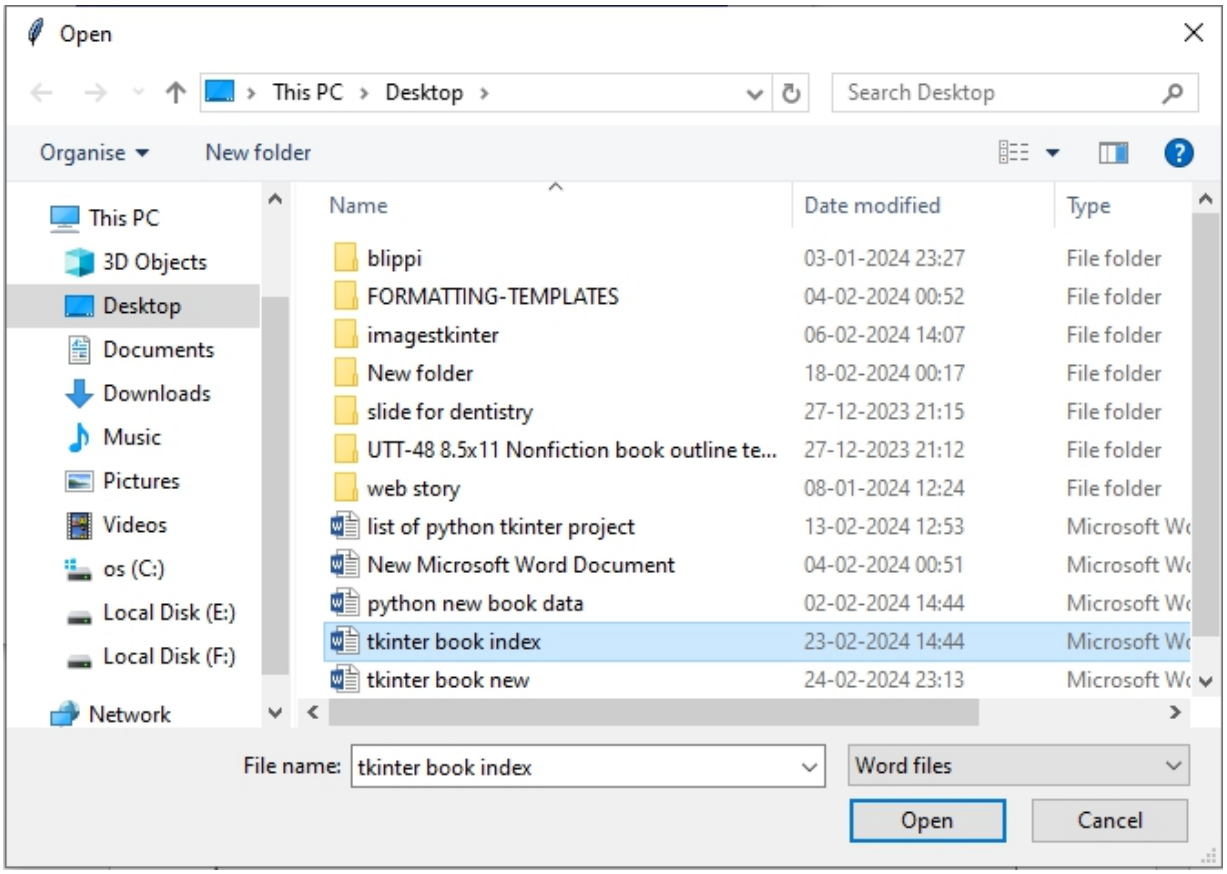
```
word_file_path = None
def browse_word_file():
    global word_file_path
    # Prompt the user to select a Word file
    word_file_path = filedialog.askopenfilename(filetypes=[("Word
files", "*.docx")])
def create_pdf():
    global word_file_path
    # Check if a file was selected
    if word_file_path:
        # If a file was selected, call the convert_to_pdf function with
the selected file path
        convert_to_pdf(word_file_path)
    else:
        # If no file was selected, update the status label to indicate it
        status_label.config(text="No file selected")
def convert_to_pdf(word_file_path):
    # Check if a Word file path is provided
    if word_file_path:
        try:
            # Load the Word document
            doc = Document(word_file_path)

            # Create a new PDF
            # Create a new instance of FPDF (PDF generation library)
            pdf = FPDF()
            # Set automatic page break feature with a margin of 15
units
            pdf.set_auto_page_break(auto=True, margin=15)
            # Add a new page to the PDF document
            pdf.add_page()
            # Set the font for the PDF to Arial with a font size of 12
            pdf.set_font("Arial", size=12)
            # Iterate over paragraphs in the Word document and add
them to the PDF
            for para in doc.paragraphs:
                pdf.multi_cell(0, 10, para.text)
```

```
            # Create PDF file path by replacing the extension
            pdf_file_path = os.path.splitext(word_file_path)[0] + ".pdf"
            # Output the PDF to the specified path
            pdf.output(pdf_file_path)

            # Set the Word file path and PDF file path variables
            file_path.set(word_file_path)
            pdf_path.set(pdf_file_path)

            # Enable the open PDF button
            open_pdf_button.config(state="normal")

            # Show a message box indicating successful conversion
            messagebox.showinfo("Conversion Completed", "PDF
conversion completed successfully!")

            # Update the labels to display PDF file path
            pdf_label.config(text="Pdf file full path :")
            pdf_path_label.config(text=f"PDF saved: {pdf_file_path}")
        except Exception as e:
            # Show an error message box if an exception occurs during
conversion
            messagebox.showerror("Error", f"An error occurred:
{str(e)}")
    else:
        # If no Word file path is provided, update the status label
        status_label.config(text="No Word file selected!")
def open_pdf():
    # Get the PDF file path from the pdf_path variable
    pdf_file_path = pdf_path.get()
    # Check if a PDF file path is available
    if pdf_file_path:
        # Open the PDF file using the default application associated
with its file type
        os.startfile(pdf_file_path)
if __name__ == "__main__":
    # Create Main tkinter window
    root = tk.Tk()
```

```
    # Set the title of the window
    root.title("Word to PDF Converter")
    # Set the size of the window to 450x350 pixels
    root.geometry("450x350")
    # Define StringVar variables to hold file paths
    file_path = tk.StringVar()
    pdf_path = tk.StringVar()
    # Label widget for the title
    tk.Label(root, text="Word to Pdf Convertor", fg="White", bg="dark
blue", font=("Helvetica",25,"bold")).pack()
    # Label widget for instruction

    tk.Label(root, text="Select a Word file to convert:", font=
("Helvetica",15,"bold")).pack(pady=10)
    # Button widget to browse for Word file
    tk.Button(root, text="Browse", fg="White", bg="dark blue",font=
("Helvetica",15,"bold"), command=browse_word_file).pack()
    # Button widget to open PDF file
    convert_pdf_button = tk.Button(root, text="Convert PDF",
fg="dark blue", bg="white",font=("Helvetica",15,"bold"),
command=create_pdf, state="normal")
    convert_pdf_button.pack(pady=5)
    status_label = tk.Label(root, text="")
    status_label.pack(pady=5)

    pdf_label = tk.Label(root, text="", fg= "dark blue",font=
("Helvetica",10,"bold"))
    pdf_label.pack(pady=5)
    # Label widget to show complete path of PDF file
    pdf_path_label = tk.Label(root, text="", font=("Helvetica",8,"bold"))
    pdf_path_label.pack(pady=5)
    # Button widget to open PDF file
    open_pdf_button = tk.Button(root, text="Open PDF", fg="dark
blue", bg="white",font=("Helvetica",15,"bold"), command=open_pdf,
state="disabled")
    open_pdf_button.pack(pady=5)
    root.mainloop()
```

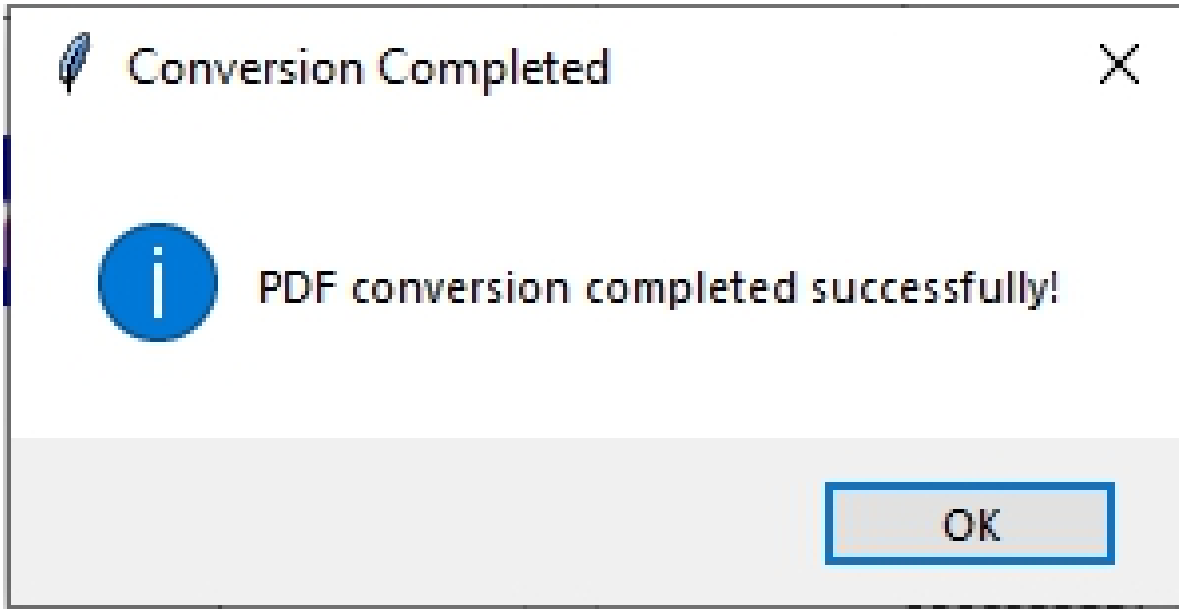
Output:

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.

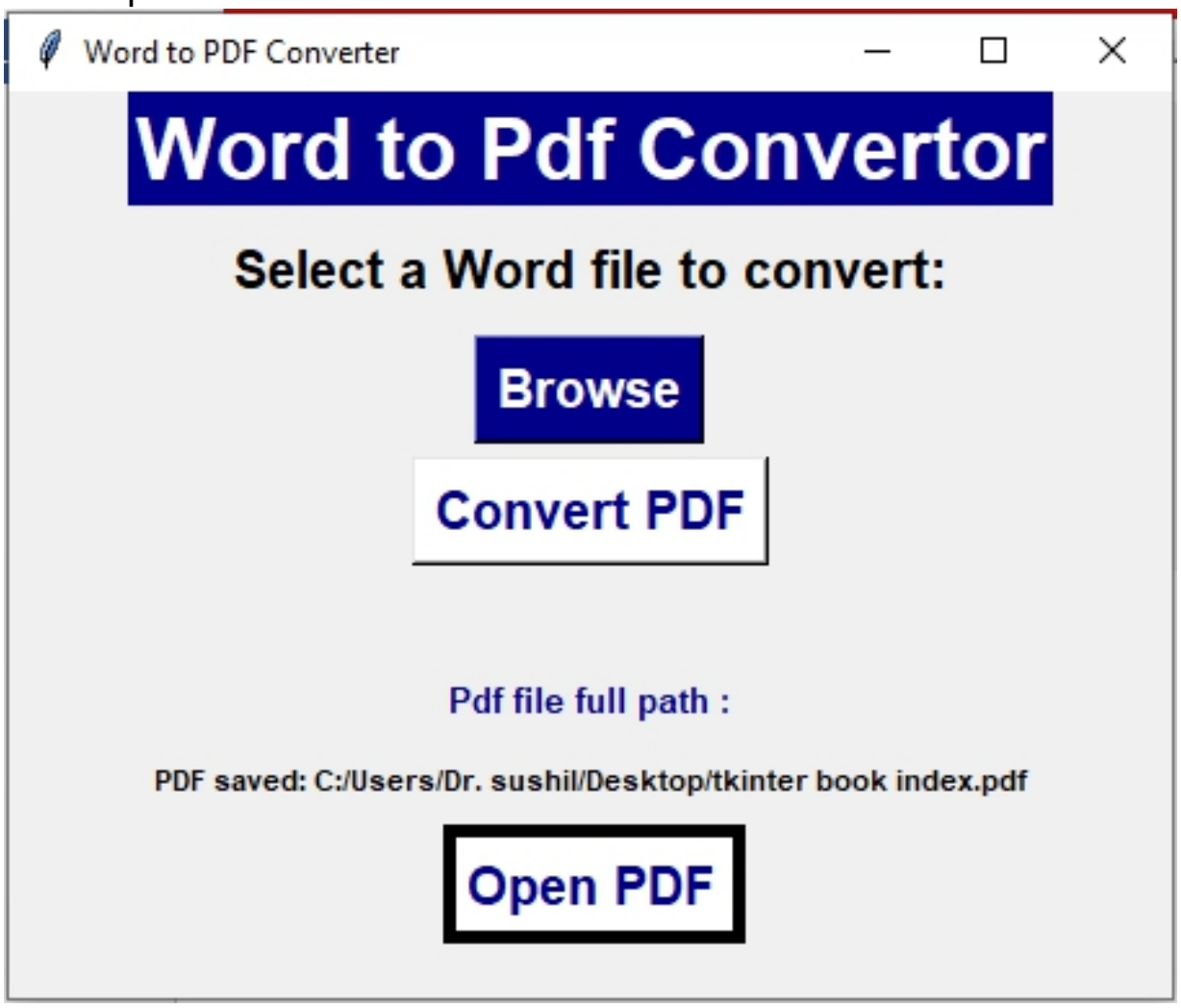


After clicking the Browse button, above open dialog box is displayed on your screen. Select the word file from the folder and click on the open button


Word to PDF Converter
Word to Pdf Convertor
Select a Word file to convert:
Browse
Convert PDF
Open PDF


Conversion Completed
PDF conversion completed successfully!
OK

Once a Word file has been successfully converted to PDF, above message box is displayed and pdf full path label is updated to show the full path of the converted PDF file on the main window.



After clicking the ‘Open PDF’Button, converted PDF file is directly open from the application.

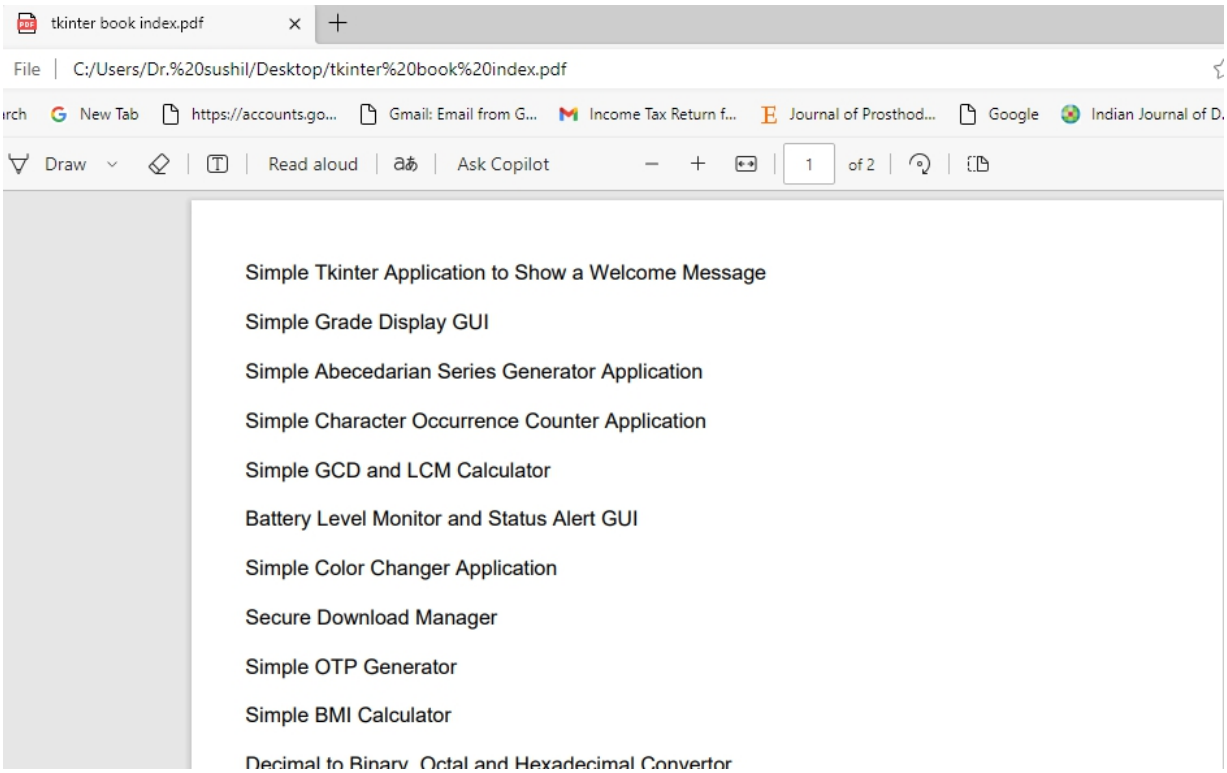


Image Zoomer Application

Project #28

Design Python GUI application for Image zoomer using tkinter

Introduction:

Welcome to the Image Zoomer application! This user-friendly tool simplifies the process of viewing and zooming images with just a few clicks. With the Image Zoomer, you can effortlessly open and view images of various formats. The application provides two buttons – "Zoom In" and "Zoom Out" – to adjust the size of the displayed image according to your preference. This allows you to focus on specific areas of interest or get a broader view with ease. To get started, simply select the "Open" option from the "File" menu at the top of the application window. This will open a dialog box where you can navigate to and select the image you wish to view. Once selected, the image will be displayed within the canvas area of the application. You can then use the "Zoom In" button to make the picture bigger or the "Zoom Out" button to make it smaller. It's like using a magnifying glass to see things closer or farther away.

To select the image file from directories or folders, we need to import filedialod module.

from tkinter import filedialog

This allow you to interact with file system to select and open files. Lets talk about filedialod module.

The filedialog Module

The filedialog module in Tkinter is a useful tool for creating file dialogs in Python GUI applications. These dialogs allow users to interact with the file system, enabling actions such as opening files, saving files, or selecting directories.

It has various functions like

askopenfilename(): Opens a dialog for selecting a single file.

asksaveasfilename(): Opens a dialog for saving a file with a specified name.
askdirectory(): Opens a dialog for selecting a directory.

Let’s use them in our project
Solution:

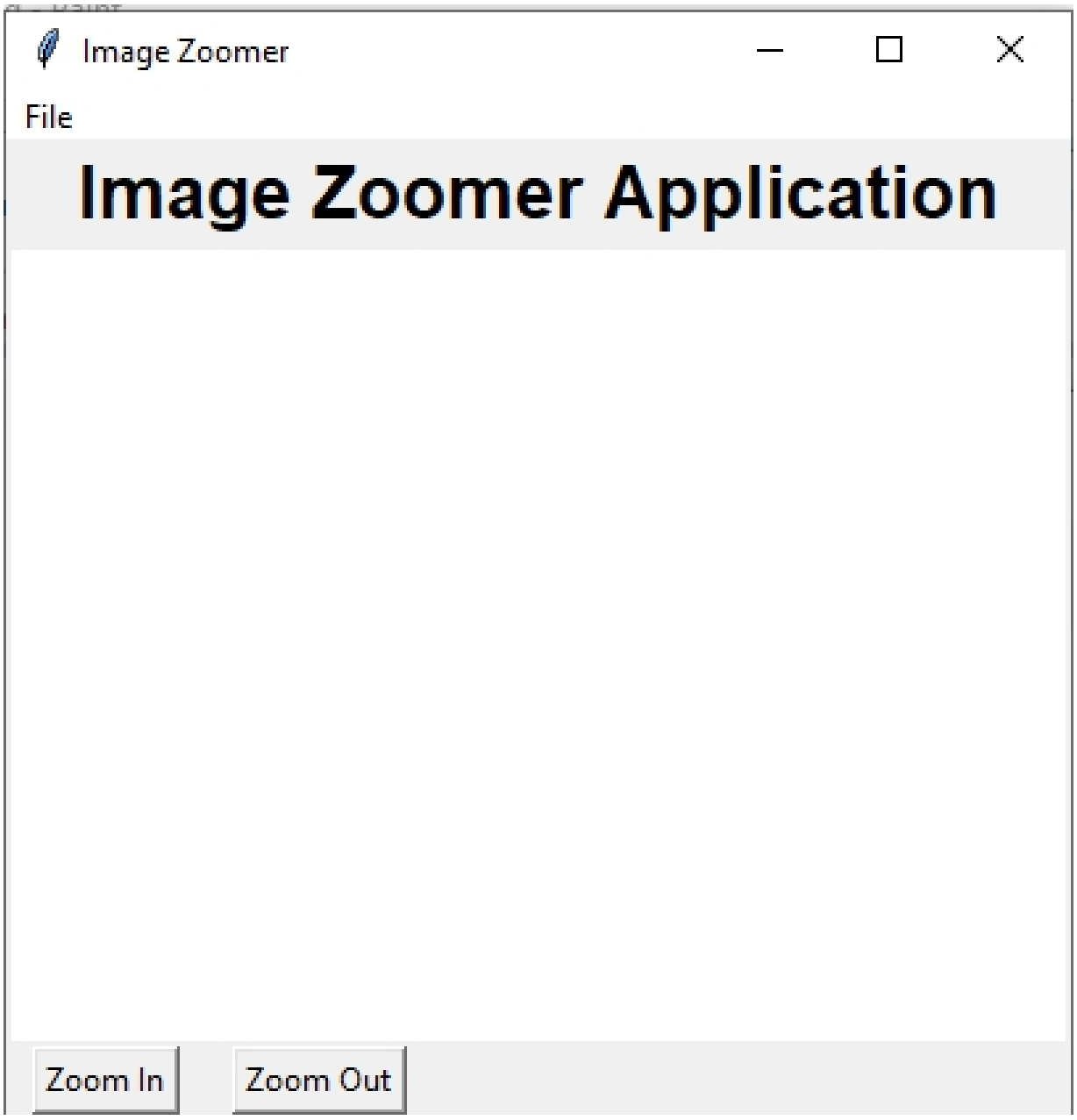
```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk

def open_image():
    global image_path
    # Open a file dialog to select an image file
    image_path = filedialog.askopenfilename(title="Select Image",
filetypes=[("Image files", "*.png;*.jpg;*.jpeg;*.gif")])
    if image_path:
        # If a valid image path is selected, display the image
        show_image(image_path)

def show_image(image_path):
    global scale_factor
    # Open the image
    image = Image.open(image_path)
    # Resize image on the scale factor
    image = image.resize((int(image.width * scale_factor),
int(image.height * scale_factor)))
    # Create a Tkinter PhotoImage
    tk_image = ImageTk.PhotoImage(image)
    # Set the fixed size for the canvas and update the scroll region
    canvas.config(scrollregion=(0, 0, image.width, image.height))
    # Create the image on the canvas
    canvas.create_image(0, 0, anchor=tk.NW, image=tk_image)
    canvas.image = tk_image

# Function to increase size of image
def zoom_in():
    global scale_factor
    # Increase the scale factor for zooming in
    scale_factor *= 1.2
```

```
    # Show updated image
    show_image(image_path)

# Function to decrease size of image
def zoom_out():
    global scale_factor
    # Decrease the scale factor for zooming out
    scale_factor /= 1.2
    # Show updated image
    show_image(image_path)

# Global variables
scale_factor = 1.0
image_path = None

# Create main window
root = tk.Tk()
root.title("Image Zoomer")

# Create label for showing title on form
title_label = tk.Label(root, text="Image Zoomer Application", font=
("Arial",20,"bold"))
title_label.pack()

# Menu bar
menu_bar = tk.Menu(root)
root.config(menu=menu_bar)

file_menu = tk.Menu(menu_bar, tearoff=0)
menu_bar.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="Open", command=open_image)

# Zoom buttons
zoom_in_button = tk.Button(root, text="Zoom In",
command=zoom_in)
zoom_out_button = tk.Button(root, text="Zoom Out",
command=zoom_out)

# Place widgets
canvas = tk.Canvas(root, bg="white", width=400, height=300)  # Set
fixed size for the canvas
```

```
canvas.pack(expand=True, fill=tk.BOTH)

zoom_in_button.pack(side=tk.LEFT, padx=10)
zoom_out_button.pack(side=tk.LEFT, padx=10)
# Run the Tkinter event loop
root.mainloop()
```

Output :

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.



At the top of the window, you'll notice a File menu bar. Click on the 'File' menu to reveal its options. From the dropdown menu, select the 'Open' submenu. This action will trigger a file dialog box, enabling you to choose an image from your directory.

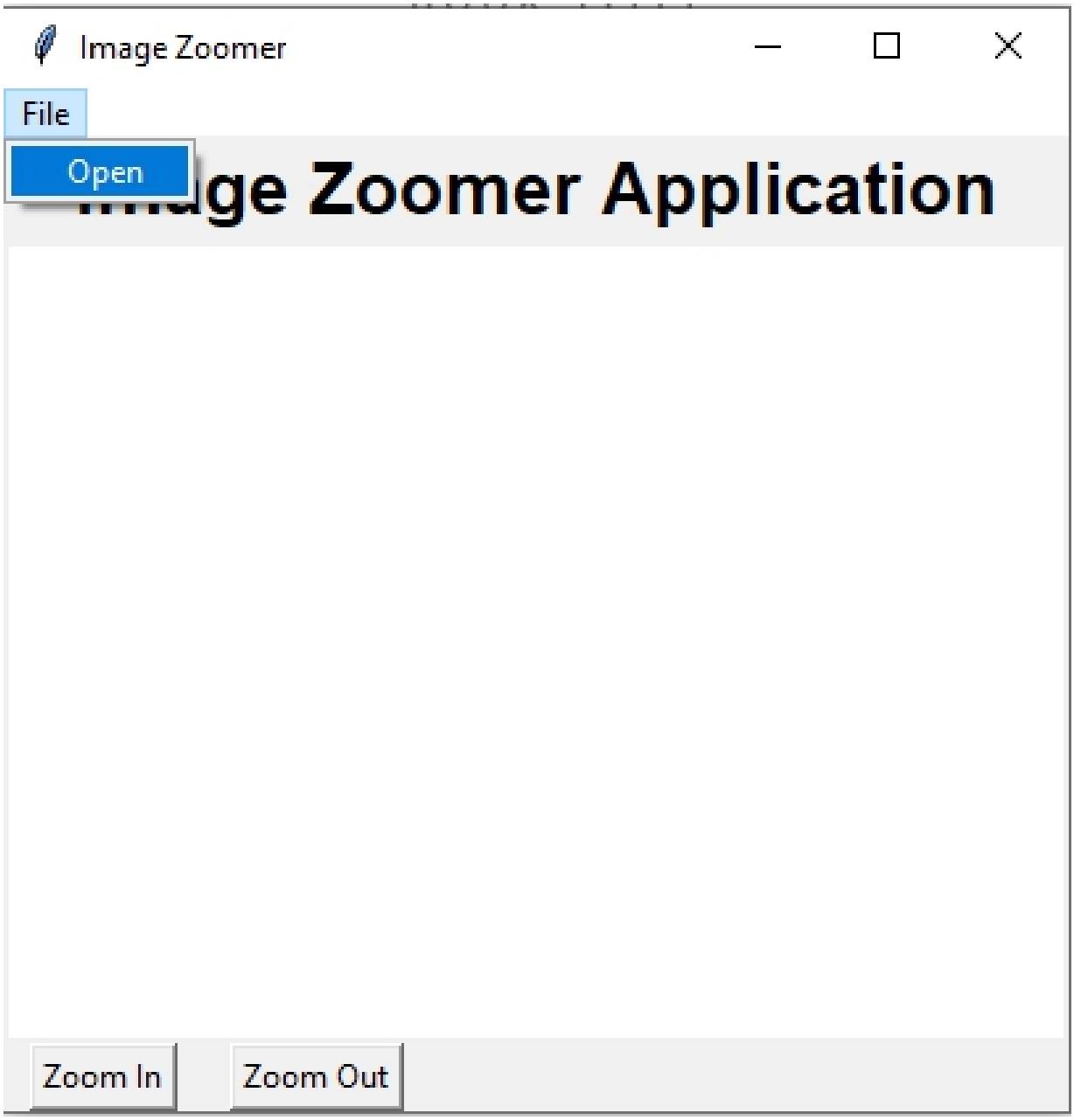
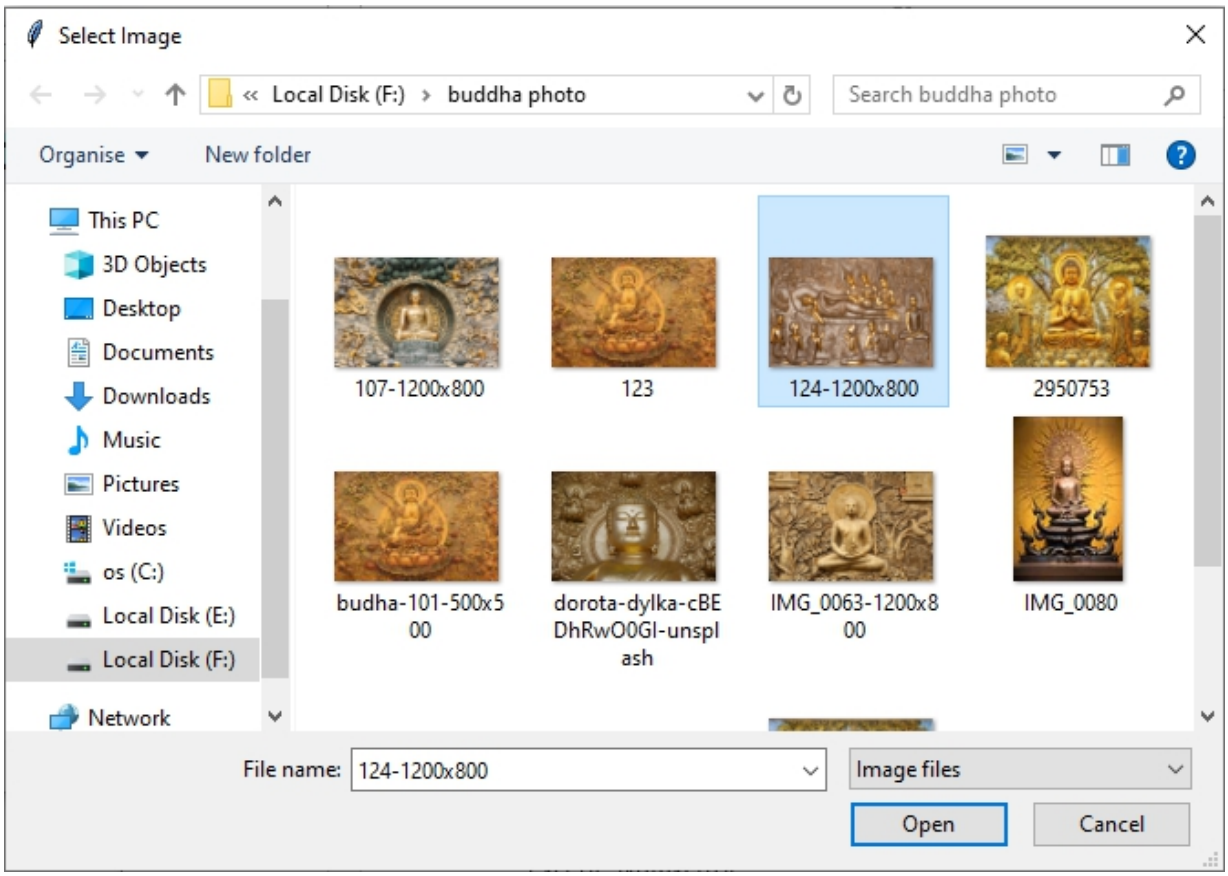
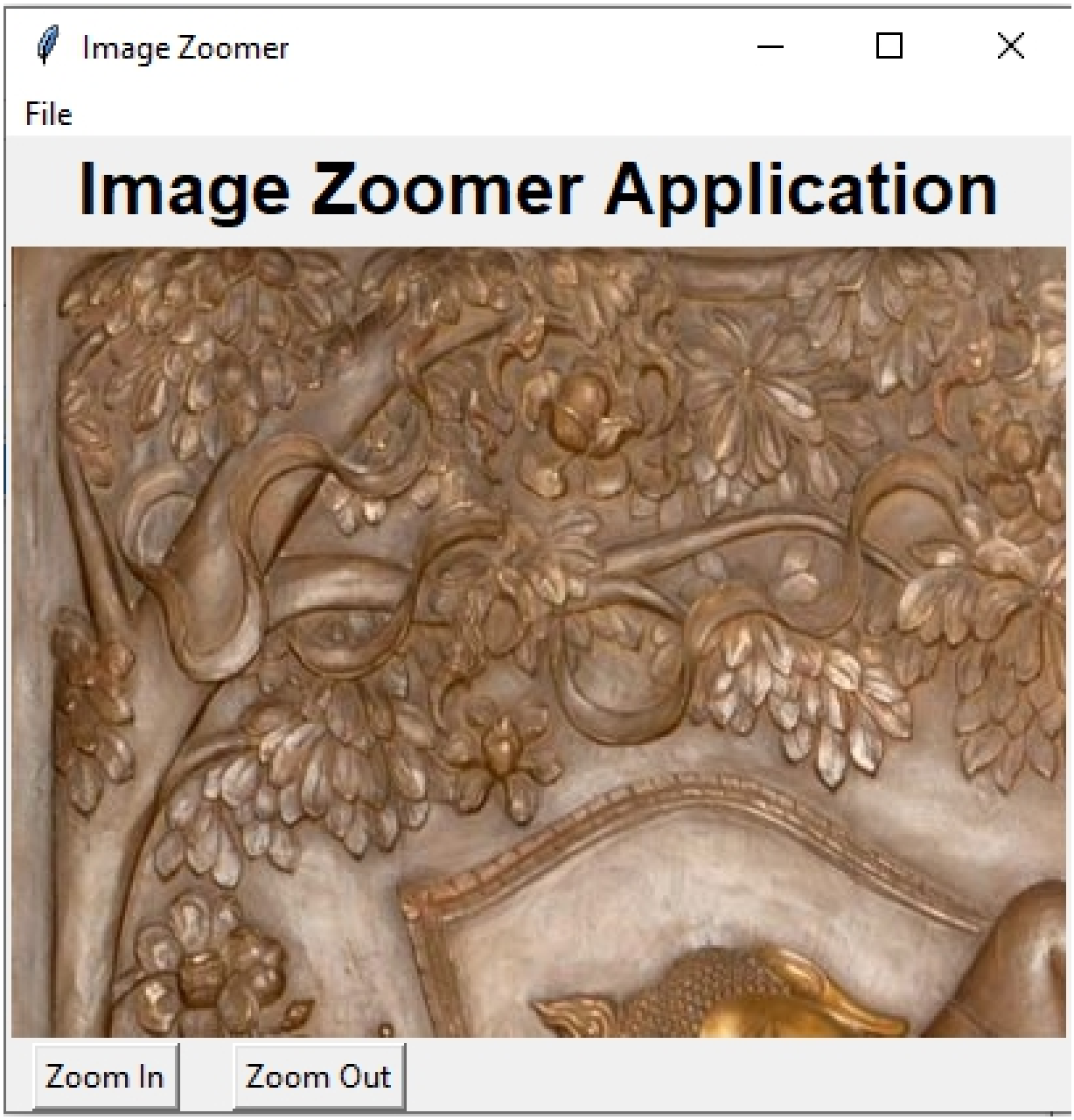


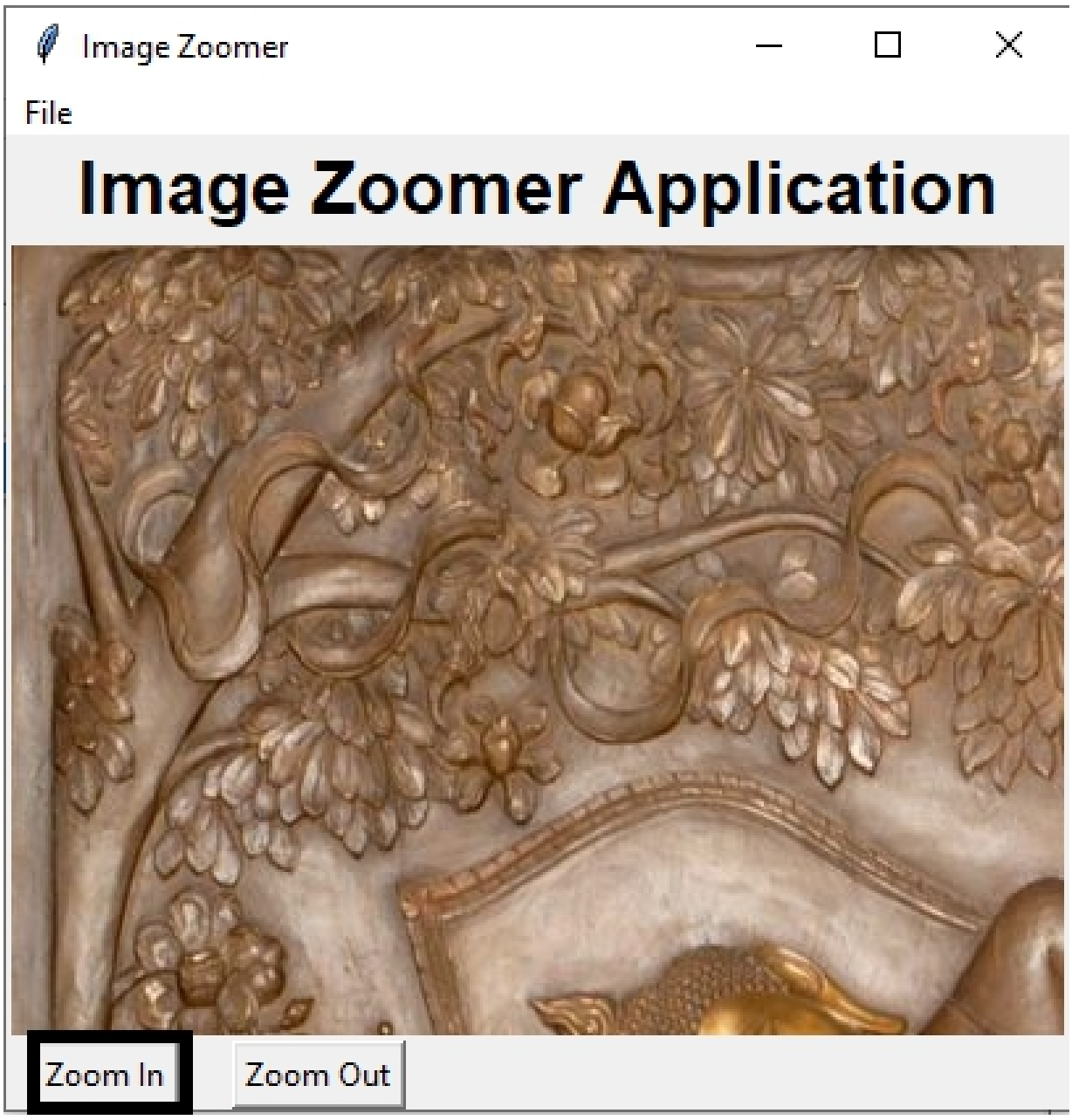
Image Zoomer
File
Open
ge Zoomer Application
Zoom In
Zoom Out



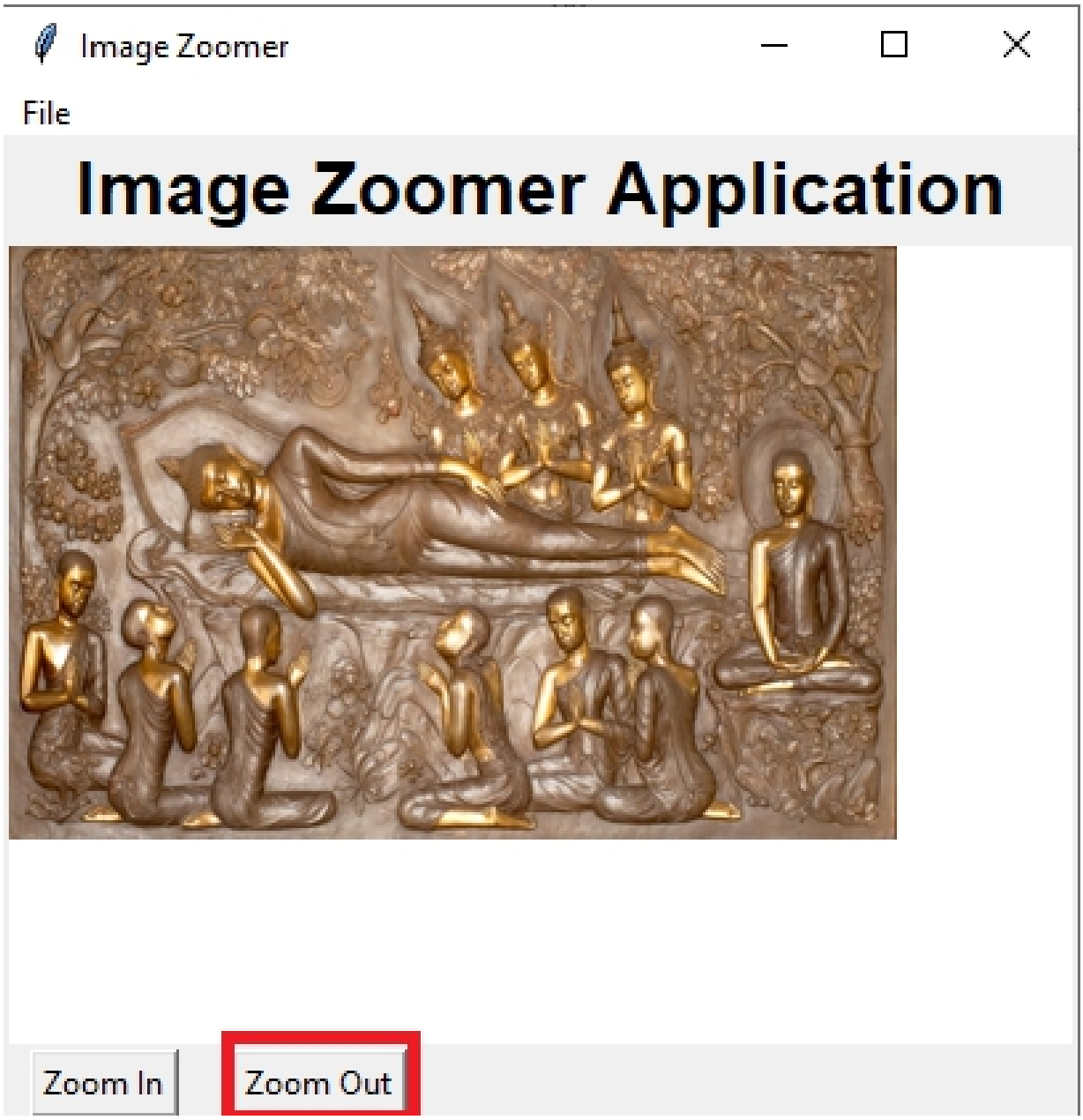
After clicking on open submenu, above dialog box is opened on your screen.



After clicking the 'Open' button, the image is loaded into the canvas area, as depicted above.



After clicking the 'Zoom in' button, the image is resized as shown above.



After clicking the 'Zoom out' button, the image size is reduced, as shown above.

Simple GUI Calculator

Project #29

Design Python GUI application for Simple GUI calculator using tkinter

Introduction:

Welcome to the Simple GUI Calculator! This easy-to-use tool helps you do math calculations quickly and easily. With the Simple GUI Calculator, you can perform arithmetic operations such as addition, subtraction, multiplication, and division, all within a clean and intuitive graphical user interface (GUI). The calculator features a familiar layout with buttons for digits and operators, making it easy to input expressions and obtain results. Whether you're in school, at work, or just need to do some basic math, our calculator is here to help.

Solution:

```
import tkinter as tk

# Function to update the display
def update_display(value):
    global s
    s = display_var.get() + str(value)
    display_var.set(s)


# Function to clear the display
def clear_display():
    display_var.set("")

# Function to evaluate the expression and display the result
def calculate():
    # Get the expression from the display entry
    expression = display_var.get()
    try:
        # Evaluate the expression and convert the result to a string
```

```
            result = str(eval(expression))
            # Set the result to the display entry
            display_var.set(result)
        except ZeroDivisionError:
            # Handle division by zero error
            display_var.set("Error: Division by zero")
        except SyntaxError:
            # Handle invalid expression syntax error
            display_var.set("Error: Invalid expression")

# Create the main window
root = tk.Tk()
root.title("Simple Calculator")

# Variable to store the display text
display_var = tk.StringVar()

# Entry widget to display the numbers and results
display_entry = tk.Entry(root, textvariable=display_var, font=("Arial",
20), bd=10, insertwidth=1, justify="right")
display_entry.grid(row=0, column=0, columnspan=4)

# Button labels
button1 = tk.Button(root, text="7", width=6, height=2,font=("Arial",14),
command=lambda :update_display(7))
button1.grid(row=1, column=0, padx=5, pady=5)

button2 = tk.Button(root, text="4", width=6, height=2,font=("Arial",14),
command=lambda :update_display(4))
button2.grid(row=2, column=0, padx=5, pady=5)

button3 = tk.Button(root, text="1",width=6, height=2, font=("Arial",14),
command=lambda :update_display(1))
button3.grid(row=3, column=0, padx=5, pady=5)

button4 = tk.Button(root, text="0",width=6, height=2, font=("Arial",14),
command=lambda :update_display(0))
button4.grid(row=4, column=0, padx=5, pady=5)

button5 = tk.Button(root, text="8", width=6, height=2,font=("Arial",14),
command=lambda :update_display(8))
```

```
button5.grid(row=1, column=1, padx=5, pady=5)

button6 = tk.Button(root, text="5",width=6, height=2, font=("Arial",14),
command=lambda :update_display(5))
button6.grid(row=2, column=1, padx=5, pady=5)

button7 = tk.Button(root, text="2",width=6, height=2, font=("Arial",14),
command=lambda :update_display(2))
button7.grid(row=3, column=1, padx=5, pady=5)

button8 = tk.Button(root, text=".", width=6, height=2,font=("Arial",14),
command=lambda :update_display("."))
button8.grid(row=4, column=1, padx=5, pady=5)

button9 = tk.Button(root, text="9",width=6, height=2, font=("Arial",14),
command=lambda :update_display(9))
button9.grid(row=1, column=2, padx=5, pady=5)


button10 = tk.Button(root, text="6",width=6, height=2, font=
("Arial",14), command=lambda :update_display(6))
button10.grid(row=2, column=2, padx=5, pady=5)

button11 = tk.Button(root, text="3", width=6, height=2,font=
("Arial",14), command=lambda :update_display(3))
button11.grid(row=3, column=2, padx=5, pady=5)
button12 = tk.Button(root, text="00",width=6, height=2, font=
("Arial",14), command=lambda :update_display(00))
button12.grid(row=4, column=2, padx=5, pady=5)
button13 = tk.Button(root, text="/",width=6, height=2, font=
("Arial",14), command=lambda :update_display("/"))
button13.grid(row=1, column=3, padx=5, pady=5)
button14 = tk.Button(root, text="-",width=6, height=2, font=
("Arial",14), command=lambda :update_display("-"))
button14.grid(row=2, column=3, padx=5, pady=5)
button15 = tk.Button(root, text="*", width=6, height=2,font=
("Arial",14), command=lambda :update_display("*"))
button15.grid(row=3, column=3, padx=5, pady=5)
button16 = tk.Button(root, text="+",width=6, height=2, font=
("Arial",14), command=lambda :update_display("+"))
button16.grid(row=4, column=3, padx=5, pady=5)
```

```
# Clear button
clear_button = tk.Button(root, text="C", width=6, height=2, font=
("Arial", 14), command=clear_display)
clear_button.grid(row=5, column=0, padx=5, pady=5)

# Calculate button
calculate_button = tk.Button(root, text="=", width=6, height=2, font=
("Arial", 14), command=calculate)
calculate_button.grid(row=5, column=1, padx=5, pady=5)


button17 = tk.Button(root, text="(", width=6, height=2, font=("Arial",
14), command=lambda:update_display("("))
button17.grid(row=5, column=2, padx=5, pady=5)

button18 = tk.Button(root, text=")", width=6, height=2, font=("Arial",
14), command=lambda:update_display(")"))
button18.grid(row=5, column=3, padx=5, pady=5)

# Start the GUI event loop
root.mainloop()
```

Output :

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen. Here's what you can expect to see:

Display Screen:

At the top of the window, there is a display screen which is made up of text field, where the input and output of arithmetic operations are shown. This screen provides a clear view of the calculations being performed.

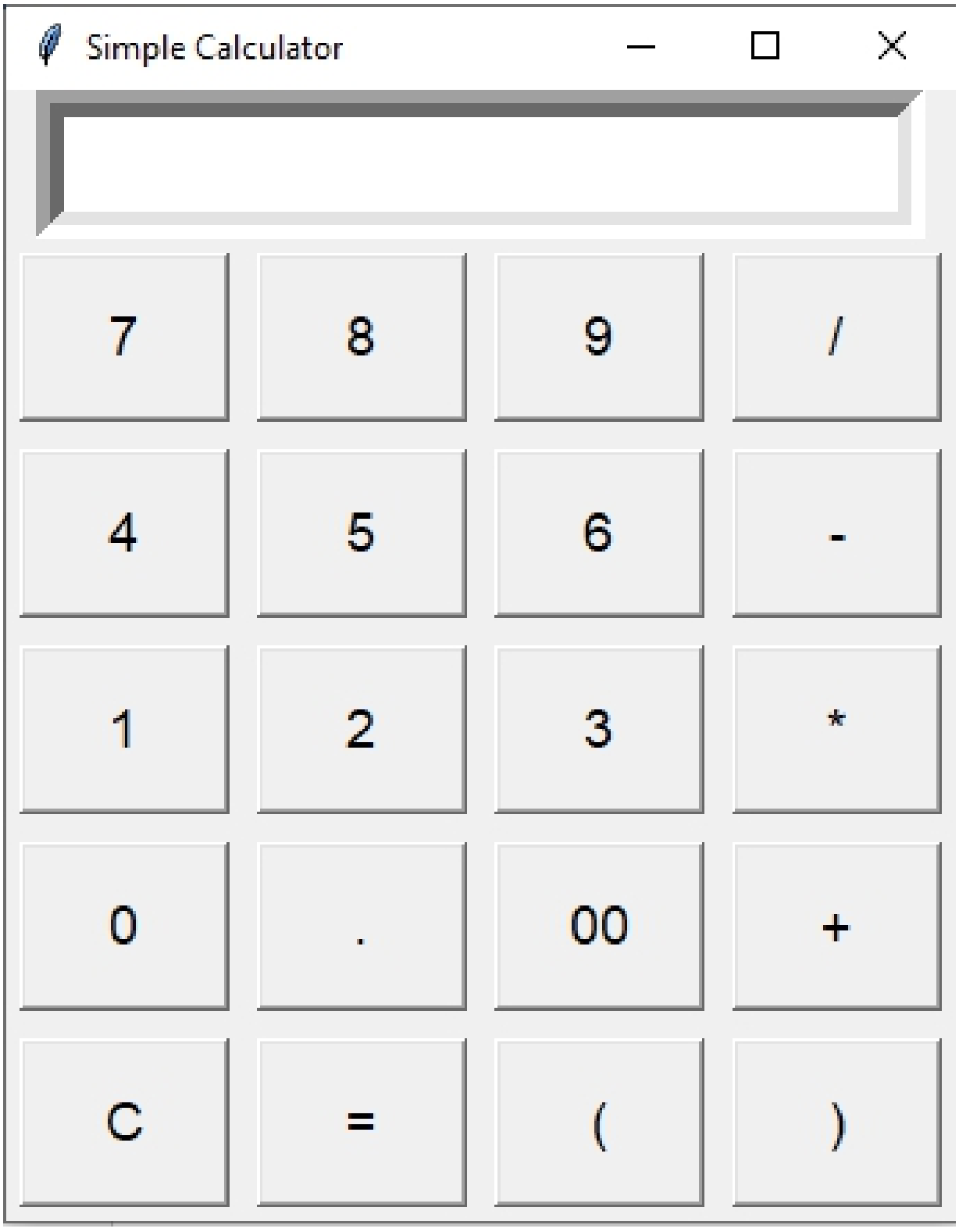
Numeric Buttons:

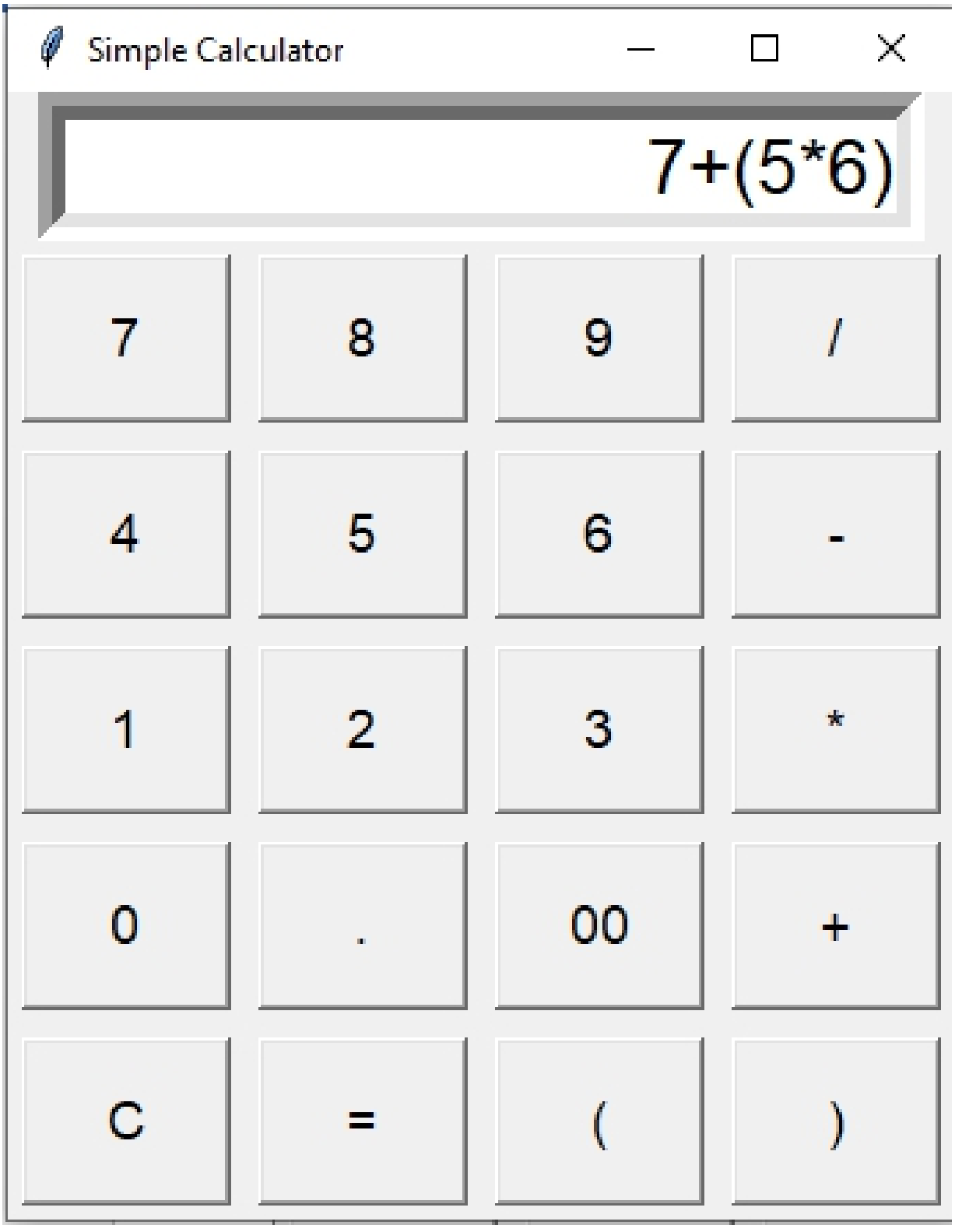
Below the display screen, there are numeric buttons from 0 to 9. These buttons allow users to input numerical values for their calculations.

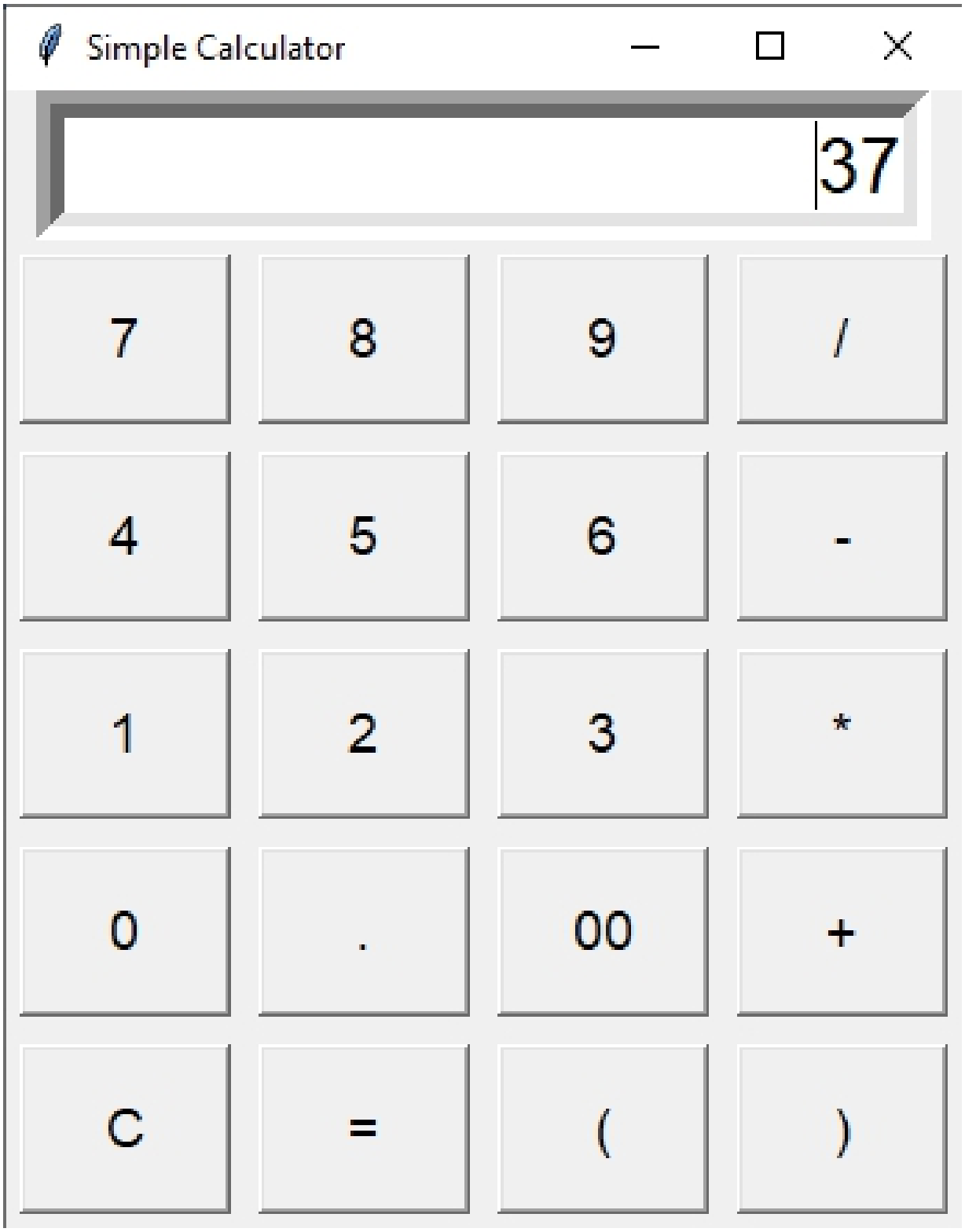
Arithmetic Operation Buttons:

Adjacent to the numeric buttons, there are buttons for common arithmetic operations such as addition (+), subtraction (-), multiplication (*), and division (/). These buttons enable users to perform basic mathematical operations easily.

Clear Button:
There is a "Clear" button provided to allow users to clear the input and reset the calculator, providing a convenient way to correct mistakes or start a new calculation.
Equals (=) Button:
Lastly, there is an "Equals" button, typically labeled "=", which when clicked, evaluates the entered expression and displays the result on the screen.


Simple Calculator
7
8
9
/
4
5
6
-
1
2
3
*
0
.
00
+
C
=
(
)


Simple Calculator
7+(5*6)
7
8
9
/
4
5
6
-
1
2
3
*
0
.
00
+
C
=
(
)



Age calculator

Project #30

Design Python GUI application for Age Calculator using tkinter

Introduction:

Welcome to the Age Calculator! This handy tool makes it easy to calculate your age in months, years, and days with just a few clicks. With the Age Calculator, you can simply enter your birthdate and click the "Calculate Age" button. The calculator will then determine your age and display the result in a messagebox, showing your age in years, months and days.

To calculate age of current date, we need to import datetime module in our program. To do this

from datetime import datetime

The code imports the datetime class from the datetime module in Python. This allows you to create and work with date and time objects within your code.

The datetime module

The datetime module in Python is a powerful tool for working with dates, times, and durations.

Once you've imported the datetime class, you can use it to:

Get the current date and time: Use datetime.now() to get a datetime object representing the current moment.

Create specific date and time objects: Use datetime(year, month, day, hour, minute, second, microsecond) to create a datetime object with the desired values.

Perform date and time calculations: Use methods like timedelta to add or subtract time from datetime objects.

Format dates and times: Use methods like strftime to format datetime objects into various string representations.

Parse dates and times from strings: Use methods like strptime to convert string representations of dates and times
into datetime objects.

Solution :

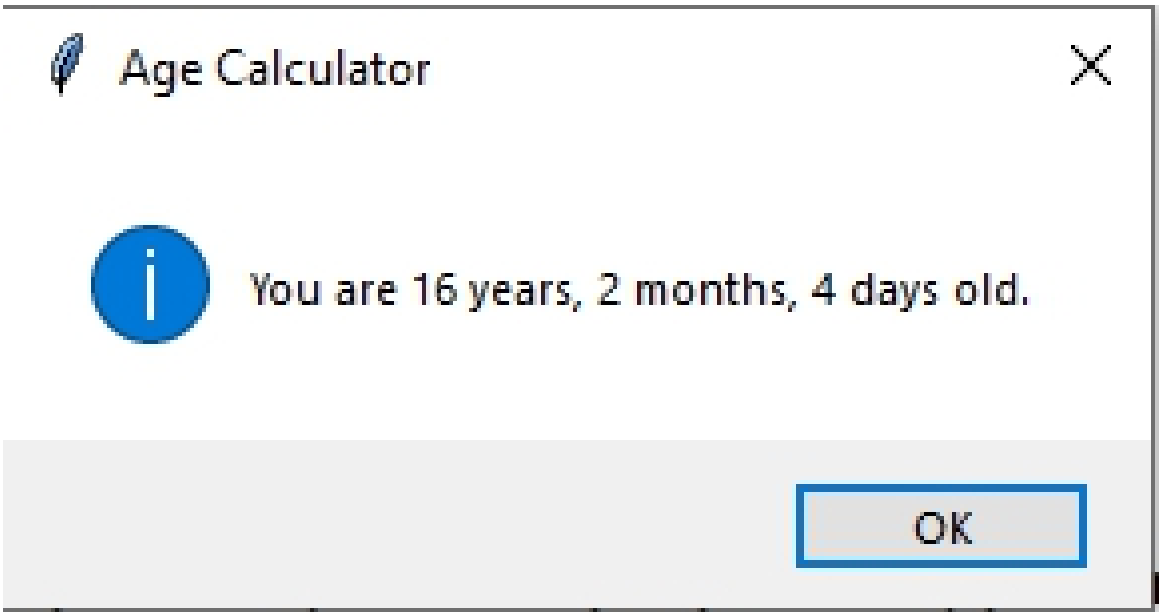
```
import tkinter as tk
from tkinter import messagebox
from datetime import datetime

def calculate_age():
    # Try to parse the birth date entered in the entry field
    try:
        # Get the birth date from the entry field and parse it into a datetime object
        birth_date = datetime.strptime(entry.get(), "%Y-%m-%d")

        # Get the current date
        current_date = datetime.now()

        # Create datetime objects for today and the birth date
        today = datetime(current_date.year, current_date.month, current_date.day)
        dob = datetime(birth_date.year, birth_date.month, birth_date.day)

        # Calculate the difference in years, months, and days
        years = (today - dob).days // 365
        months = ((today - dob).days % 365) // 30
        days = ((today - dob).days % 365) % 30

        # Display the calculated age in a message box
        messagebox.showinfo("Age Calculator", f"You are {years} years, {months} months, {days} days old.")

    # If an error occurs during parsing the date, show an error message
    except ValueError:
        messagebox.showerror("Error", "Please enter a valid date in the format YYYY-MM-DD.")

# Create the Tkinter window
root = tk.Tk()
root.title("Age Calculator")
```

```
# Create label for title to display on form
title_label = tk.Label(root, text="Age Calculator in years,months and
days", fg="yellow", bg="black", font=("Arial", 20, "bold"))
title_label.pack()

# Create a label for the entry field
label = tk.Label(root, text="Enter your birthdate (YYYY-MM-DD)",
font=("Arial",13,"bold"))
label.pack()

# Create an entry field for entering the birthdate
entry = tk.Entry(root)
entry.pack()

# Create a button to trigger the age calculation
calculate_button = tk.Button(root, text="Calculate Age", font=
("Arial",13,"bold"), command=calculate_age)
calculate_button.pack()

# Run the Tkinter event loop
root.mainloop()
```

Output :

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen. Here what will you find in GUI:

At the top of the window, you'll find a title stating 'Age Calculator in Years, Months, and Days'. This title clearly communicates the purpose of the application. Below the title, there's a text box provided for entering your date of birth. Directly beneath the text box, you'll see a button labeled 'Calculate Age', which initiates the age calculation process.



After clicking the 'Calculate Age' button, a message box is displayed, showing the calculated age in years, months, and days, as depicted above.

Stopwatch and Date Display

Project #31

Design a Python GUI for stopwatch and date display application using tkinter.

Introduction:

Welcome to the Simple GUI Application! This application serves two useful purposes: a Stopwatch and a Date Display. Created using Python and Tkinter, it provides a user-friendly interface for managing time and staying updated on the current date.
Stopwatch: You can use the Stopwatch to measure time. Click "Start" to begin timing, "Stop" to pause it, and "Reset" to start over.
Date Display: The Date Display shows today's date in the format "Month Day, Year." It updates every day automatically.

Solution:

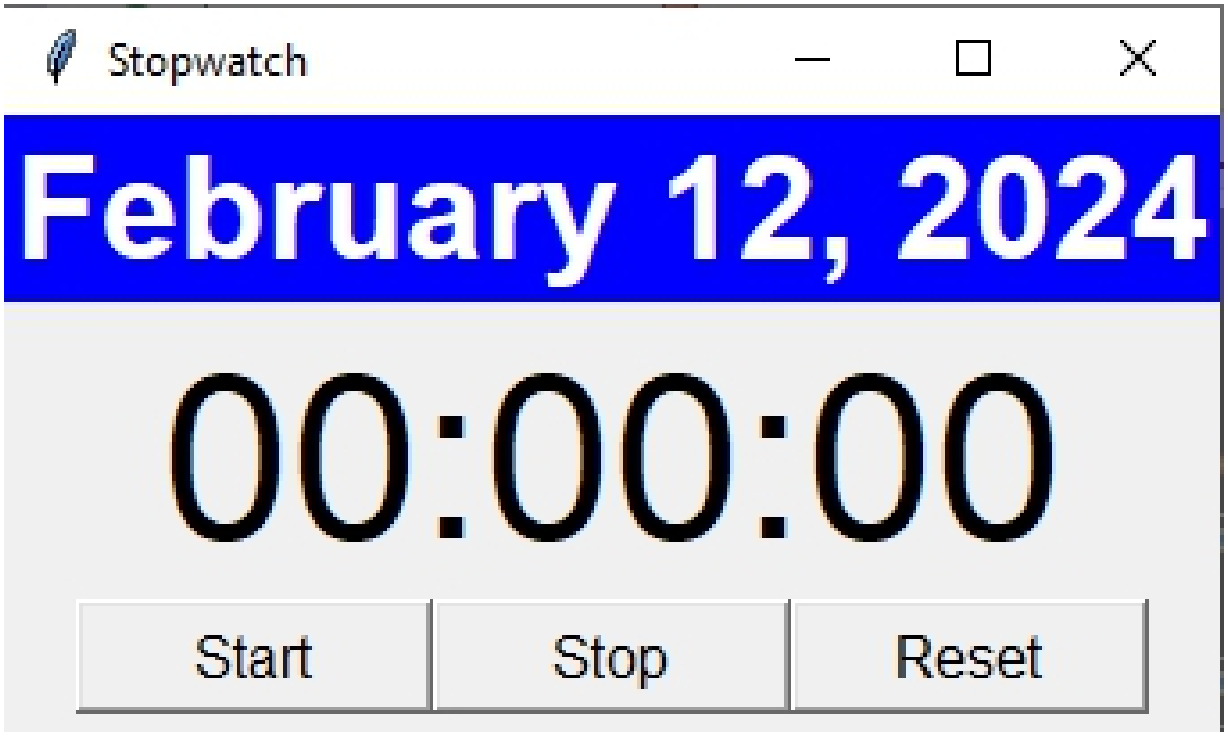
```
import tkinter as tk
from datetime import datetime

def start_stop():
    global is_running, start_time
    is_running = not is_running
    if is_running:
        start_time = datetime.now()
        update_time()

def stop():
    global is_running
    is_running = False

def reset():
    global elapsed_time
    elapsed_time = 0
    time_label.config(text="00:00:00")

def update_time():
    global elapsed_time
```

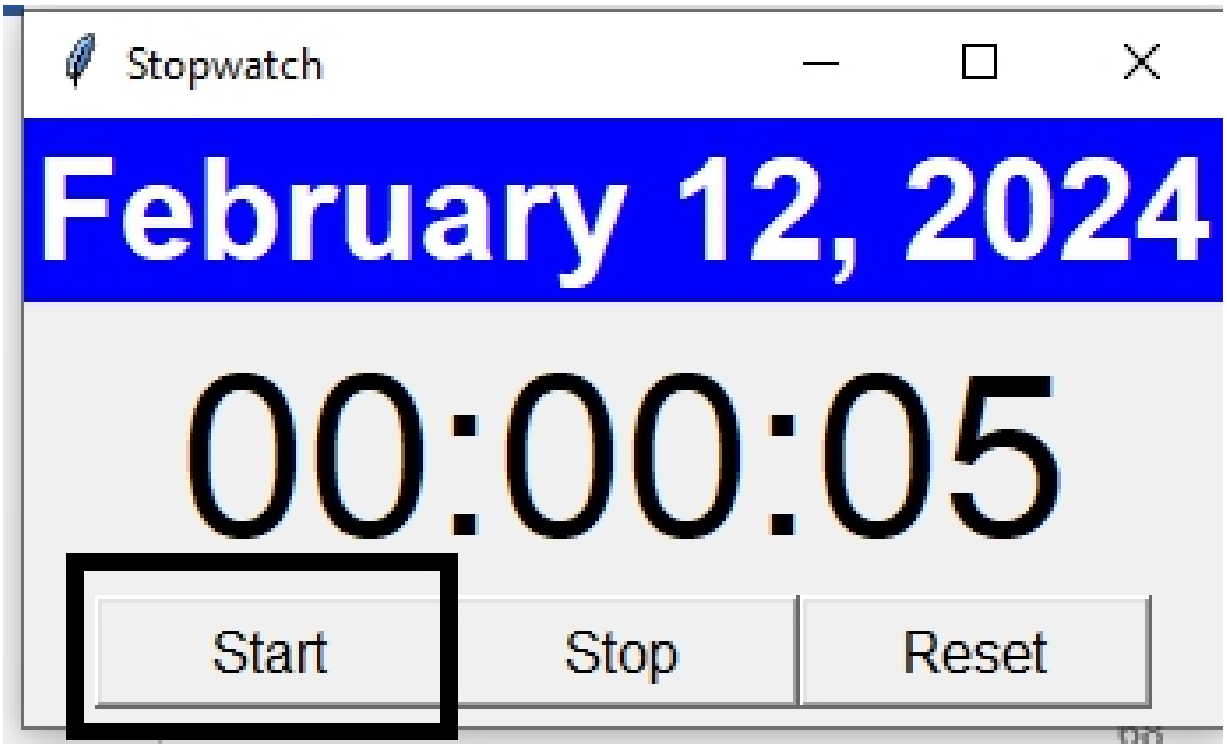
```
    if is_running:
        elapsed_time = (datetime.now() - start_time).total_seconds()
        hours = int(elapsed_time // 3600)
        minutes = int((elapsed_time % 3600) // 60)
        seconds = int(elapsed_time % 60)
        time_string = "{:02d}:{:02d}:{:02d}".format(hours, minutes,
seconds)
        time_label.config(text=time_string)
        root.after(1000, update_time)

def display_date():
    date_time = datetime.now()
    date_string = date_time.strftime("%B %d, %Y")
    date_label.config(text=date_string)


root = tk.Tk()
root.title("Stopwatch")

date_label = tk.Label(root, text="", font=("Helvetica", 30, "bold"),
fg="white", bg="blue")
date_label.pack()

display_date()

is_running = False
elapsed_time = 0

button_frame = tk.Frame(root)
time_label = tk.Label(button_frame, text="00:00:00", font=
("Helvetica", 48))
time_label.pack()

start_button = tk.Button(button_frame, text="Start", width=10,
command=start_stop, font=30)
start_button.pack(side=tk.LEFT)

stop_button = tk.Button(button_frame, text="Stop", width=10,
command=stop, font=30)
stop_button.pack(side=tk.LEFT)
```

```
reset_button = tk.Button(button_frame, text="Reset", width=10,
command=reset, font=30)
reset_button.pack(side=tk.LEFT)

button_frame.pack(anchor="center", pady=5)
root.mainloop()
```

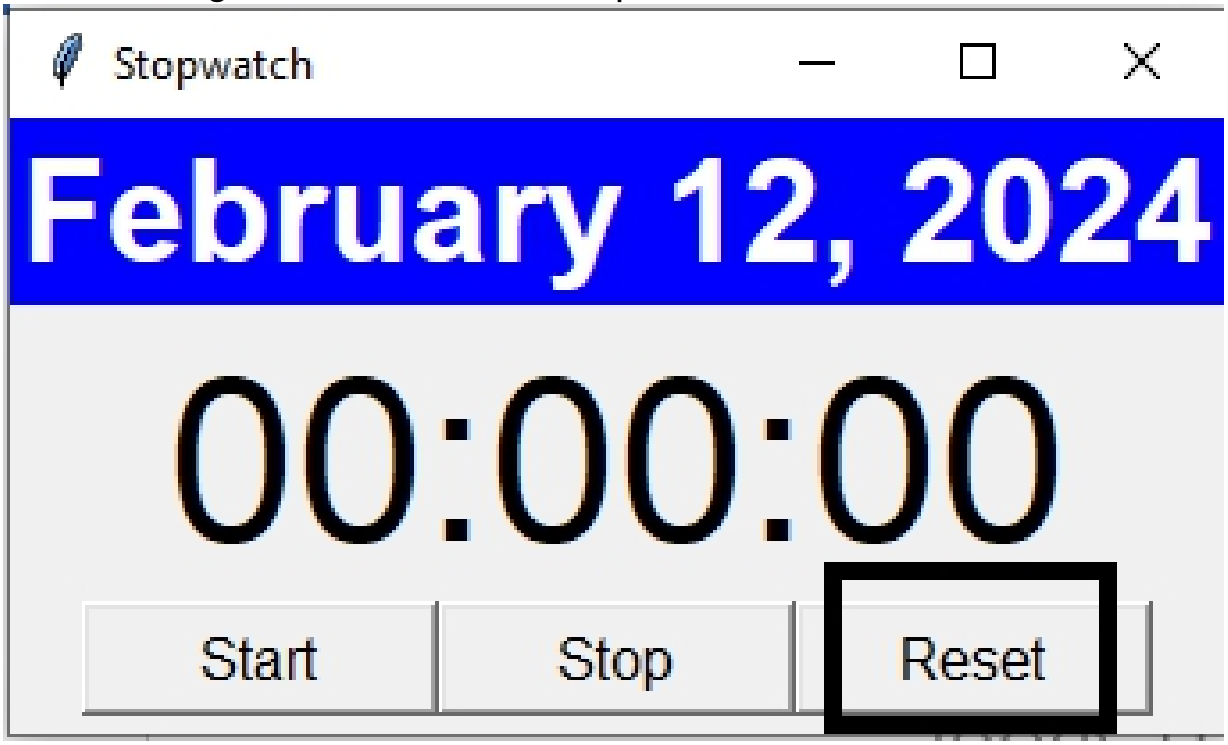
Output :
After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen. Here what will you see in GUI:

At the top of the window, there will be a title that reads "Stopwatch". This title indicates the purpose of the application. You will see current date which will update after 24 hours and three buttons: stat,stop and reset on application window.
Initially, the stopwatch is in its default state.





After clicking the “Start button”, stopwatch is started as above.



After selecting the 'reset' button, the stopwatch is reset to its initial state, as shown above.

Drag and Drop Application

Project #32

Design and implement a Python Tkinter application that allows users to drag and drop objects within a canvas.

Introduction:

Welcome to the Drag and Drop Canvas App! This easy-to-use tool lets you move things around on a canvas by clicking and dragging them. Whether you're doodling, designing, or just playing around, our app makes it simple to move objects where you want them. With the Drag and Drop Canvas App, you can click on an object, drag it around the canvas, and drop it where you like. It's like moving things around on a piece of paper, but on your computer screen!

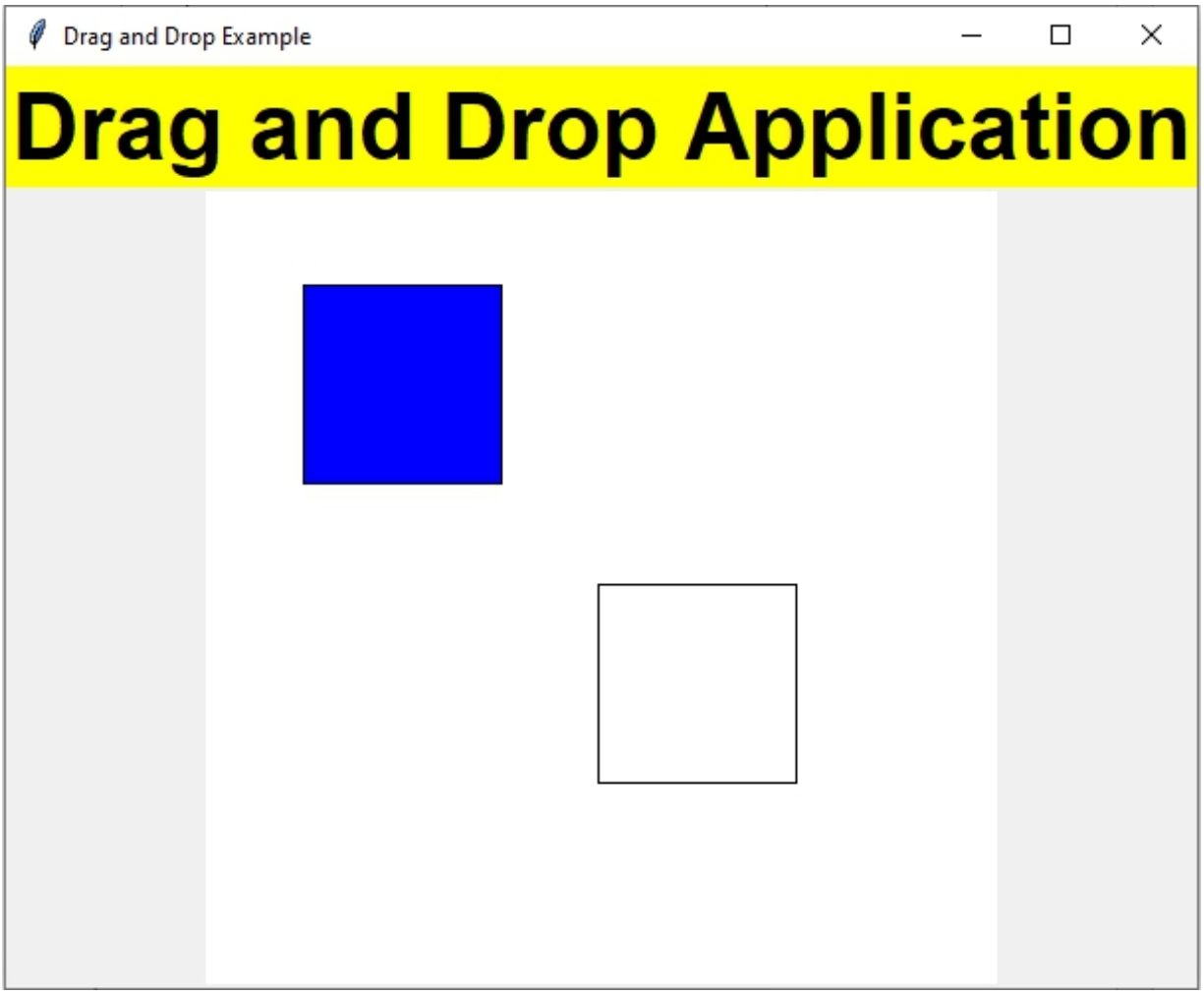
Solution:

```
import tkinter as tk
# Function to handle the start of dragging
def on_drag_start(event):
    global drag_data
    # Find the closest item to the mouse click position
    drag_data["item"] = canvas.find_closest(event.x, event.y)[0]
    # Record the initial mouse position
    drag_data["x"] = event.x
    drag_data["y"] = event.y

# Function to handle dragging motion
def on_drag_motion(event):
    global drag_data
    # Calculate the movement offset
    delta_x = event.x - drag_data["x"]
    delta_y = event.y - drag_data["y"]
    # Move the dragged item by the offset
    canvas.move(drag_data["item"], delta_x, delta_y)
    # Update the initial mouse position
    drag_data["x"] = event.x
    drag_data["y"] = event.y
```

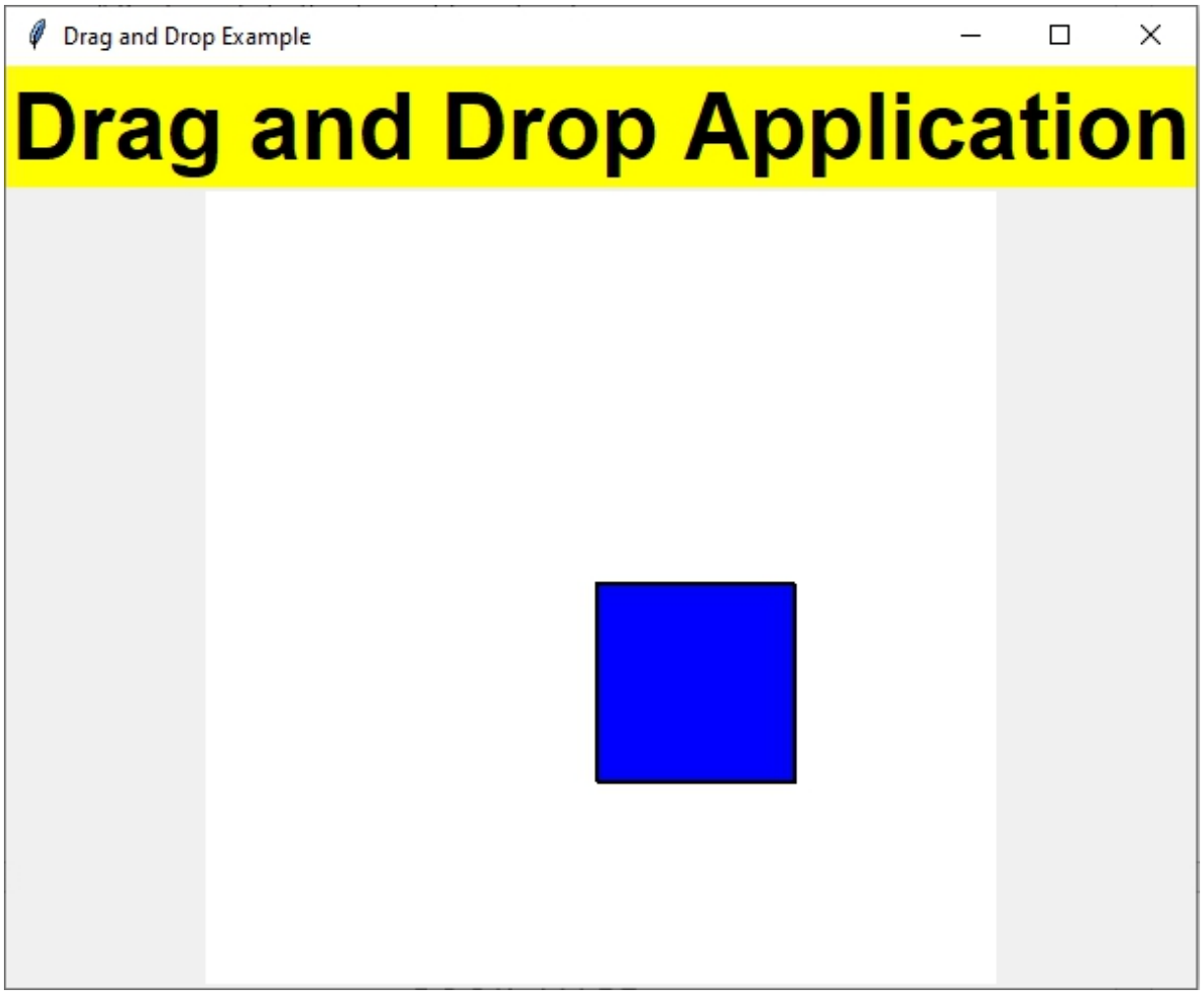
```
# Function to handle the end of dragging
def on_drag_end(event):
    # Reset the dragged item
    drag_data["item"] = None

# Create the Tkinter window
root = tk.Tk()
root.title("Drag and Drop Example")

# Create a Label for title
title_label = tk.Label(root, text="Drag and Drop Application", font=
("Helvetica",35,"bold"), bg= "yellow")
title_label.pack()

# Create a canvas widget
canvas = tk.Canvas(root, width=400, height=400, bg="white")
canvas.pack()

# Create a rectangle object to drag
rect = canvas.create_rectangle(50, 50, 150, 150, fill="blue",
tags="draggable")

# Bind events to the draggable rectangle
canvas.tag_bind("draggable", "<Button-1>", on_drag_start)
canvas.tag_bind("draggable", "<B1-Motion>", on_drag_motion)
canvas.tag_bind("draggable", "<ButtonRelease-1>", on_drag_end)

# Create a box where the rectangle can be dropped
box = canvas.create_rectangle(200, 200, 300, 300, outline="black")

# Initialize drag data
drag_data = {"x": 0, "y": 0, "item": None}

# Start the Tkinter event loop
root.mainloop()
```

Output :

After running the provided code, you'll see a graphical user interface (GUI) window displayed on your screen.



The application window displays a rectangle object. You can drag this rectangle and place it inside another rectangle.



After dragging and placing the blue rectangle onto the white rectangle, the application will appear as shown above.

Traffic Signal Simulator

Project #33

Design and implement a Python Tkinter application for traffic signal simulator

Introduction:

Welcome to the Traffic Signal Simulator! This simple application recreates the experience of a traffic signal using a canvas. With just a click of a button, you can start the simulation and watch as the traffic lights change colors. The simulator consists of three ovals representing the traffic lights: red, green, and orange. Initially, all lights are turned off. When you click the "Start" button, the simulation begins, and the lights start to turn on and off in a specific sequence. As the simulation runs, you'll see the traffic lights transition from red to green to orange, mimicking the typical traffic signal pattern. This provides a visual representation of how traffic signals work and helps users understand the sequence of colors used in real-life traffic lights.

Solution

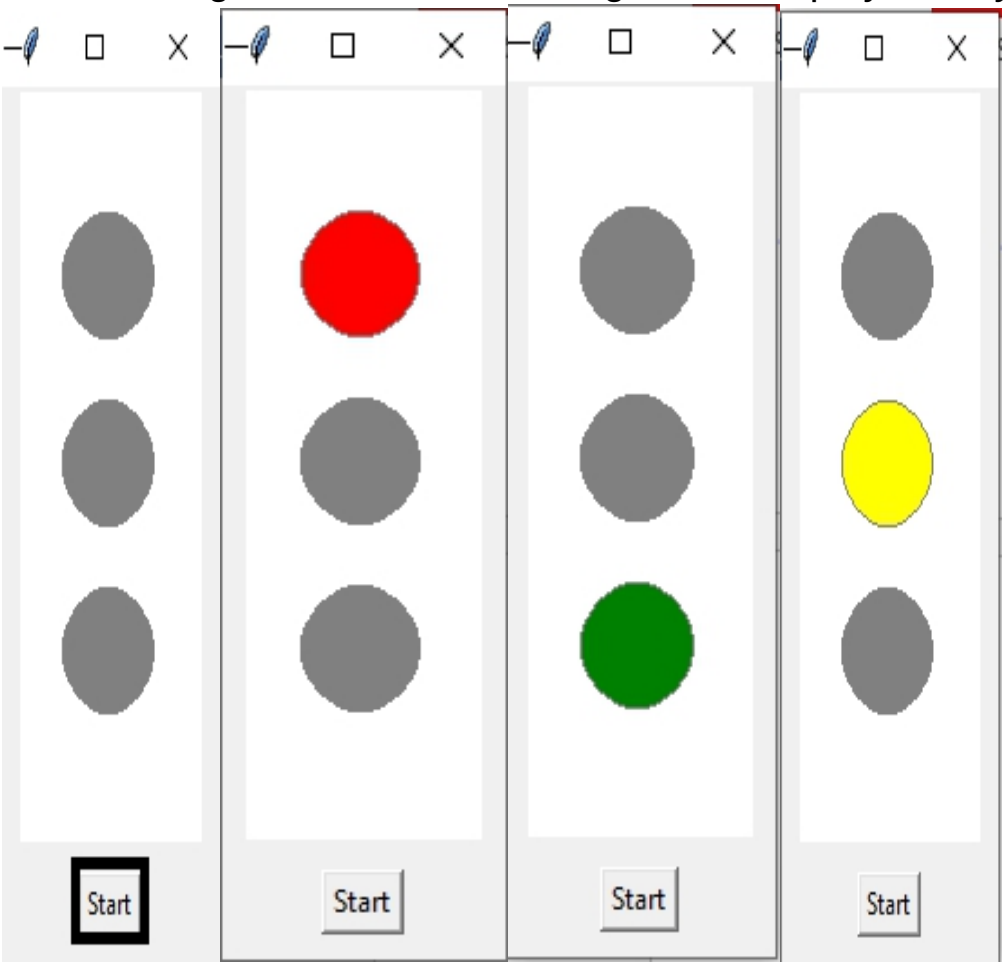
```
import tkinter as tk

def setup_traffic_signal(canvas):
    # Red light
    red_light = canvas.create_oval(25, 50, 75, 100, fill="gray", outline="gray")

    # Yellow light
    yellow_light = canvas.create_oval(25, 125, 75, 175, fill="gray", outline="gray")

    # Green light
    green_light = canvas.create_oval(25, 200, 75, 250, fill="gray", outline="gray")

    return red_light, yellow_light, green_light
```

```
def start_simulation(canvas, lights):
    # Define the function to transition to the next state
    def next_state():
        change_light(canvas, lights, "red")
        canvas.after(2000, lambda: change_light(canvas, lights,
"yellow"))
        canvas.after(3000, lambda: change_light(canvas, lights,
"green"))
        canvas.after(5000, reset_lights)

    # Define the function to reset the lights and restart the simulation
    def reset_lights():
        change_light(canvas, lights, "red")
        canvas.after(2000, lambda: change_light(canvas, lights,
"yellow"))
        canvas.after(3000, lambda: change_light(canvas, lights,
"green"))
        canvas.after(5000, next_state)

    # Start the simulation by transitioning to the first state
    next_state()

def change_light(canvas, lights, color):
    # Turn off all lights
    for light in lights.values():
        canvas.itemconfig(light, fill="gray")
    # Turn on the specified light
    canvas.itemconfig(lights[color], fill=color)

def main():
    # Create the main Tkinter window
    root = tk.Tk()
    root.title("Traffic Signal Simulator")
    # Create the canvas to display the traffic signal lights
    canvas = tk.Canvas(root, width=100, height=300, bg="white")
    canvas.pack()
    # Set up the traffic signal lights on the canvas
    lights = {"red": None, "yellow": None, "green": None}
```

```
    lights["red"], lights["yellow"], lights["green"] =
setup_traffic_signal(canvas)
    # Create the "Start" button to initiate the simulation
    start_button = tk.Button(root, text="Start", command=lambda:
start_simulation(canvas, lights))
    start_button.pack(pady=10)
    # Start the Tkinter event loop
    root.mainloop()
# Check if the script is executed directly
if __name__ == "__main__":
    main()
```

Output :

After running above code, following GUI is displayed on your screen.



After clicking the 'start' button, the traffic signal light transitions occur as depicted above.

Secure Password Validator

Project #34

Design and implement a Python Tkinter GUI for secure password validator for your account

Introduction:

Welcome to the Secure Password Validator! This handy tool helps you create strong and secure passwords for your accounts. With cyber threats on the rise, it's more important to protect your personal information online. The Secure Password Validator ensures that your passwords meet the necessary criteria to withstand hacking attempts and keep your accounts safe. The Password Validator program checks the validity of a password based on the following criteria:
Length: Password length should be between 6 and 12 characters.
Characters: Password must contain at least one lowercase letter (a-z), one uppercase letter (A-Z), one digit (0-9), and one special character from the set [$, #, @].
Confirmation: The program also allows you to confirm your password to ensure accuracy during input.

To get started, simply enter your desired password in the designated field and click the "Validate Password" button. If your password meets the specified criteria, you will receive a confirmation message. Otherwise, the program will provide feedback on what changes are needed to make your password compliant.

Solution:

```
import tkinter as tk
from tkinter import messagebox

# Define global variables
confirm_password_entry = None

def check_password_validity(password):
    error_messages = []
```

```
    if len(password) < 6 or len(password) > 12:
        error_messages.append("Password length should be between
6 and 12 characters")
    if not any(char.isdigit() for char in password):
        error_messages.append("Password should contain at least 1
digit (0-9)")
    if not any(char.islower() for char in password):
        error_messages.append("Password should contain at least 1
lowercase letter (a-z)")
    if not any(char.isupper() for char in password):
        error_messages.append("Password should contain at least 1
uppercase letter (A-Z)")
    if not any(char in ['$','#','@'] for char in password):
        error_messages.append("Password should contain at least 1
of the following characters: $, #, @")
    return error_messages

def validate_password():
    global confirm_password_entry
    # Get password from user
    password = password_entry.get()
    # Call check_password_validity function
    error_messages = check_password_validity(password)
    if error_messages:
        # Display error messages in red color
        error_label.config(text="\n".join(error_messages), fg="red")
    else:
        # Clear error messages if password is valid
        error_label.config(text="")
        # Proceed with confirming the password
        confirm_password_label = tk.Label(root, text="Confirm
Password:",font=("Arial",13,"bold"), bg="white")
        confirm_password_label.grid(row=6, column=0)
        confirm_password_entry = tk.Entry(root, show="*")
        confirm_password_entry.grid(row=6, column=1)
        button_validate.config(state="disabled")
        validate_confirm_button = tk.Button(root, text="Validate
Confirm Password",width=25, height=1, bg="blue", fg="white",
```

```
                                        font=("Arial", 15,
"bold"),command=validate_confirm_password)
        validate_confirm_button.grid(row=8, column=0,columnspan=3,
padx=5, pady=5)

def validate_confirm_password():
    global confirm_password_entry
    # Get confirm password from user
    confirm_password = confirm_password_entry.get()
    password = password_entry.get()
    if password != confirm_password:
        error_label.config(text="Passwords do not match",    fg="red")
    else:
        error_label.config(text="Passwords match", fg="green")

# Create main window
root = tk.Tk()
root.title("Password Validator")
font_bold = ("Helvetica", 20, "bold")
# Create label for title on window
title_label = tk.Label(root, text="Secure Password Validator",
fg="purple", font=font_bold)
title_label.grid(row=1, columnspan=3, padx=5, pady=5)
# Create label for mail
mail_label = tk.Label(root, text="New Yahoo email", font=
("Arial",13,"bold"))
mail_label.grid(row=3, column=0)
# Create mail entry field
mail_entry = tk.Entry(root)
mail_entry.grid(row=3, column=1, columnspan=3)
mail_entry.config(width=40)
# Create label for password
password_label = tk.Label(root, text="Password", font=
("Arial",13,"bold"))
password_label.grid(row=4, column=0)
# Create password entry field
password_entry = tk.Entry(root, show= "*")
password_entry.grid(row=4, column=1, columnspan=3)
```

```
password_entry.config(width=40)

# Create validate button
button_validate = tk.Button(root, text="Validate Password",
bg="blue", fg="white", font=("Arial", 15, "bold"),
                            relief=tk.RAISED, bd=3, width=15, height=1,
                            command=validate_password,       cursor="h
and2")
button_validate.grid(row=5,column=0,columnspan=3,padx=5,
pady=5)

# Create label to display error messages
error_label = tk.Label(root, text="", fg="red")
error_label.grid(row=9,column=0,columnspan=3, padx=5, pady=5)

# Run the Tkinter event loop
root.mainloop()
```

Output:

After executing the provided code, the graphical user interface (GUI) will be displayed on your screen. In the GUI, you will find:

Title: At the top of the window, there will be a title stating "Secure Password Validator", indicating the purpose of the application.

Text Boxes: Below the title, there will be two text boxes: one for entering the user's email and the other for entering the password.

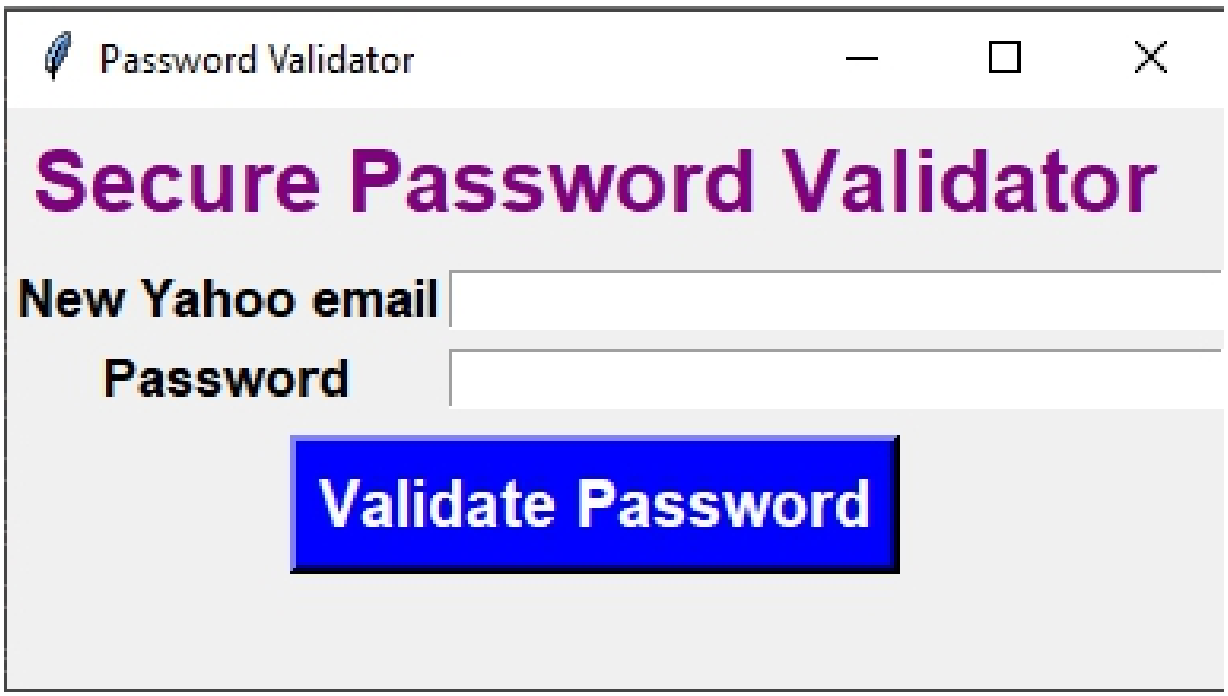
Validate Password Button: Directly below the password text box, there will be a button labeled "Validate Password". Clicking on this button triggers the validation process for the entered password.

Error Messages: If the password entered does not meet the specified conditions, error messages providing instructions for creating a valid password will be displayed below the button on the main window.

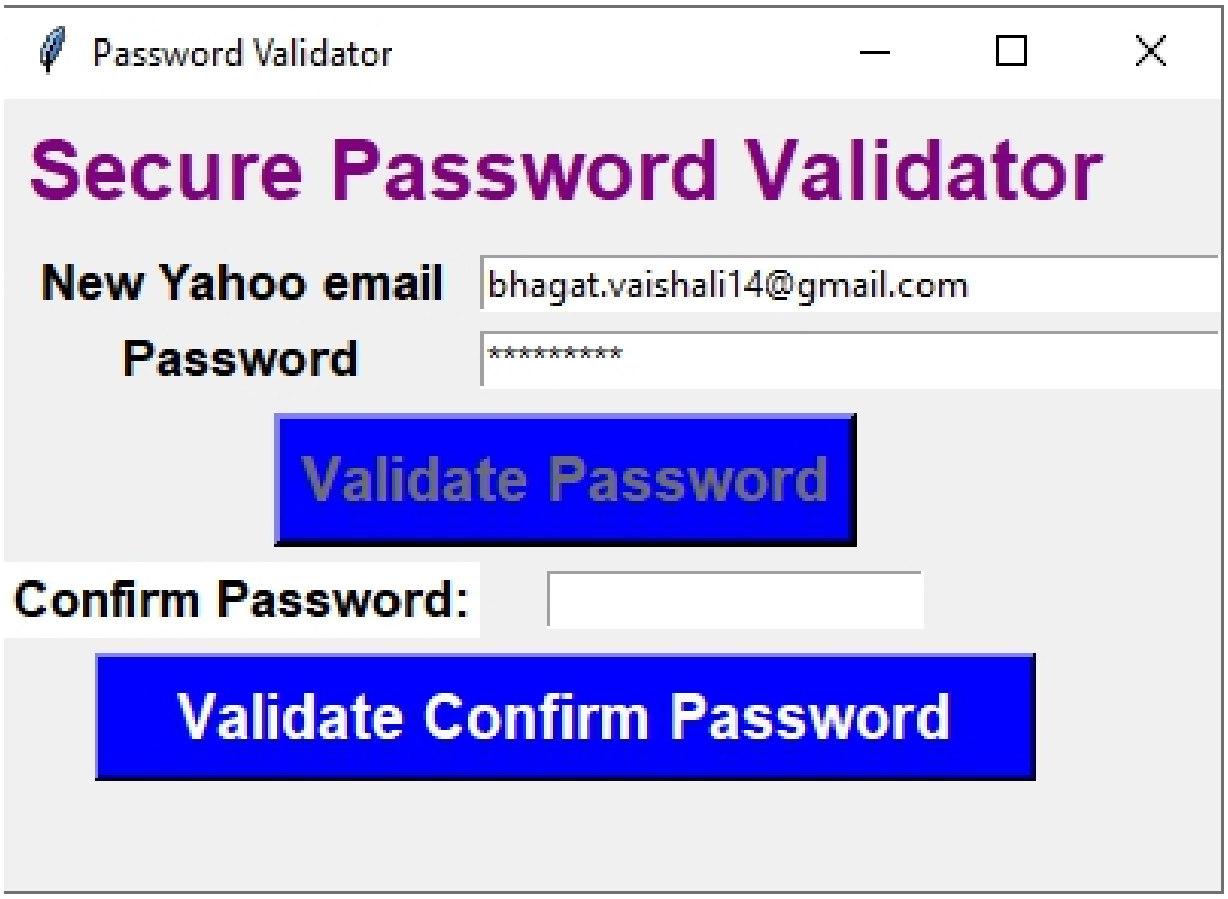
Confirm Password Label and Entry Widget: If the password meets the conditions, a "Confirm Password" label and an entry widget will automatically appear below the error messages. Users can enter their confirmation password into this entry widget.

Validate Confirm Button: Additionally, a "Validate Confirm" button will appear next to the confirmation entry widget. Clicking on this button

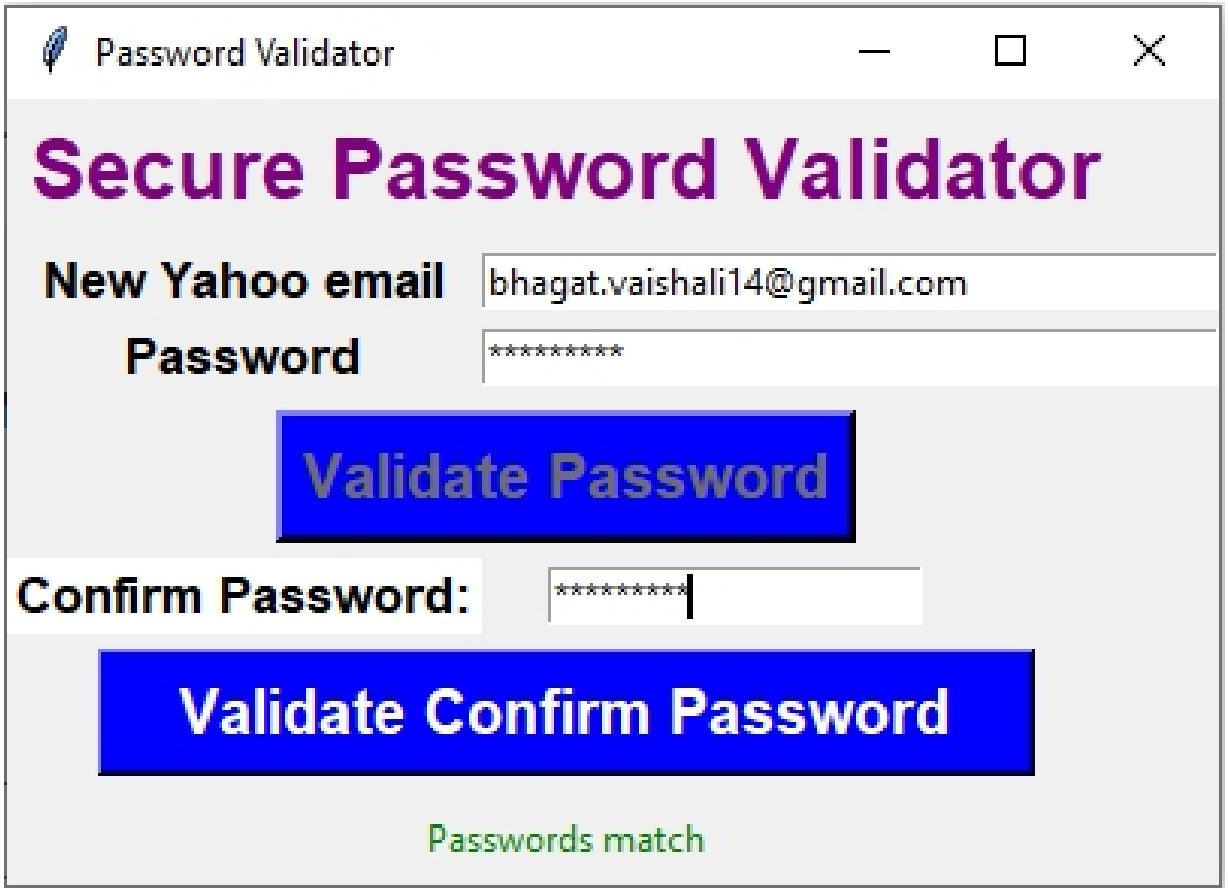
triggers the validation process for the confirmation password.
Confirmation Message: If the confirmation password matches the original password, a confirmation message will be displayed below the button, indicating that the password is secure and validated.



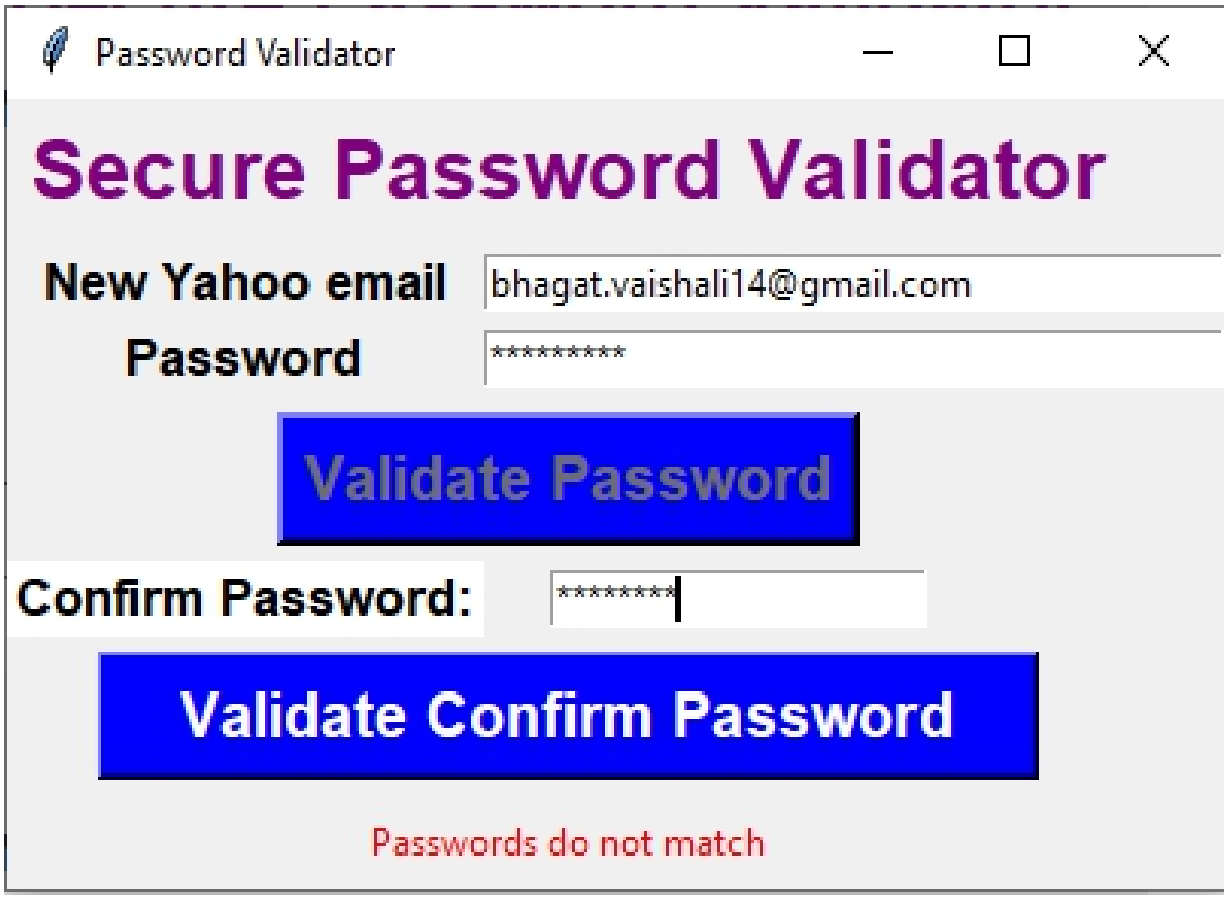
Here user entered password did not match the certain conditions, so all instructions for creating password are shown in windows as above



Here, the user entered a valid password. As a result, the 'Confirm Password' label and entry field are automatically displayed. Additionally, the 'Validate Confirm Password' button is also shown on the window as described above.



In the above output, the user enters the correct password in the confirm password entry field and clicks on the 'Validate Confirm Password' button. Consequently, the 'Password Match' label is automatically displayed.



In the above output, the user enters the incorrect password in the confirm password entry field and clicks on the 'Validate Confirm Password' button. Consequently, the 'Password do not Match' label is automatically displayed."

Email Login System with Dynamic Account Creation

Project #35

Design a python tkinter application for Email Login System

Introduction:

Welcome to our email login system, where accessing your account is made simple and secure. When you enter your email address, our system checks if it's registered. If it is, you'll be directed to the sign-in form. However, if your email isn't registered yet, you'll be prompted to create an account effortlessly. With clear options like "Next" and "Create Account" on the sign-in form and "Continue" and "Sign In" on the account creation form, navigating through our system is intuitive. Rest assured, our system verifies registered emails and provides appropriate messages every step of the way, ensuring a smooth user experience.

Solution:

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
# Declare global variables
login_window = None
signup_window = None
registered_emails = []
# Define functions
def create_account():
    global signup_window
    if signup_window is not None:
        # Get the entered email from the entry widget
        email = mail_entry.get()
        # Perform email registration
        register_user(email)
```

```
        # Close the signup window upon successful sign-in
        signup_window.destroy()
def register_user(email):
    if email in registered_emails:
        messagebox.showerror("Error", "Email already exists!")
    else:
        registered_emails.append(email)
        messagebox.showinfo("Success", "You are successfully
registered!")

def sign_in():
    global login_window
    entered_email = username_entry.get()

    # Check if the entered email exists in the list of registered emails
    if entered_email in registered_emails:
        # Perform sign-in process here
        # For demonstration purposes, let's assume sign-in was
successful
        messagebox.showinfo("Success", "You are successfully
signed in!")
        # Close the login window upon successful sign-in
        login_window.destroy()
    else:
        # Inform the user that their email is not registered
        messagebox.showinfo("Account Not Found", "Your email is not
registered. Please create an account.")
def open_signup_form():
    global signup_window, login_window
    if login_window is not None:
        login_window.destroy()
    signup_window = tk.Toplevel()
    signup_window.title("Yahoo Sign Up Form")
    italic_font = ("Helvetica", 30, "italic")
    title_label = tk.Label(signup_window, text="Yahoo!", fg="purple",
font=italic_font)
    title_label.grid(row=1, columnspan=2, padx=5, pady=5)
```

```
    header_label = tk.Label(signup_window, text="Create a Yahoo
account", font=("Arial", 15, "bold"))
    header_label.grid(row=2, columnspan=2, padx=5, pady=5)
    info_frame = tk.Frame(signup_window, bg="white", bd=5,
relief="ridge", width=200, height=100, padx=10, pady=10)
    info_frame.grid(padx=5, pady=5)
    font_bold = ("Arial", 10, "bold")
    full_name_label = tk.Label(info_frame, text="Full Name:",
font=font_bold, fg="black", bg="white")
    full_name_label.grid(row=2, column=0)
    full_name_entry = tk.Entry(info_frame)
    full_name_entry.grid(row=2, column=1, columnspan=3)
    full_name_entry.config(width=45)  # Adjust the width value as
needed
    global mail_entry
    mail_label = tk.Label(info_frame, text="New Yahoo email",
font=font_bold, bg="white")
    mail_label.grid(row=3, column=0)
    mail_entry = tk.Entry(info_frame)
    mail_entry.grid(row=3, column=1, columnspan=3)
    mail_entry.config(width=45)
    password_label = tk.Label(info_frame, text="Password",
font=font_bold, bg="white")
    password_label.grid(row=4, column=0)
    password_entry = tk.Entry(info_frame, show="*")
    password_entry.grid(row=4, column=1, columnspan=3)
    password_entry.config(width=45)
    dob_frame = tk.Frame(signup_window, bg="white", bd=5,
relief="ridge", width=200, height=100, padx=10, pady=10)
    dob_frame.grid(padx=5, pady=5)
    label_dob = tk.Label(dob_frame, text="Date of Birth",
font=font_bold, bg="white")
    label_dob.grid(row=5, column=0, padx=5, pady=5)
    label_month = tk.Label(dob_frame, text="Month ", font=font_bold,
bg="white")
    label_month.grid(row=6, column=0, padx=(5, 5), pady=5,
sticky="e")  # Adjusted padding
```

```
    months = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
'August', 'September', 'October',
              'November', 'December']
    combo_month = ttk.Combobox(dob_frame, values=months,
state="readonly")
    combo_month.current(0)  # Default selection
    combo_month.grid(row=6, column=1, padx=(0, 5), pady=5,
sticky="w")  # Adjusted padding
    combo_month.config(width=10)
    # Day entry
    label_day = tk.Label(dob_frame, text="Day ", font=font_bold,
bg="white")
    label_day.grid(row=6, column=2, padx=(0, 5), pady=5,
sticky="e")  # Adjusted padding
    entry_day = tk.Entry(dob_frame)
    entry_day.grid(row=6, column=3, padx=(0, 5), pady=5,
sticky="w")  # Adjusted padding
    entry_day.config(width=10)
    # Year entry
    label_year = tk.Label(dob_frame, text="Year ", font=font_bold,
bg="white")
    label_year.grid(row=6, column=4, padx=(5, 0), pady=5,
sticky="e")  # Adjusted padding
    entry_year = tk.Entry(dob_frame)
    entry_year.grid(row=6, column=5, padx=(0, 5), pady=5,
sticky="w")  # Adjusted padding
    entry_year.config(width=10)
    button_signup = tk.Button(signup_window, text="Continue",
bg="blue", fg="white", font=("Arial", 15, "bold"),
                              relief=tk.RAISED, bd=3, width=15, height=1,
                              command=create_account, cursor="hand2")
    button_signup.grid(padx=5, pady=5)
    label = tk.Label(signup_window, text="Already have an account
?", font=("Arial", 10, "bold"))
    label.grid(padx=5, pady=5)
    button_login = tk.Button(signup_window, text="Sign In",
bg="blue", fg="white", font=("Arial", 15, "bold"),
```

```
                        relief=tk.RAISED, bd=3, width=15, height=1,
                        command=open_login_form, cursor="hand2")
    button_login.grid(padx=5, pady=5)


def open_login_form():
    global login_window
    login_window = tk.Toplevel()
    login_window.title("Yahoo Login Form")
    italic_font = ("Helvetica", 30, "italic")
    title_label = tk.Label(login_window, text="Yahoo!", fg="purple",
font=italic_font)
    title_label.grid(row=1, columnspan=2, padx=5, pady=5)
    header_label = tk.Label(login_window, text="Sign in to Yahoo
Mail", font=("Arial", 15, "bold"))
    header_label.grid(row=2, columnspan=2, padx=5, pady=5)
    subheader_label = tk.Label(login_window, text="Sign in using
your Yahoo account", font=("Arial", 12))
    subheader_label.grid(row=3, columnspan=2, padx=5, pady=5)
    info_frame = tk.Frame(login_window, bg="white", bd=5,
relief="ridge", width=200, height=100, padx=10, pady=10)
    info_frame.grid(padx=5, pady=5)
    font_bold = ("Arial",10,"bold")
    full_name_label = tk.Label(info_frame, text="Username, email or
mobile:", font=font_bold, fg="black", bg="white")
    full_name_label.grid(row=2, column=0)
    full_name_entry = tk.Entry(info_frame)
    full_name_entry.grid(row=2, column=1, columnspan=3)
    full_name_entry.config(width=45)
    button_signup = tk.Button(login_window, text="Next", bg="blue",
fg="white", font=("Arial", 15, "bold"),
                      relief=tk.RAISED, bd=3, width=15, height=1,
                      command=sign_in, cursor="hand2")
    button_signup.grid(padx=5,pady=5)
    label = tk.Label(login_window, text="Don't have an    account ?",
font=("Arial", 10, "bold"))
    label.grid(padx=5,pady=5)
```

```
    button_login = tk.Button(login_window, text="Create an account",
bg="blue", fg="white", font=("Arial", 15, "bold"),
                    relief=tk.RAISED, bd=3, width=15, height=1,
                    command=open_signup_form, cursor="hand2")
    button_login.grid(padx=5,pady=5)
# Main window
root = tk.Tk()
root.title("Yahoo Login")
italic_font = ("Helvetica", 30, "italic")
title_label = tk.Label(root, text="Yahoo!", fg="purple", font=italic_font)
title_label.grid(row=1, columnspan=3, padx=5, pady=5)
header_label = tk.Label(root, text="Sign in to Yahoo Mail", font=
("Arial", 15, "bold"))
header_label.grid(row=2, columnspan=3, padx=5, pady=5)
subheader_label = tk.Label(root, text="Sign in using your Yahoo
account", font=("Arial", 12))
subheader_label.grid(row=3, columnspan=3, padx=5, pady=5)
font_bold = ("Arial", 10, "bold")
username_label = tk.Label(root, text="Username, email, or mobile:",
font=font_bold, fg="black")
username_label.grid(row=4, column=0, columnspan=3)
username_entry = tk.Entry(root)
username_entry.grid(row=5, column=0, columnspan=3)
username_entry.config(width=45)
button_login = tk.Button(root, text="Next", bg="blue", fg="white",
font=("Arial", 15, "bold"),
                    relief=tk.RAISED, bd=3, width=15, height=1,
                    command=sign_in, cursor="hand2")
button_login.grid(row=6, column=0, columnspan=3, padx=5,
pady=5)
label = tk.Label(root, text="Don't have an account?", font=("Arial",
10, "bold"))
label.grid(row=7, column=0, columnspan=3, padx=5, pady=5)
button_signup = tk.Button(root, text="Create an account", bg="blue",
fg="white", font=("Arial", 15, "bold"),
                    relief=tk.RAISED, bd=3, width=15, height=1,
                    command=open_signup_form, cursor="hand2")
```

```
button_signup.grid(row=8, column=0, columnspan=3, padx=5,
pady=5)
root.mainloop()
```

Output:

Upon running the provided code for the Email Login System with Dynamic Account Creation, you'll encounter a Graphical User Interface (GUI) window designed to facilitate user authentication and account creation. Here's what you can expect to see:

Main Window:

The main window includes input fields where users can enter their username, email, or mobile number for authentication.

If the user is already registered, a "Next" button is provided, allowing them to proceed with the login process.

If the user is not registered, a "Create an Account" button is provided.

Login Process:

If the user is registered and clicks the "Next" button, the system verifies their credentials and grants access to their account.

If the user is not registered and clicks the "Create an Account" button, a message box pops up, informing them that their email is not registered and prompting them to create an account.

Account Creation Window:

Upon clicking the "Create an Account" button, another window opens, displaying a form where users can enter their full name, new Yahoo email, password, and date of birth.

A "Continue" button is provided to submit the form and proceed with account creation.

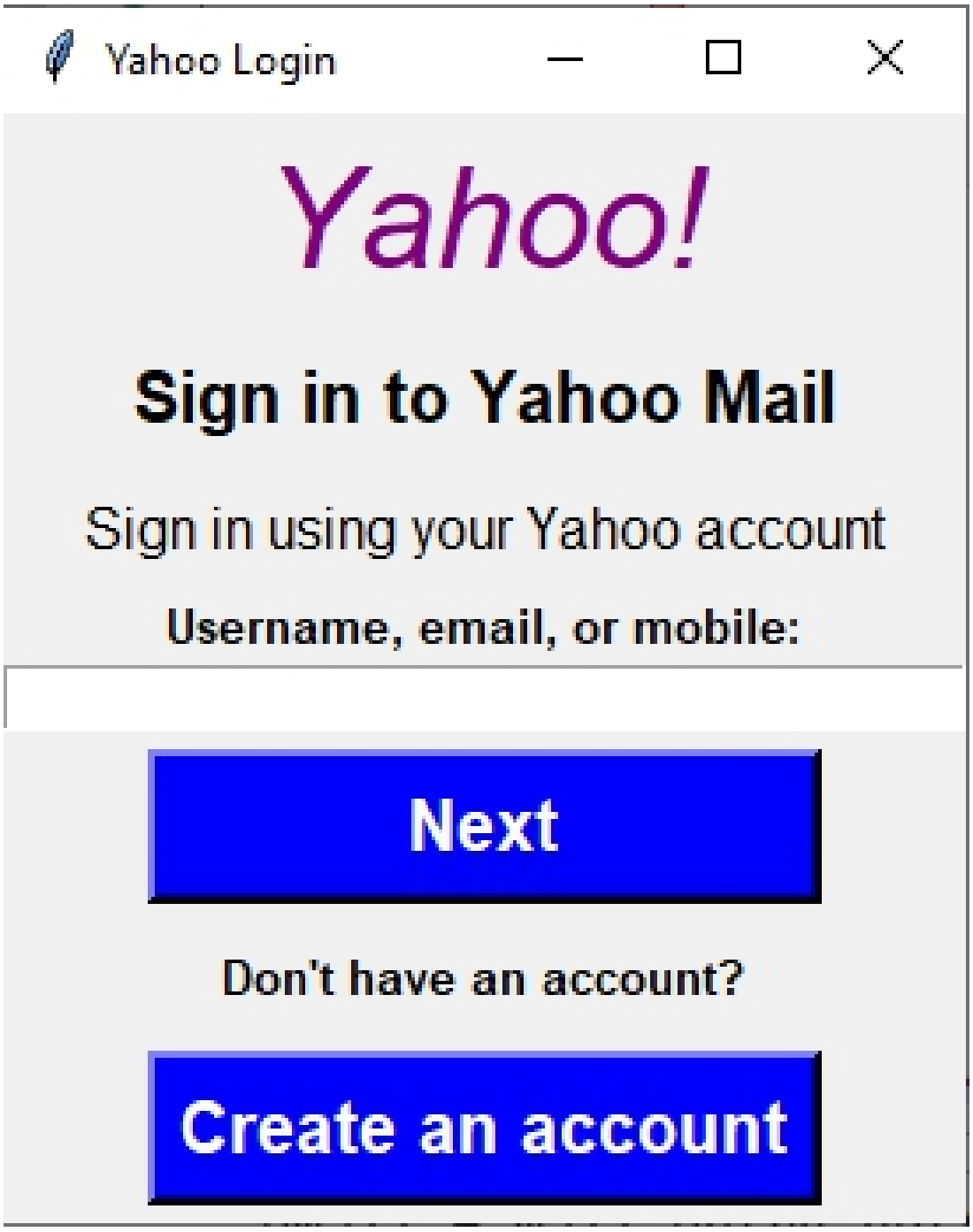
Successful Registration:

After clicking the "Continue" button, a message pops up, confirming that the user has been registered successfully.

Sign In Process:

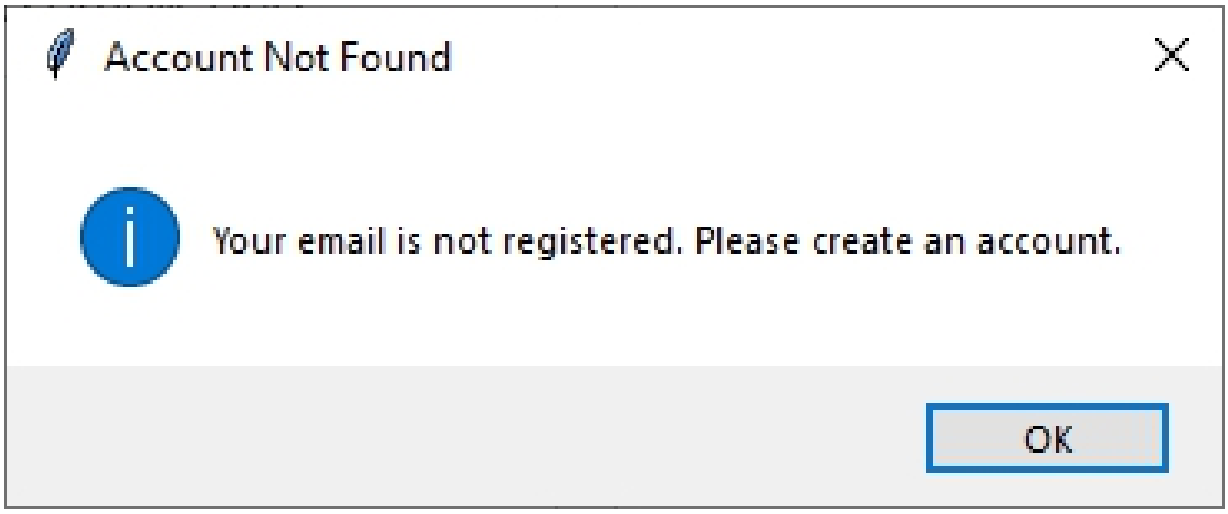
In the account creation form, a "Sign In" button is available.

When the user enters their new email and clicks the "Sign In" button, a message appears, indicating that they have successfully signed in.

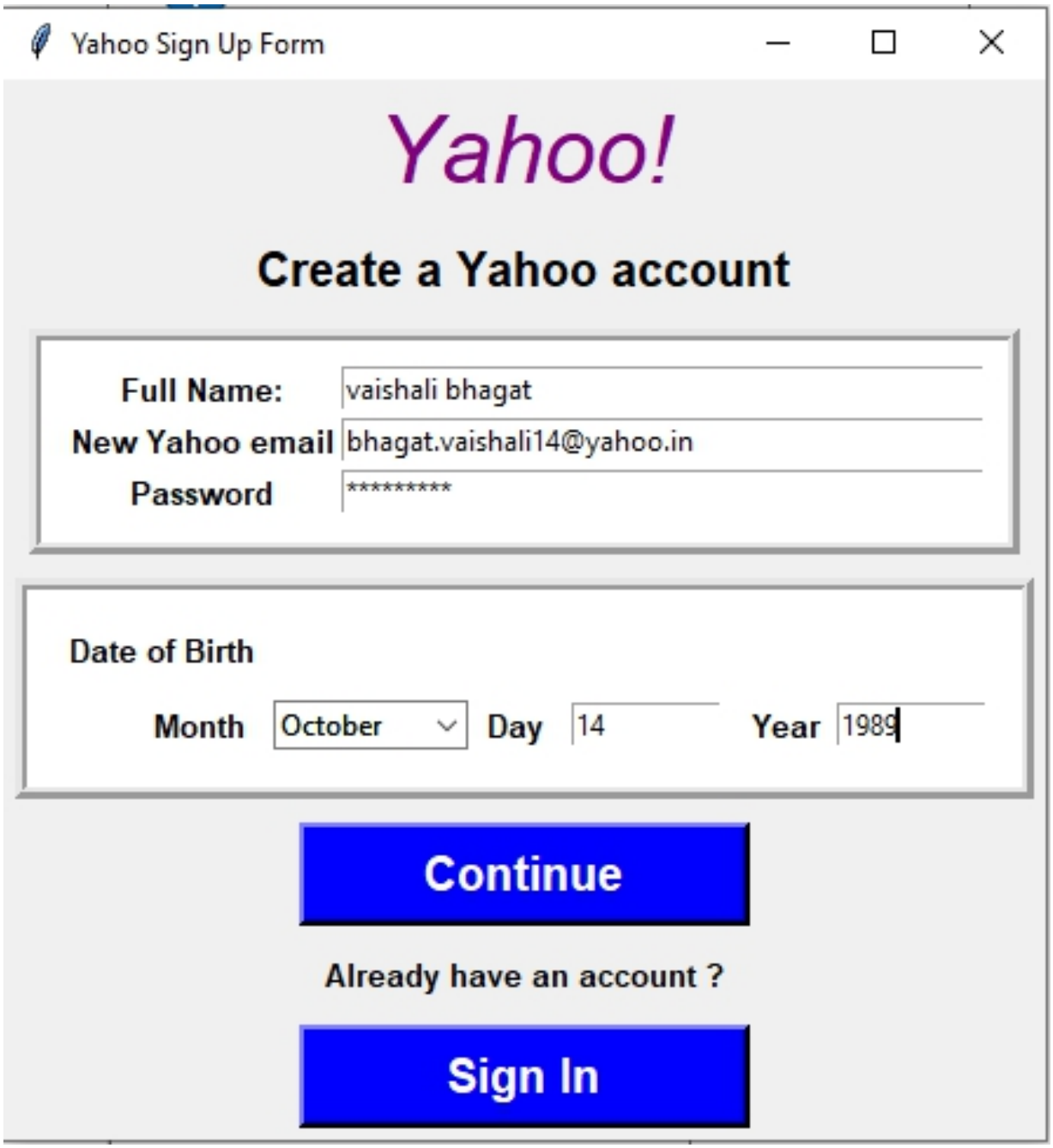


Main window of Email Login system

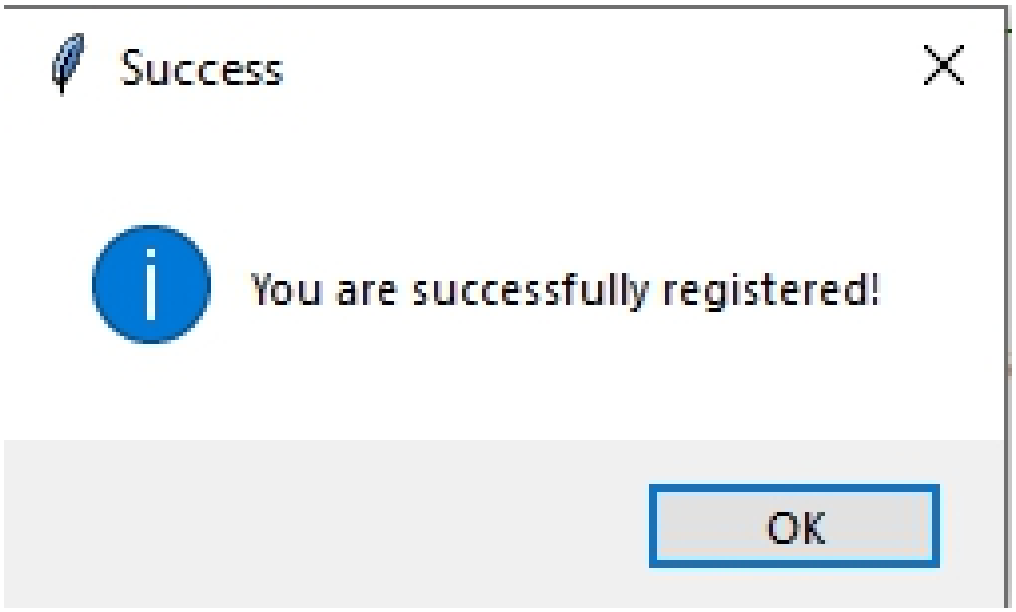
Above window shows login process



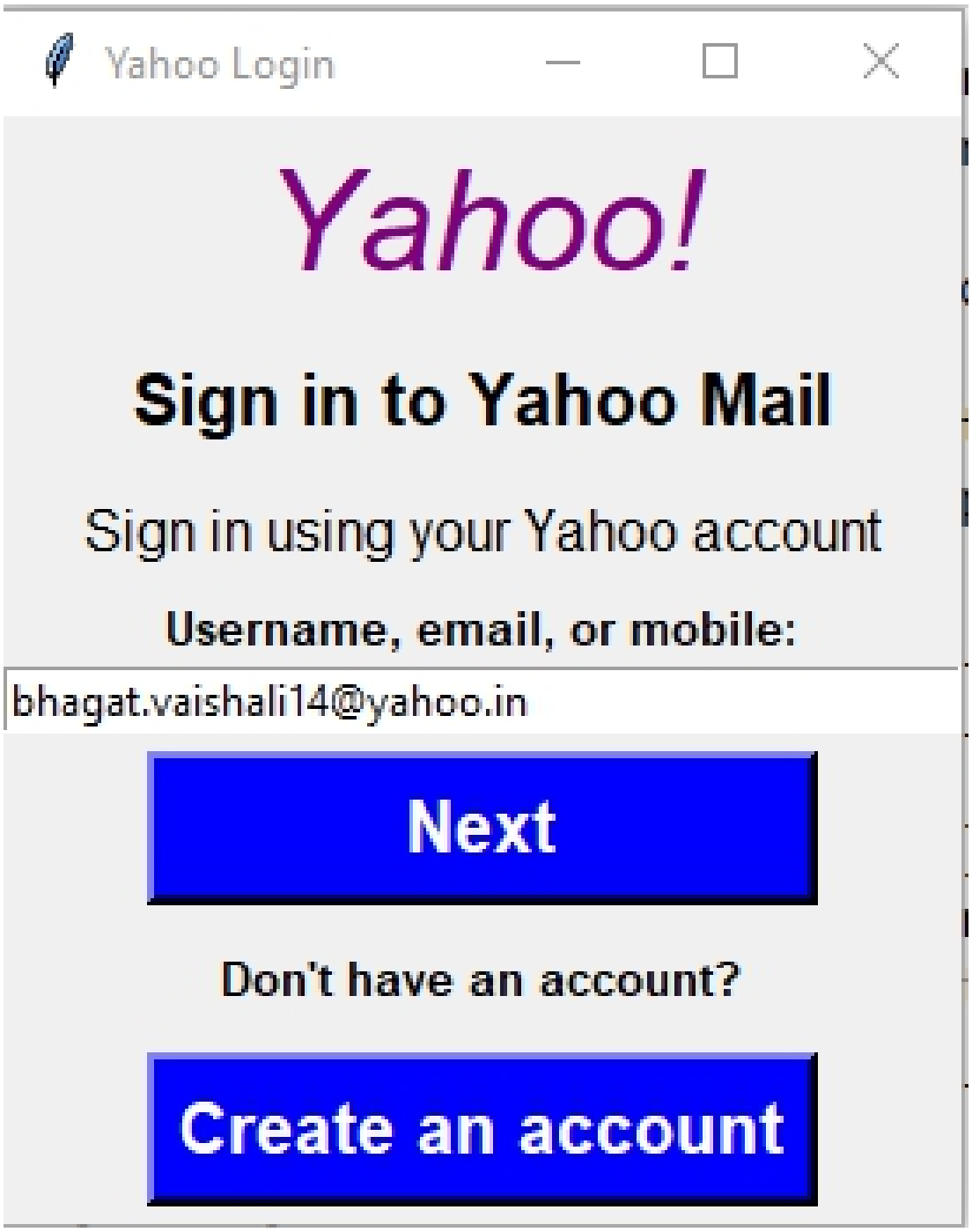
Above messagebox is displayed when email id is not registered.



Above window is Account creation window



Above messagebox is displayed when account is successfully created.



After account creation, above login window is appeared on your screen where you can enter your emailed.

Above Messagebox is displayed when user enter registered email id in login form.
